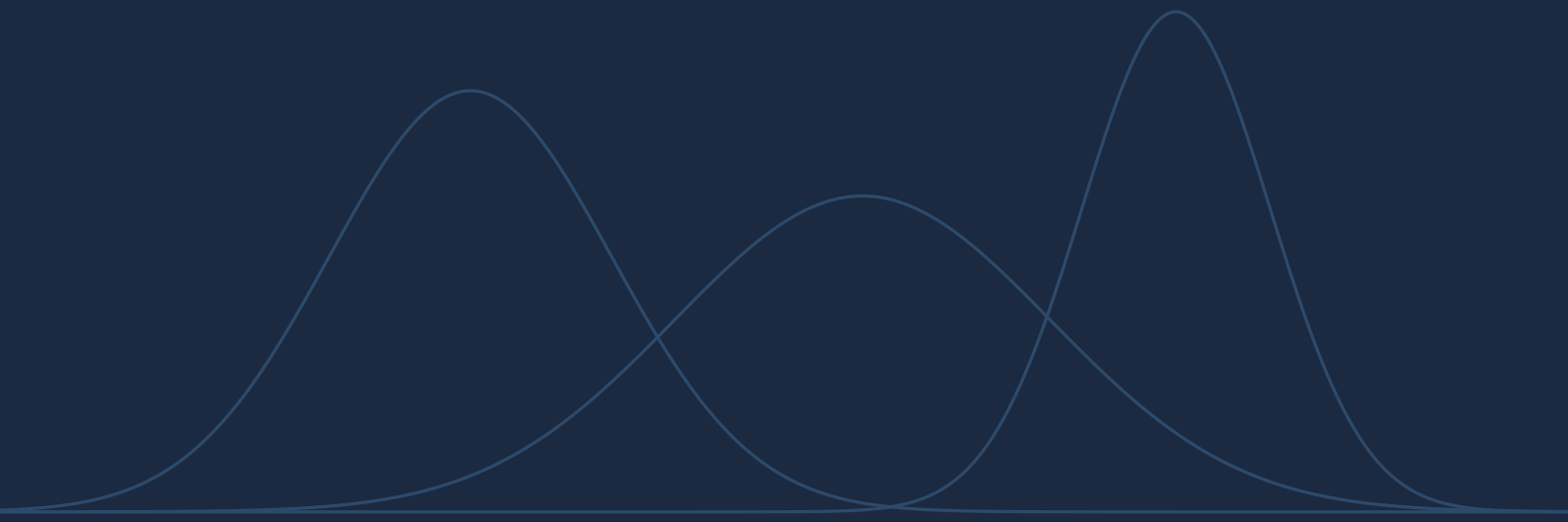


Statistics & Statistical Modelling

with Python

A complete MSc reference: theory, formulas, executed Jupyter code and interpretation

Probability • Inference • Estimation • Hypothesis testing • ANOVA
Simple & multiple regression • Time series & forecasting • Factor & cluster analysis
Maximum likelihood • Logistic regression • Bayesian inference

NumPy • pandas • SciPy • statsmodels • scikit-learn • matplotlib • PyMC

Edition: October 2026

Contents

How to Use This Guide 7

How the code in this PDF was produced 7

Setting up your environment 7

The Python statistics toolkit: what each library does 8

The data sets used in this guide 9

1. Descriptive Statistics and Exploratory Data Analysis 12

1.1 Types of variables 12

1.2 Measures of location 13

1.3 Measures of spread 13

1.4 Shape: skewness and kurtosis 14

1.5 The five-number summary and describe() 15

1.6 Summaries by group 15

1.7 Association: covariance and correlation 16

1.8 Visual EDA 17

1.9 Outliers and missing values 18

1.10 Standardisation and transformations 19

Chapter summary 20

2. Basic Probability 21

2.1 Sample spaces, events and the axioms 21

2.2 Simulation: the frequentist definition in code 21

2.3 Counting: permutations and combinations 22

2.4 Conditional probability and independence 23

2.5 Law of total probability and Bayes' theorem 23

2.6 Random variables, expectation and variance 24

2.7 Joint, marginal and conditional distributions 25

Chapter summary 25

3. Probability Distributions with scipy.stats 26

3.1 The scipy.stats interface 26

3.2 Discrete distributions 27

3.3 Continuous distributions 28

3.4 Relationships between distributions 30

3.5 Fitting a distribution to data 31

3.6 The multivariate normal 32

Chapter summary 33

4. Introduction to Statistical Inference 34

4.1 Population, sample, parameter, statistic 34

4.2 Seeing a sampling distribution 34

4.3 The law of large numbers . . . 35
4.4 The central limit theorem . . . 36
4.5 Standard error and the t statistic . . . 36
4.6 The bootstrap: a sampling distribution from one sample . . . 37
4.7 Frequentist and Bayesian inference: a first look . . . 38
Chapter summary . . . 38

5. Estimation: Point Estimates and Confidence Intervals . . . 39

5.1 Properties of estimators . . . 39
5.2 Method of moments . . . 40
5.3 Confidence intervals: the idea . . . 41
5.4 Confidence interval for a mean . . . 41
5.5 Confidence interval for a proportion . . . 42
5.6 Confidence interval for a variance and SD . . . 43
5.7 Confidence interval for a difference in means . . . 43
5.8 Bootstrap confidence intervals . . . 44
5.9 Sample size planning . . . 44
Chapter summary . . . 45

6. Hypothesis Testing . . . 46

6.1 The framework . . . 46
6.2 A p-value from first principles . . . 46
6.3 One-sample t test . . . 47
6.4 Two-sample t tests . . . 48
6.5 Tests for proportions and categorical data . . . 49
6.6 Checking assumptions: normality and equal variances . . . 50
6.7 Nonparametric tests . . . 51
6.8 Power and sample size . . . 52
6.9 Multiple testing . . . 53
6.10 Reporting and interpreting tests well . . . 54
Chapter summary . . . 54

7. Analysis of Variance (ANOVA) . . . 56

7.1 The idea: partitioning variability . . . 56
7.2 One-way ANOVA from scratch and with SciPy . . . 56
7.3 Assumptions and diagnostics . . . 58
7.4 Nonparametric alternative: Kruskal-Wallis . . . 59
7.5 Post-hoc comparisons . . . 59
7.6 Two-way ANOVA and interactions . . . 60
7.7 ANCOVA: adjusting for a covariate . . . 63
7.8 Repeated measures . . . 63
Chapter summary . . . 64

8. Simple Linear Regression 65

8.1 The model and its assumptions 65

8.2 Least squares estimation 65

8.3 Inference for the coefficients 66

8.4 Goodness of fit: R^2 and the ANOVA decomposition 67

8.5 Confidence and prediction intervals 67

8.6 Regression diagnostics 68

8.7 Influence in action 70

8.8 Transformations and interpreting log models 70

8.9 Regression with statsmodels 71

8.10 Correlation, causation and regression to the mean 71

Chapter summary 72

9. Multiple Linear Regression 73

9.1 The model in matrix form 73

9.2 An OLS engine from scratch 73

9.3 'Holding other variables constant' and omitted variable bias 75

9.4 Testing groups of coefficients: the nested F test 76

9.5 Interactions 76

9.6 Polynomial terms for curvature 77

9.7 Multicollinearity 77

9.8 Diagnostics and robust standard errors 78

9.9 Standardised coefficients 79

9.10 Model selection 79

9.11 Regularisation: ridge, lasso and elastic net 80

9.12 A predictive workflow with a held-out test set 82

9.13 The statsmodels formula interface 82

Chapter summary 83

10. Time Series Analysis and Forecasting 84

10.1 Time series in pandas 84

10.2 Components and classical decomposition 85

10.3 Stationarity 86

10.4 Autocorrelation: ACF and PACF 87

10.5 Train/test split and benchmark forecasts 89

10.6 Exponential smoothing (Holt-Winters) 89

10.7 ARIMA and seasonal ARIMA 90

10.8 Comparing forecasts and building prediction intervals 91

10.9 Time series cross-validation 92

10.10 Forecasting with regression on lagged features 92

10.11 The statsmodels time series toolkit 93

Chapter summary . . . 94

11. Factor Analysis and Principal Component Analysis . . . 95

11.1 PCA: the mathematics . . . 95

11.2 How many components? . . . 96

11.3 The factor analysis model . . . 97

11.4 Is the data suitable for factor analysis? . . . 98

11.5 How many factors? Parallel analysis . . . 98

11.6 Extraction and rotation . . . 99

11.7 Factor scores and reliability . . . 101

11.8 The factor_analyzer library . . . 101

Chapter summary . . . 102

12. Cluster Analysis . . . 103

12.1 Distance and the importance of scaling . . . 103

12.2 Hierarchical (agglomerative) clustering . . . 104

12.3 K-means . . . 105

12.4 Choosing the number of clusters . . . 105

12.5 Gaussian mixture models: probabilistic clustering . . . 107

12.6 DBSCAN: density-based clusters of any shape . . . 107

12.7 Clusters are always found: a sanity check on noise . . . 108

12.8 Profiling and interpreting clusters . . . 108

Chapter summary . . . 109

13. Maximum Likelihood Methods . . . 111

13.1 The likelihood function . . . 111

13.2 Closed-form MLEs for common models . . . 112

13.3 Numerical maximum likelihood . . . 112

13.4 Properties of MLEs and the Fisher information . . . 113

13.5 The trinity of likelihood tests . . . 113

13.6 Confidence intervals: Wald versus profile likelihood . . . 114

13.7 Linear regression as maximum likelihood . . . 115

13.8 An advanced example: censored survival data . . . 115

13.9 The EM algorithm . . . 116

13.10 Maximum likelihood in statsmodels . . . 116

Chapter summary . . . 117

14. Logistic Regression . . . 118

14.1 Odds, log-odds and the logistic function . . . 118

14.2 Estimation by maximum likelihood (IRLS) . . . 119

14.3 Interpreting the model . . . 120

14.4 Comparing models: deviance and likelihood ratio tests . . . 121

14.5 Classification: thresholds, confusion matrices and ROC . . . 122

14.6 Calibration . . . 124
14.7 Practical issues . . . 124
14.8 More than two categories: multinomial logistic regression . . . 125
14.9 Logistic regression in statsmodels . . . 125
Chapter summary . . . 126
15. Introduction to Bayesian Inference . . . 127
15.1 The beta-binomial model: conjugacy . . . 127
15.2 Summarising a posterior . . . 129
15.3 Posterior predictive distribution . . . 129
15.4 Other conjugate models . . . 130
15.5 Bayesian A/B testing . . . 130
15.6 Grid approximation for non-conjugate models . . . 131
15.7 Markov chain Monte Carlo: Metropolis-Hastings from scratch . . . 132
15.8 Bayes factors and Bayesian model comparison . . . 134
15.9 Bayesian linear regression and hierarchical models . . . 135
15.10 Probabilistic programming with PyMC and ArviZ . . . 135
15.11 A Bayesian workflow . . . 136
Chapter summary . . . 136
16. Reference Appendix: Choosing Methods, Cheat Sheets and Reading List 137
A. Which test should I use? . . . 137
B. Function cheat sheet . . . 138
C. Essential formula sheet . . . 139
D. Twelve pitfalls that cost marks (and credibility) . . . 139
E. Reading list . . . 140
F. How to keep using this guide . . . 140

How to Use This Guide

This guide is written as a lifelong companion for an MSc-level course in statistics and statistical modelling, with Python as the working language. Every chapter follows the same rhythm: the **idea** (what question the method answers and why it works), the **mathematics** (the minimum formulas you must be able to write in an exam), the **code** (Jupyter cells you can copy and run), and the **interpretation** (what the numbers mean for the data set, which is what examiners and employers actually care about).

Code moves from beginner to advanced inside every chapter. Early cells use high-level functions so you get answers quickly. Later cells rebuild the same method from scratch in NumPy, which is how you prove to yourself (and to a viva panel) that you understand what the library is doing. Wherever possible, the from-scratch result is checked against the library result.

How the code in this PDF was produced

Every grey box labelled **IN [Python / Jupyter cell]** was actually executed while this PDF was built, and the cream box labelled **OUT** underneath it is the real output, figures included. Nothing in those outputs is typed by hand. A small number of cells use `statsmodels` or `pymc`, the two standard libraries for classical and Bayesian modelling. Those cells are labelled differently and carry a purple box. They are written against the current public APIs, and each one sits next to an executed from-scratch or SciPy equivalent, so you know what numbers to expect when you run them.

PRACTICAL TIP
A companion set of Jupyter notebooks (one per chapter) is supplied with this PDF. They contain exactly the same cells in the same order, so you can read here and run there.

Setting up your environment

The cleanest setup is a dedicated environment. With Anaconda or Miniconda installed, run the following in a terminal (not in Python). The versions shown are the ones this guide was tested against; anything newer will almost certainly work.

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
# --- terminal commands (run once) ---
# conda create -n stats python=3.12 -y
# conda activate stats
# pip install numpy pandas scipy statsmodels scikit-learn matplotlib seaborn
# pip install jupyterlab pymc arviz factor_analyzer pingouin
# jupyter lab
```

RUN THIS IN YOUR JUPYTER
These are terminal commands, shown commented out so the cell is safe to run.

IN [Python / Jupyter cell]

```
# The standard import block used at the top of almost every notebook
import numpy as np                # arrays, linear algebra, random numbers
import pandas as pd               # tabular data (DataFrame), grouping, time series
import scipy.stats as stats       # distributions and classical hypothesis tests
import matplotlib.pyplot as plt   # plotting engine
```

```
import sklearn                    # machine-learning style modelling
import scipy, matplotlib

pd.set_option("display.width", 100)
pd.set_option("display.max_columns", 12)
pd.set_option("display.precision", 4)

for lib in (np, pd, scipy, matplotlib, sklearn):
    print(f"{lib.__name__:<12}{lib.__version__}")
```

OUT

```
numpy       2.4.4
pandas      3.0.2
scipy       1.17.1
matplotlib  3.10.8
sklearn     1.8.0
```

The Python statistics toolkit: what each library does

Students often get lost because the same task can be done in four libraries. The table below is the mental map used throughout the guide. A simple rule: use **SciPy** for distributions and single tests, **statsmodels** when you want a classical statistical model with standard errors, p-values and diagnostics, **scikit-learn** when the goal is prediction and validation, and **PyMC** when you want a full Bayesian posterior.

Library	What it does to your data set	Typical calls in this guide
NumPy	Fast numerical arrays; vectorised maths; linear algebra (matrix inverse, eigenvalues); random number generation.	np.mean, np.linalg.lstsq, np.random.default_rng
pandas	Loads, cleans, reshapes and summarises tables. Group-by, pivot, missing values, datetime indexing and resampling.	pd.read_csv, df.describe, df.groupby, df.resample
SciPy (scipy.stats)	100+ probability distributions (pdf, cdf, quantiles, sampling, fitting) and classical tests: t, chi-square, ANOVA, normality, nonparametric.	stats.norm, stats.ttest_ind, stats.f_oneway
SciPy (optimize)	Numerical optimisation: the engine for maximum likelihood when no closed form exists.	optimize.minimize
statsmodels	Classical statistical models with full inference tables: OLS, GLM, logistic, ANOVA tables, ARIMA, exponential smoothing, diagnostics.	smf.ols, smf.logit, anova_lm, ARIMA
scikit-learn	Prediction-focused models, preprocessing, cross-validation, PCA, factor analysis, clustering, metrics.	LinearRegression, KMeans, PCA, cross_val_score
matplotlib / seaborn	Plotting. seaborn adds statistical plots (distributions, regression lines, heatmaps) on top of matplotlib.	plt.hist, sns.boxplot, sns.heatmap
PyMC + ArviZ	Probabilistic programming: define priors and likelihood, sample the posterior with MCMC; ArviZ summarises and plots it.	pm.Model, pm.sample, az.summary
factor_analyzer / pingouin	Specialist helpers: KMO, Bartlett, rotations (factor_analyzer); tidy test outputs and effect sizes (pingouin).	FactorAnalyzer, calculate_kmo

The data sets used in this guide

To keep the guide runnable offline and fully reproducible, it uses two kinds of data. **Real data sets bundled with scikit-learn**: Iris (flower measurements, 3 species), Wine (13 chemical measurements, 3 cultivars), Diabetes (disease progression, 10 predictors) and Breast Cancer Wisconsin (30 tumour features, malignant or benign). These ship inside the library, so no download is needed. **Synthetic data sets with known truth**: a student performance data set and a monthly sales series. Synthetic data is a superpower when learning. Because we wrote down the true parameters, we can check whether a confidence interval really contains the truth, whether a regression recovers the true slopes, and whether a test rejects when it should.

The student performance data set

Each of 300 rows is a student. The final exam score is generated as score = 10 + 2.5 × study_hours + 1.5 × sleep_hours + 0.15 × attendance + 6 × prior_gpa + method effect + noise, where the teaching method effect is 0 for A, 4 for B and 7 for C, and the noise has standard deviation 6. Whether a student passed follows a separate true logistic model. Keep these numbers in mind: later chapters will try to rediscover them from the data alone.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()

print(df.shape)
print(df.head())
```

OUT

```
(300, 8)
  method gender  study_hours  sleep_hours  attendance  prior_gpa  final_score  passed
0      C      M         6.98         6.42        79.1       3.91         75.7       1
1      B      M         1.74         7.00        95.5       2.22         61.2       0
2      C      M         3.94         8.08        93.2       3.22         78.2       1
3      B      F         2.96         6.39        73.2       2.24         54.5       1
4      A      M         6.82         7.89        93.4       2.95         79.2       1
```

IN [Python / Jupyter cell]

```
# Save it once so you can also open it in Excel or load it in any notebook
df.to_csv("students.csv", index=False)
df2 = pd.read_csv("students.csv")      # this is how you would load your own CSV file
print(df2.dtypes)
```

OUT

```
method             str
gender             str
study_hours    float64
sleep_hours    float64
attendance     float64
prior_gpa      float64
final_score    float64
passed           int64
dtype: object
```

The monthly sales series

A monthly series from January 2012 to December 2025 with an upward trend, a yearly seasonal pattern whose amplitude grows with the level, and autocorrelated noise. It is used in the time series chapter.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_sales(seed=7):
    """Monthly retail sales, Jan 2012 to Dec 2025: trend + yearly seasonality
    + AR(1) noise. Multiplicative-looking seasonality grows with the level."""
    rng = np.random.default_rng(seed)
    idx = pd.date_range("2012-01-01", "2025-12-01", freq="MS")
    t = np.arange(len(idx))
    trend = 200 + 2.2 * t
    season = 1 + 0.18 * np.sin(2 * np.pi * (t % 12) / 12 - 1.2) \
               + 0.08 * np.cos(4 * np.pi * (t % 12) / 12)
    e = np.zeros(len(t))
    for i in range(1, len(t)):
        e[i] = 0.55 * e[i - 1] + rng.normal(0, 9)
    y = trend * season + e
    return pd.Series(y.round(1), index=idx, name="sales")

sales = make_sales()

print(sales.head(), "\n...")
print("length:", len(sales), "| from", sales.index.min().date(), "to", sales.index.max().date())
```

OUT

```
2012-01-01    182.4
2012-02-01    187.5
2012-03-01    193.3
2012-04-01    202.6
2012-05-01    221.2
Freq: MS, Name: sales, dtype: float64
...
length: 168 | from 2012-01-01 to 2025-12-01
```

Loading the real data sets

IN [Python / Jupyter cell]

```
from sklearn.datasets import load_iris, load_wine, load_diabetes, load_breast_cancer

iris = load_iris(as_frame=True).frame          # as_frame=True returns a pandas DataFrame
```

```
iris["species"] = pd.Categorical.from_codes(iris["target"], load_iris().target_names)
wine = load_wine(as_frame=True).frame
diab = load_diabetes(as_frame=True, scaled=False).frame
bc = load_breast_cancer(as_frame=True).frame

for name, d in [("iris", iris), ("wine", wine), ("diabetes", diab), ("breast cancer", bc)]:
    print(f"{name:<14} rows={d.shape[0]:<4} cols={d.shape[1]}")
```

OUT

```
iris           rows=150  cols=6
wine           rows=178  cols=14
diabetes       rows=442  cols=11
breast cancer  rows=569  cols=31
```

PRACTICAL TIP

Other classic data sets you will meet in textbooks are one line away once you are online: `sns.load_dataset('tips')`, `sm.datasets.get_rdataset('mtcars').data` (any R data set), and `sm.datasets.co2.load_pandas().data`.

CHAPTER 1

Descriptive Statistics and Exploratory Data Analysis

Before any model, you describe. Descriptive statistics compress a data set into a few numbers about **location** (where the data sits), **spread** (how variable it is), **shape** (symmetry, tails) and **association** (how variables move together). Exploratory data analysis (EDA), a term coined by John Tukey, adds pictures. Most modelling mistakes in practice are EDA mistakes: an outlier nobody looked at, a skewed variable treated as normal, or a categorical variable stored as a number.

1.1 Types of variables

The type of a variable decides which statistics and tests are legal. Pandas reflects this through dtypes: numeric columns are `float64/int64`, text is `str/object`, and you should explicitly mark categories with `category`.

Type	Definition	Example (student data)	Valid summaries
Nominal	Categories with no order	method (A, B, C), gender	Counts, proportions, mode
Ordinal	Ordered categories, unequal gaps	grade bands (D, C, B, A)	Median, percentiles, mode
Interval	Numeric, arbitrary zero	temperature in Celsius	Mean, SD (ratios meaningless)
Ratio	Numeric, true zero	study_hours, final_score	Everything incl. ratios, CV
Binary	Two outcomes, coded 0/1	passed	Proportion (= mean of 0/1)

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()

df["method"] = df["method"].astype("category")   # declare categorical variables explicitly
```

```
df["gender"] = df["gender"].astype("category")
df.info()                                          # dtypes, non-null counts, memory
```

OUT

```
<class 'pandas.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   method       300 non-null    category
 1   gender       300 non-null    category
 2   study_hours  300 non-null    float64
 3   sleep_hours  300 non-null    float64
 4   attendance   300 non-null    float64
 5   prior_gpa    300 non-null    float64
 6   final_score  300 non-null    float64
 7   passed       300 non-null    int64
dtypes: category(2), float64(5), int64(1)
memory usage: 14.8 KB
```

1.2 Measures of location

The **mean** is the balance point of the data, the **median** is the middle value, and the **mode** is the most frequent value. The mean uses every observation, which makes it efficient but sensitive to outliers. The median is *robust*: changing the largest value to a billion does not move it. A **trimmed mean** drops a fixed percentage from each tail, a compromise between the two.

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{median} = x_{((n+1)/2)} \qquad \bar{x}_w = \frac{\sum w_i x_i}{\sum w_i}$$

IN [Python / Jupyter cell]

```
import numpy as np
import scipy.stats as stats

x = df["final_score"]
print("mean          :", round(x.mean(), 3))
print("median        :", x.median())
print("mode (binned) :", x.round(-1).mode().tolist())   # mode suits discrete data
print("10% trimmed   :", round(stats.trim_mean(x, 0.10), 3))  # cut 10% from each tail
print("geometric mean:", round(stats.gmean(df["study_hours"]), 3))   # for growth rates, ratios
print("harmonic mean :", round(stats.hmean(df["study_hours"]), 3))   # for rates (speed, P/E)

# Robustness demo: add one absurd outlier
x_out = pd.concat([x, pd.Series([10_000])])
print("\nwith outlier -> mean:", round(x_out.mean(), 1), "| median:", x_out.median())
```

OUT

```
mean          : 68.74
median        : 67.9
mode (binned) : [70.0]
10% trimmed   : 68.288
geometric mean: 5.121
harmonic mean : 4.398

with outlier -> mean: 101.7 | median: 67.9
```

1.3 Measures of spread

Spread tells you how much individual observations differ from the centre. The **variance** is the average squared deviation and the **standard deviation** (SD) is its square root, back in the original units. The sample variance divides by n − 1 rather than n; this is **Bessel's correction**, which makes it an unbiased estimator of the population variance (proved in Chapter 5). The

interquartile range (IQR = Q3 − Q1) and the **median absolute deviation** (MAD) are the robust counterparts.

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \qquad s = \sqrt{s^2} \qquad CV = \frac{s}{\bar{x}} \qquad IQR = Q_3 - Q_1$$

WATCH OUT
NumPy and pandas disagree by default. `np.var(x)` uses ddof=0 (divide by n), while `pd.Series.var()` uses ddof=1 (divide by n − 1). In statistics you almost always want ddof=1. Pass it explicitly to avoid silent errors.

IN [Python / Jupyter cell]

```
print("variance ddof=1 (pandas default):", round(x.var(), 3))
print("variance ddof=0 (numpy default) :", round(np.var(x), 3))
print("variance ddof=1 via numpy       :", round(np.var(x, ddof=1), 3))
print("standard deviation              :", round(x.std(), 3))
print("range                           :", x.max() - x.min())
q1, q3 = x.quantile([0.25, 0.75])
print("Q1, Q3, IQR                     :", q1, q3, round(q3 - q1, 3))
print("IQR via scipy                   :", stats.iqr(x))
mad = stats.median_abs_deviation(x, scale="normal")
print("MAD (scaled to match SD)        :", round(mad, 3))
print("coefficient of variation        :", round(x.std() / x.mean(), 4))
```

OUT

```
variance ddof=1 (pandas default): 102.181
variance ddof=0 (numpy default) : 101.841
variance ddof=1 via numpy       : 102.181
standard deviation              : 10.108
range                           : 54.5
Q1, Q3, IQR                     : 61.85 74.4 12.55
IQR via scipy                   : 12.550000000000011
MAD (scaled to match SD)        : 9.489
coefficient of variation        : 0.1471
```

1.4 Shape: skewness and kurtosis

Skewness measures asymmetry: positive skew means a long right tail (income, study hours), where mean > median. **Kurtosis** measures tail heaviness. SciPy reports *excess* kurtosis by default (normal = 0); positive values mean heavier tails and more extreme outliers than a normal distribution. Rough guide: |skew| < 0.5 is fairly symmetric, 0.5 to 1 is moderate, above 1 is strong.

IN [Python / Jupyter cell]

```
num = df.select_dtypes("number").drop(columns="passed")
shape = pd.DataFrame({"skew": num.apply(stats.skew),
                      "excess_kurtosis": num.apply(stats.kurtosis),
                      "mean_minus_median": num.mean() - num.median()})
print(shape.round(3))
```

OUT

```
              skew  excess_kurtosis  mean_minus_median
study_hours  1.231            2.575              0.367
sleep_hours -0.077           -0.192             -0.022
attendance  -0.743            0.184             -2.343
prior_gpa   -0.045           -0.445              0.019
final_score  0.445            0.268              0.840
```

Study hours were generated from a gamma distribution, so the positive skew (and mean above median) is exactly what theory predicts. Attendance is negatively skewed because most students

attend a lot and a few attend little.

1.5 The five-number summary and describe()

IN [Python / Jupyter cell]

```
print(df.describe().T)                                    # numeric summary, transposed
print()
print(df.describe(include="category").T)                  # categorical summary
print()
print(df["final_score"].describe(percentiles=[.05, .25, .5, .75, .95]))
```

OUT

```
              count     mean      std    min     25%     50%      75%    max
study_hours   300.0   5.8269   2.9593   0.93   3.680   5.460   6.9825  20.62
sleep_hours   300.0   6.9629   1.1223   4.00   6.220   6.985   7.7350  10.00
attendance    300.0  78.4070  13.0636  35.20  71.625  80.750  88.6500  99.20
prior_gpa     300.0   3.0193   0.4459   1.71   2.700   3.000   3.3425   4.00
final_score   300.0  68.7403  10.1085  45.20  61.850  67.900  74.4000  99.70
passed        300.0   0.7500   0.4337   0.00   0.750   1.000   1.0000   1.00

       count unique top freq
method   300      3   A  111
gender   300      2   M  165

count    300.0000
mean      68.7403
std       10.1085
min       45.2000
5%        53.3950
25%       61.8500
50%       67.9000
75%       74.4000
95%       86.6650
max       99.7000
Name: final_score, dtype: float64
```

1.6 Summaries by group

Most real questions are comparative: does method C score higher than method A? Group-by is the single most important pandas operation for statistics.

IN [Python / Jupyter cell]

```
g = df.groupby("method", observed=True)["final_score"]
print(g.agg(["count", "mean", "median", "std", "min", "max"]).round(2))
print()
# Multiple variables and custom functions at once
summary = df.groupby("method", observed=True).agg(
    n=("final_score", "size"),
    mean_score=("final_score", "mean"),
    sd_score=("final_score", "std"),
    pass_rate=("passed", "mean"),
    mean_study=("study_hours", "mean"))
print(summary.round(3))
```

OUT

```
        count   mean  median    std   min   max
method
A         111  66.12   66.00   9.74  45.2  91.8
B         102  68.03   67.15   9.24  46.6  97.5
C          87  72.92   72.40  10.32  52.6  99.7

          n  mean_score  sd_score  pass_rate  mean_study
method
A       111      66.115     9.741      0.802       6.157
B       102      68.030     9.235      0.696       5.422
C        87      72.922    10.321      0.747       5.881
```

IN [Python / Jupyter cell]

```
# Cross-tabulation of two categorical variables (counts and row proportions)
print(pd.crosstab(df["method"], df["passed"], margins=True))
print()
print(pd.crosstab(df["method"], df["passed"], normalize="index").round(3))
```

OUT

```
passed   0    1  All
method
A       22   89  111
B       31   71  102
C       22   65   87
All     75  225  300

passed      0      1
method
A       0.198  0.802
B       0.304  0.696
C       0.253  0.747
```

1.7 Association: covariance and correlation

Covariance measures whether two variables move together, but its scale depends on the units. **Pearson's correlation** r standardises it into $[-1, 1]$ and measures *linear* association. **Spearman's** ρ is Pearson's r computed on ranks, so it captures any monotonic relationship and resists outliers. **Kendall's** τ counts concordant versus discordant pairs and is preferred for small samples with many ties.

$$\operatorname{cov}(x,y) = \frac{1}{n-1}\sum (x_i - \bar{x})(y_i - \bar{y}) \qquad r = \frac{\operatorname{cov}(x,y)}{s_x s_y}$$

IN [Python / Jupyter cell]

```
print("covariance matrix:\n", num.cov().round(2), "\n")
print("Pearson correlation:\n", num.corr().round(3), "\n")
r, p = stats.pearsonr(df["study_hours"], df["final_score"])
rho, p_s = stats.spearmanr(df["study_hours"], df["final_score"])
tau, p_k = stats.kendalltau(df["study_hours"], df["final_score"])
print(f"study vs score: Pearson r={r:.3f} (p={p:.2e}), Spearman={rho:.3f}, Kendall={tau:.3f}")
```

OUT

```
covariance matrix:
              study_hours  sleep_hours  attendance  prior_gpa  final_score
study_hours          8.76        -0.12        0.35      -0.01        19.61
sleep_hours         -0.12         1.26        0.15      -0.00         0.99
attendance           0.35         0.15      170.66      -0.55        25.77
prior_gpa           -0.01        -0.00       -0.55       0.20         1.12
final_score         19.61         0.99       25.77       1.12       102.18

Pearson correlation:
              study_hours  sleep_hours  attendance  prior_gpa  final_score
study_hours         1.000       -0.035       0.009     -0.005        0.656
sleep_hours        -0.035        1.000       0.010     -0.009        0.087
attendance          0.009        0.010       1.000     -0.094        0.195
prior_gpa          -0.005       -0.009      -0.094      1.000        0.248
final_score         0.656        0.087       0.195      0.248        1.000

study vs score: Pearson r=0.656 (p=3.05e-38), Spearman=0.604, Kendall=0.431
```

WATCH OUT

Correlation is not causation, and zero correlation is not independence. $Y = X^2$ with X symmetric around 0 has Pearson r ≈ 0 even though Y is a perfect function of X. Always plot.

IN [Python / Jupyter cell]

```
# Anscombe-style demonstration: same correlation, very different data
rng = np.random.default_rng(0)
xs = rng.uniform(-3, 3, 200)
print("r(x, x^2) =", round(np.corrcoef(xs, xs**2)[0, 1], 3), " <- strong dependence, r ~ 0")
```

OUT

```
r(x, x^2) = 0.103  <- strong dependence, r ~ 0
```

1.8 Visual EDA

Four plots cover most of EDA: a **histogram/KDE** for one distribution, a **box plot** to compare groups and flag outliers (points beyond 1.5 × IQR from the box), a **scatter plot** for two numeric variables, and a **correlation heatmap** for many.

IN [Python / Jupyter cell]

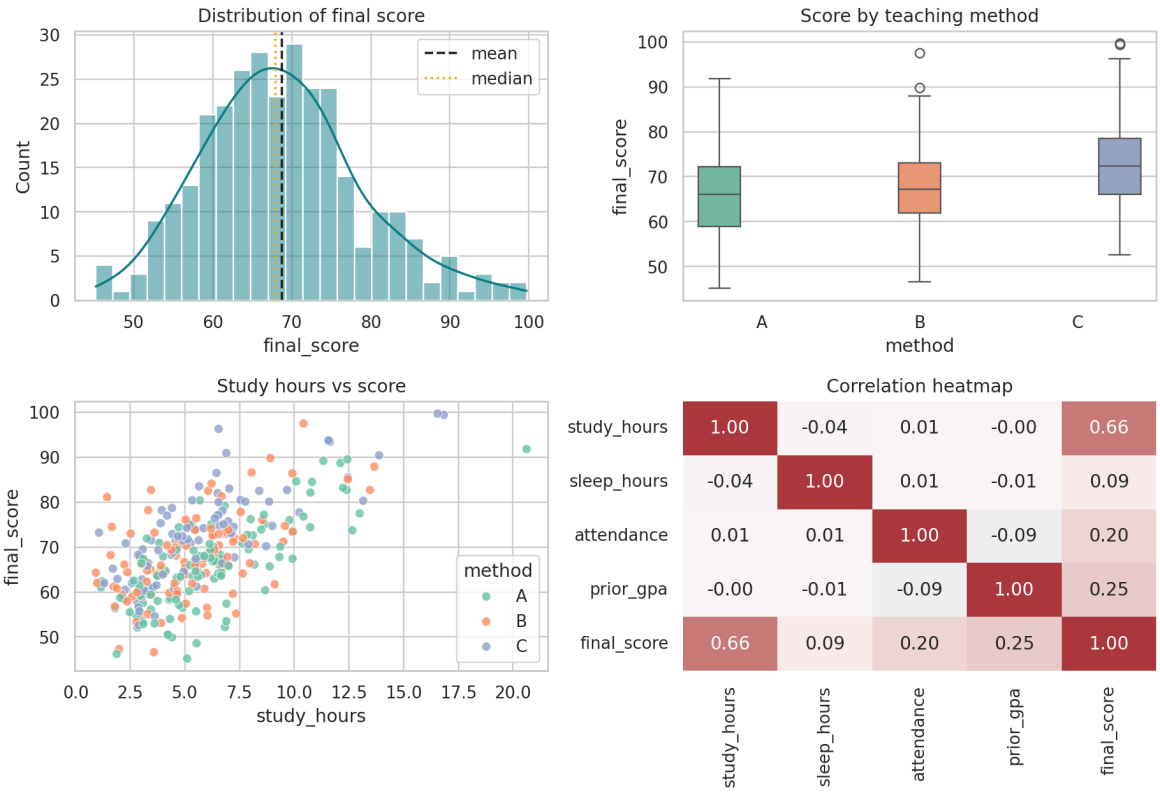
```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid", context="notebook")

fig, ax = plt.subplots(2, 2, figsize=(11, 7.5))
sns.histplot(df["final_score"], kde=True, bins=25, ax=ax[0, 0], color="#0E7C86")
ax[0, 0].axvline(df["final_score"].mean(), color="k", ls="--", label="mean")
ax[0, 0].axvline(df["final_score"].median(), color="orange", ls=":", label="median")
ax[0, 0].legend(); ax[0, 0].set_title("Distribution of final score")

sns.boxplot(data=df, x="method", y="final_score", ax=ax[0, 1], hue="method",
            palette="Set2", legend=False)
ax[0, 1].set_title("Score by teaching method")

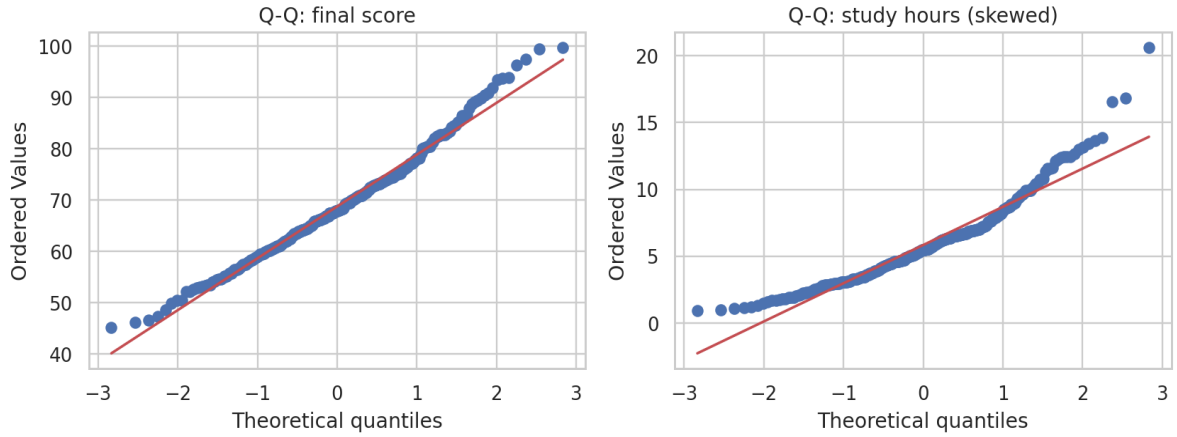
sns.scatterplot(data=df, x="study_hours", y="final_score", hue="method",
                ax=ax[1, 0], palette="Set2", alpha=.8)
ax[1, 0].set_title("Study hours vs score")

sns.heatmap(num.corr(), annot=True, fmt=".2f", cmap="vlag", center=0, ax=ax[1, 1],
            cbar=False)
ax[1, 1].set_title("Correlation heatmap")
plt.tight_layout(); plt.show()
```



IN [Python / Jupyter cell]

```
# Q-Q plot: the best single check of normality (points on the line = normal)
fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
stats.probplot(df["final_score"], dist="norm", plot=ax[0])
ax[0].set_title("Q-Q: final score")
stats.probplot(df["study_hours"], dist="norm", plot=ax[1])
ax[1].set_title("Q-Q: study hours (skewed)")
plt.tight_layout(); plt.show()
```



On a Q-Q plot, a right-skewed variable bends upward at the right end (the largest values are larger than a normal would predict). That is exactly the pattern in study hours.

1.9 Outliers and missing values

Two classic outlier rules: the **IQR rule** (flag values below Q1 − 1.5 IQR or above Q3 + 1.5 IQR) and the **z-score rule** (|z| > 3). The z-score rule is circular, because the outliers inflate the SD that is used to detect them, so a robust z based on the median and MAD is better. Never delete outliers automatically. Investigate them: they may be data entry errors, or the most interesting

observations in the data set.

IN [Python / Jupyter cell]

```
def outlier_report(s):
    q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1
    iqr_flag = (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)
    z = (s - s.mean()) / s.std()
    rz = (s - s.median()) / stats.median_abs_deviation(s, scale="normal")   # robust z
    return pd.Series({"IQR rule": iqr_flag.sum(), "|z|>3": (z.abs() > 3).sum(),
                      "|robust z|>3": (rz.abs() > 3).sum()})

print(num.apply(outlier_report).T)
```

OUT

```
             IQR rule  |z|>3  |robust z|>3
study_hours        15      3             8
sleep_hours         0      0             0
attendance          5      2             3
prior_gpa           1      0             0
final_score         7      2             3
```

IN [Python / Jupyter cell]

```
# Missing data: detect, quantify, and handle
dm = df.copy()
rng = np.random.default_rng(1)
dm.loc[rng.choice(dm.index, 25, replace=False), "sleep_hours"] = np.nan   # inject missingness
print("missing per column:\n", dm.isna().sum()[lambda s: s > 0])
print("share missing     :", round(dm["sleep_hours"].isna().mean(), 3))

complete = dm.dropna()                                              # listwise deletion
mean_imp = dm["sleep_hours"].fillna(dm["sleep_hours"].mean())    # mean imputation
grp_imp = dm.groupby("method", observed=True)["sleep_hours"] \
            .transform(lambda s: s.fillna(s.median()))           # group-wise median
print("rows after dropna :", len(complete))
print("SD original / mean-imputed:", round(df["sleep_hours"].std(), 3), "/",
      round(mean_imp.std(), 3), " <- imputation shrinks variance")
```

OUT

```
missing per column:
 sleep_hours    25
dtype: int64
share missing     : 0.083
rows after dropna : 275
SD original / mean-imputed: 1.122 / 1.073  <- imputation shrinks variance
```

EXAM / VIVA ANGLE

Know the three missingness mechanisms for your viva: **MCAR** (missing completely at random; deletion is unbiased), **MAR** (missingness depends on observed variables; use multiple imputation, e.g. `sklearn.impute.IterativeImputer`), and **MNAR** (depends on the missing value itself; requires modelling the missingness).

1.10 Standardisation and transformations

A **z-score** $z = (x - \bar{x})/s$ expresses each value in SD units, making variables comparable. For skewed positive data, a **log** or **Box-Cox** transform often produces near-normality and stabilises variance, which matters for regression and ANOVA assumptions.

IN [Python / Jupyter cell]

```
from sklearn.preprocessing import StandardScaler

z_manual = (df["study_hours"] - df["study_hours"].mean()) / df["study_hours"].std()
```

```
z_sklearn = StandardScaler().fit_transform(df[["study_hours"]]).ravel()  # uses ddof=0!
print("manual z (ddof=1) SD :", round(z_manual.std(), 4))
print("sklearn z (ddof=0) SD:", round(z_sklearn.std(ddof=1), 4))

bc_values, lam = stats.boxcox(df["study_hours"])     # finds the lambda by maximum likelihood
print(f"\nBox-Cox lambda = {lam:.3f}  (0 would mean log transform)")
print("skew before:", round(stats.skew(df["study_hours"]), 3),
      "| log:", round(stats.skew(np.log(df["study_hours"])), 3),
      "| Box-Cox:", round(stats.skew(bc_values), 3))
```

OUT

```
manual z (ddof=1) SD : 1.0
sklearn z (ddof=0) SD: 1.0017

Box-Cox lambda = 0.281  (0 would mean log transform)
skew before: 1.231 | log: -0.478 | Box-Cox: 0.007
```

Chapter summary

Describe location, spread, shape and association, by group, before modelling. Prefer the median and IQR when data are skewed or contain outliers. Always pass `ddof=1` when you mean the sample variance. Use Q-Q plots to judge normality, Spearman or Kendall when relationships are monotonic but not linear, and treat outliers and missing values as questions to investigate rather than nuisances to delete.

PRACTICAL TIP

Core references for this chapter: Tukey, *Exploratory Data Analysis* (1977); Freedman, Pisani & Purves, *Statistics*, chapters 3 to 9; McKinney, *Python for Data Analysis* (3rd ed.), chapters 5 to 10.

CHAPTER 2

Basic Probability

Probability is the language of uncertainty, and every inferential method later in this guide is an application of a handful of probability rules. This chapter covers those rules, shows how to compute exact answers with combinatorics, and, most usefully in practice, how to **verify any probability by simulation**. If you can simulate it, you can check it.

2.1 Sample spaces, events and the axioms

A **random experiment** has a **sample space** S of possible outcomes. An **event** A is a subset of S. Kolmogorov's three axioms define a probability function: P(A) ≥ 0; P(S) = 1; and for mutually exclusive events, P(A ∪ B) = P(A) + P(B). Everything else is derived, including the complement rule and the addition rule.

$$P(A^c) = 1 - P(A) \qquad P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Two interpretations matter. The **frequentist** view says P(A) is the long-run relative frequency of A in repeated trials. The **Bayesian** view says P(A) is a degree of belief, updated by evidence. Both obey the same axioms; they differ in what they allow probability to be attached to (Chapter 15).

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
from itertools import product

# Enumerate the sample space of two dice and compute probabilities by counting
S = pd.DataFrame(list(product(range(1, 7), repeat=2)), columns=["d1", "d2"])
S["total"] = S.d1 + S.d2
A = S.total == 7                  # event A: total is 7
B = (S.d1 == 6) | (S.d2 == 6)     # event B: at least one six
print("|S| =", len(S))
print("P(A)       =", A.mean())
print("P(B)       =", round(B.mean(), 4), "(exact 11/36 =", round(11 / 36, 4), ")")
print("P(A and B) =", round((A & B).mean(), 4))
print("P(A or B)  =", round((A | B).mean(), 4),
      "= P(A)+P(B)-P(AB) =", round(A.mean() + B.mean() - (A & B).mean(), 4))
```

OUT

```
|S| = 36
P(A)       = 0.16666666666666666
P(B)       = 0.3056 (exact 11/36 = 0.3056 )
P(A and B) = 0.0556
P(A or B)  = 0.4167 = P(A)+P(B)-P(AB) = 0.4167
```

2.2 Simulation: the frequentist definition in code

A modern habit worth building: whenever you derive a probability, confirm it by simulation. NumPy's `default_rng` is the recommended random number generator; always set a seed so results are reproducible.

IN [Python / Jupyter cell]

```
import matplotlib.pyplot as plt

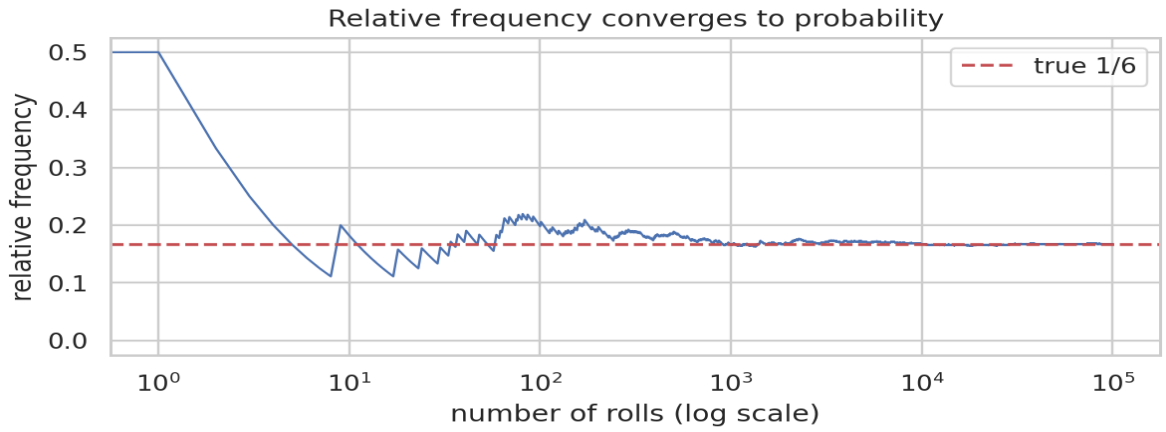
rng = np.random.default_rng(2026)          # seeded generator: reproducible results
```

```
n_sims = 100_000
d1 = rng.integers(1, 7, n_sims)            # integers in [1, 7)
d2 = rng.integers(1, 7, n_sims)
hit = (d1 + d2) == 7
print("simulated P(total=7):", hit.mean(), "| exact:", round(1 / 6, 5))

# Convergence of relative frequency (law of large numbers, previewed)
running = np.cumsum(hit) / np.arange(1, n_sims + 1)
plt.figure(figsize=(8, 3))
plt.semilogx(running, lw=1)
plt.axhline(1 / 6, color="r", ls="--", label="true 1/6")
plt.xlabel("number of rolls (log scale)"); plt.ylabel("relative frequency")
plt.title("Relative frequency converges to probability"); plt.legend(); plt.show()
```

OUT

```
simulated P(total=7): 0.167 | exact: 0.16667
```



2.3 Counting: permutations and combinations

For equally likely outcomes, P(A) = |A| / |S|, so probability becomes counting. The number of ordered arrangements of k items chosen from n is a **permutation**; unordered selections are **combinations**.

$$ {}^{n}P_{k} = \frac{n!}{(n-k)!} \qquad \binom{n}{k} = \frac{n!}{k!\,(n-k)!} $$

IN [Python / Jupyter cell]

```
from math import comb, perm, factorial

print("ways to choose a 3-person committee from 10:", comb(10, 3))
print("ways to fill president/VP/secretary from 10:", perm(10, 3))
# Probability of exactly 2 aces in a 5-card poker hand (hypergeometric counting)
p_two_aces = comb(4, 2) * comb(48, 3) / comb(52, 5)
print("P(exactly 2 aces in 5 cards):", round(p_two_aces, 5))

# The birthday problem: P(at least two of k people share a birthday)
def p_shared(k):
    p_none = np.prod([(365 - i) / 365 for i in range(k)])
    return 1 - p_none
for k in (10, 23, 50):
    sims = rng.integers(0, 365, size=(20_000, k))
    sim_p = np.mean([len(np.unique(r)) < k for r in sims])
    print(f"k={k:>2}: exact={p_shared(k):.4f}  simulated={sim_p:.4f}")
```

OUT

```
ways to choose a 3-person committee from 10: 120
ways to fill president/VP/secretary from 10: 720
P(exactly 2 aces in 5 cards): 0.03993
k=10: exact=0.1169  simulated=0.1187
```

```
k=23: exact=0.5073  simulated=0.5144
k=50: exact=0.9704  simulated=0.9700
```

2.4 Conditional probability and independence

Conditional probability restricts the sample space to the event B that we know has happened. Events are **independent** when knowing B tells you nothing about A. Independence is a modelling assumption behind almost every test in this guide (independent observations), so it is worth understanding precisely.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \qquad A \perp B \Leftrightarrow P(A \cap B) = P(A)P(B)$$

IN [Python / Jupyter cell]

```python
# Using the dice sample space S from before
P = lambda ev: ev.mean()
C = S.d1 == 6                       # first die is a six
D = S.total >= 10                   # total at least 10
print("P(D)      =", round(P(D), 4))
print("P(D | C)  =", round(P(D & C) / P(C), 4), " <- knowing the first die is 6 changes it")
E = S.d2 % 2 == 0                   # second die even
print("P(C & E)  =", round(P(C & E), 4),
      "| P(C)P(E) =", round(P(C) * P(E), 4), "-> independent")
```

OUT

```
P(D)      = 0.1667
P(D | C)  = 0.5  <- knowing the first die is 6 changes it
P(C & E)  = 0.0833 | P(C)P(E) = 0.0833 -> independent
```

2.5 Law of total probability and Bayes' theorem

If B_1, ..., B_k partition the sample space, any event's probability is a weighted average over the partition. Bayes' theorem inverts a conditional probability: from P(evidence | hypothesis) to P(hypothesis | evidence). This single formula is the foundation of Bayesian inference in Chapter 15.

$$P(A) = \sum_j P(A \mid B_j)P(B_j) \qquad P(B_j \mid A) = \frac{P(A \mid B_j)P(B_j)}{\sum_i P(A \mid B_i)P(B_i)}$$

Worked example: a diagnostic test

A disease has prevalence 1%. A test has **sensitivity** P(+ | disease) = 0.95 and **specificity** P(− | healthy) = 0.90. If a person tests positive, what is the probability they have the disease? Intuition says about 95%. Bayes says otherwise.

IN [Python / Jupyter cell]

```python
def bayes_positive(prev, sens, spec):
    p_pos = sens * prev + (1 - spec) * (1 - prev)     # law of total probability
    return sens * prev / p_pos                        # Bayes' theorem

ppv = bayes_positive(prev=0.01, sens=0.95, spec=0.90)
print(f"P(disease | positive) = {ppv:.4f}")

# Verify by simulating one million people
N = 1_000_000
sick = rng.random(N) < 0.01
positive = np.where(sick, rng.random(N) < 0.95, rng.random(N) < 0.10)
print(f"simulated             = {sick[positive].mean():.4f}")
```

```
# How PPV depends on prevalence (the base-rate effect)
for prev in (0.001, 0.01, 0.05, 0.2, 0.5):
    print(f"prevalence {prev:>5}: PPV = {bayes_positive(prev, .95, .90):.3f}")
```

OUT

```
P(disease | positive) = 0.0876
simulated             = 0.0876
prevalence 0.001: PPV = 0.009
prevalence  0.01: PPV = 0.088
prevalence  0.05: PPV = 0.333
prevalence   0.2: PPV = 0.704
prevalence   0.5: PPV = 0.905
```

KEY IDEA

Ignoring the prior (base rate) is called the **base-rate fallacy**. With a rare condition, most positives are false positives, even with a good test. The same logic explains why p-values are not the probability that the null hypothesis is true (Chapter 6).

2.6 Random variables, expectation and variance

A **random variable** X maps outcomes to numbers. A discrete X has a probability mass function (pmf) p(x) = P(X = x); a continuous X has a density f(x), where probabilities are areas under f. The **expected value** is the probability-weighted average and the **variance** is the expected squared distance from the mean.

$$E[X] = \sum_x x\,p(x) \;\text{ or }\; \int x f(x)\,dx \qquad \mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - (E[X])^2$$

These rules for linear combinations are used constantly, for example to derive the standard error of a mean: E[aX + b] = aE[X] + b; Var(aX + b) = a^2Var(X); E[X + Y] = E[X] + E[Y] always; and Var(X + Y) = Var(X) + Var(Y) + 2Cov(X, Y), which reduces to Var(X) + Var(Y) when X and Y are independent.

IN [Python / Jupyter cell]

```
# A discrete random variable defined by a table: number of defects per item
x = np.array([0, 1, 2, 3, 4])
p = np.array([0.50, 0.25, 0.15, 0.07, 0.03])
assert np.isclose(p.sum(), 1)
EX = np.sum(x * p)
VarX = np.sum(x**2 * p) - EX**2
print(f"E[X] = {EX:.3f}, Var(X) = {VarX:.4f}, SD = {np.sqrt(VarX):.4f}")

# Check by simulation, and check Var(aX+b) = a^2 Var(X)
draws = rng.choice(x, size=200_000, p=p)
print("simulated mean, var:", draws.mean().round(3), draws.var(ddof=1).round(4))
print("Var(3X+5) theory   :", 9 * VarX, "| simulated:", np.var(3 * draws + 5, ddof=1).round(3))
```

OUT

```
E[X] = 0.880, Var(X) = 1.1856, SD = 1.0889
simulated mean, var: 0.878 1.1801
Var(3X+5) theory   : 10.6704 | simulated: 10.621
```

IN [Python / Jupyter cell]

```
# Var(X+Y) with correlated variables: the covariance term matters
cov = [[1.0, 0.6], [0.6, 2.0]]
XY = rng.multivariate_normal([0, 0], cov, size=200_000)
s = XY.sum(axis=1)
print("Var(X)+Var(Y)          =", 1 + 2)
print("Var(X)+Var(Y)+2Cov(X,Y) =", 1 + 2 + 2 * 0.6)
print("simulated Var(X+Y)     =", round(s.var(ddof=1), 3))
```

OUT

```
Var(X)+Var(Y)          = 3
Var(X)+Var(Y)+2Cov(X,Y) = 4.2
simulated Var(X+Y)     = 4.196
```

2.7 Joint, marginal and conditional distributions

For two discrete variables, the **joint** distribution P(X = x, Y = y) is a table. Summing rows or columns gives the **marginals**; dividing a row by its total gives a **conditional** distribution. A crosstab of real data is exactly an estimated joint distribution.

IN [Python / Jupyter cell]

```python
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True).frame
iris["species"] = pd.Categorical.from_codes(iris.target, load_iris().target_names)
iris["long_petal"] = iris["petal length (cm)"] > 4

joint = pd.crosstab(iris.species, iris.long_petal, normalize="all")
print("joint P(species, long_petal):\n", joint.round(3), "\n")
print("marginal P(long_petal):\n", joint.sum(axis=0).round(3), "\n")
print("conditional P(long_petal | species):\n",
      pd.crosstab(iris.species, iris.long_petal, normalize="index").round(3))
```

OUT

```
joint P(species, long_petal):
 long_petal  False  True
species
setosa      0.333  0.000
versicolor  0.107  0.227
virginica   0.000  0.333

marginal P(long_petal):
 long_petal
False    0.44
True     0.56
dtype: float64

conditional P(long_petal | species):
 long_petal  False  True
species
setosa       1.00   0.00
versicolor   0.32   0.68
virginica    0.00   1.00
```

Chapter summary

Probability follows from three axioms. Conditional probability restricts the sample space; independence means conditioning changes nothing. Bayes' theorem reverses conditioning and exposes the base-rate fallacy. Expectation and variance obey simple linear rules that underpin standard errors. Above all, verify by simulation with a seeded `np.random.default_rng`.

PRACTICAL TIP

References: Blitzstein & Hwang, *Introduction to Probability* (2nd ed.), chapters 1 to 7 (free online, outstanding for intuition); Ross, *A First Course in Probability*; Rice, *Mathematical Statistics and Data Analysis*, chapters 1 to 4.

CHAPTER 3

Probability Distributions with scipy.stats

A probability distribution is a model for how a random variable behaves. Choosing the right one is half of statistical modelling: counts are often Poisson, waiting times exponential, proportions binomial, measurement errors normal. SciPy implements more than 100 distributions behind one consistent interface, so learning the interface once unlocks all of them.

3.1 The scipy.stats interface

Every distribution object offers the same methods. You can pass parameters each time or **freeze** a distribution with fixed parameters. Continuous distributions are controlled by `loc` and `scale` (plus shape parameters), which is where most beginner errors come from: for example, `stats.expon(scale=1/rate)` and `stats.norm(loc=mu, scale=sigma)` where scale is the SD, not the variance.

Method	Meaning	Question it answers
`pmf(k) / pdf(x)`	Mass or density at a point	How likely is exactly k? How dense is x?
`cdf(x)`	P(X ≤ x)	What fraction falls at or below x?
`sf(x)`	1 − cdf(x) = P(X > x), computed accurately	Upper-tail probability (p-values)
`ppf(q)`	Inverse cdf (quantile function)	Which x has q of the mass below it? (critical values)
`isf(q)`	Inverse survival function	Upper critical value, e.g. isf(0.025)
`rvs(size, random_state)`	Random draws	Simulation
`mean(), var(), std(), stats(moments='mvsk')`	Moments	Theoretical mean, variance, skew, kurtosis
`interval(conf)`	Central interval with given probability	Where do 95% of values lie?
`fit(data)`	Maximum likelihood fit of parameters	Which parameters best explain my data?

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt

Z = stats.norm(loc=0, scale=1)          # a frozen standard normal
print("pdf(0)        :", round(Z.pdf(0), 5))
print("P(Z <= 1.96)  :", round(Z.cdf(1.96), 5))
print("P(Z > 1.96)   :", round(Z.sf(1.96), 5))
print("97.5% quantile:", round(Z.ppf(0.975), 5))
print("95% interval  :", np.round(Z.interval(0.95), 4))
print("mean, var, skew, kurt:", [float(m) for m in Z.stats(moments="mvsk")])
print("5 draws       :", Z.rvs(size=5, random_state=1).round(3))
```

OUT

```
pdf(0)        : 0.39894
P(Z <= 1.96)  : 0.975
P(Z > 1.96)   : 0.025
```

```
97.5% quantile: 1.95996
95% interval  : [-1.96  1.96]
mean, var, skew, kurt: [0.0, 1.0, 0.0, 0.0]
5 draws       : [ 1.624 -0.612 -0.528 -1.073  0.865]
```

3.2 Discrete distributions

Distribution	Models	pmf	Mean / Variance	scipy
Bernoulli(p)	One yes/no trial	$p^x(1-p)^{1-x}$	p / p(1−p)	`stats.bernoulli(p)`
Binomial(n, p)	Successes in n independent trials	$C(n,k)\ p^k(1-p)^{n-k}$	np / np(1−p)	`stats.binom(n, p)`
Poisson(λ)	Count of rare events in fixed time/space	$e^{-\lambda}\lambda^k/k!$	λ / λ	`stats.poisson(mu)`
Geometric(p)	Trials until first success	$(1-p)^{k-1}p$	$1/p$ / $(1-p)/p^2$	`stats.geom(p)`
Neg. binomial(r, p)	Failures before r-th success; overdispersed counts	$C(k+r-1,k)\ p^r(1-p)^k$	$r(1-p)/p$ / $r(1-p)/p^2$	`stats.nbinom(r, p)`
Hypergeometric	Successes drawn without replacement	C(K,k)C(N−K,n−k)/C(N, n)	nK/N	`stats.hypergeom(M, n, N)`

IN [Python / Jupyter cell]

```python
# Binomial: a multiple-choice test, 20 questions, 4 options, pure guessing
B = stats.binom(n=20, p=0.25)
print("P(exactly 5 correct)  :", round(B.pmf(5), 4))
print("P(at most 5)          :", round(B.cdf(5), 4))
print("P(pass, i.e. >= 10)   :", round(B.sf(9), 6), "  <- sf(9) = P(X > 9) = P(X >= 10)")
print("mean, var             :", B.mean(), B.var())

# Poisson: a call centre receives 4.2 calls per minute on average
Po = stats.poisson(mu=4.2)
print("\nP(no calls in a minute):", round(Po.pmf(0), 4))
print("P(more than 8 calls)   :", round(Po.sf(8), 4))
print("calls exceeded only 1% of minutes:", Po.ppf(0.99))

# Hypergeometric: 50 items, 6 defective, inspect 10 without replacement
H = stats.hypergeom(M=50, n=6, N=10)      # M=population, n=successes in pop, N=draws
print("\nP(at least 1 defective in sample):", round(H.sf(0), 4))
```

OUT

```
P(exactly 5 correct)  : 0.2023
P(at most 5)          : 0.6172
P(pass, i.e. >= 10)   : 0.013864   <- sf(9) = P(X > 9) = P(X >= 10)
mean, var             : 5.0 3.75

P(no calls in a minute): 0.015
P(more than 8 calls)   : 0.0279
calls exceeded only 1% of minutes: 10.0

P(at least 1 defective in sample): 0.7585
```

WATCH OUT

For discrete variables, P(X ≥ k) = `sf(k-1)`, not `sf(k)`. This off-by-one error is the most common mistake with discrete distributions in SciPy.

IN [Python / Jupyter cell]

```
# Poisson approximation to the binomial when n is large and p small (lambda = n p)
n, p = 1000, 0.003
k = np.arange(0, 11)
cmp = pd.DataFrame({"k": k, "binomial": stats.binom.pmf(k, n, p),
                    "poisson": stats.poisson.pmf(k, n * p)})
print(cmp.round(5).to_string(index=False))
```

OUT

```
 k  binomial  poisson
 0    0.0496   0.0498
 1    0.1491   0.1494
 2    0.2241   0.2240
 3    0.2244   0.2240
 4    0.1683   0.1680
 5    0.1009   0.1008
 6    0.0503   0.0504
 7    0.0215   0.0216
 8    0.0080   0.0081
 9    0.0027   0.0027
10    0.0008   0.0008
```

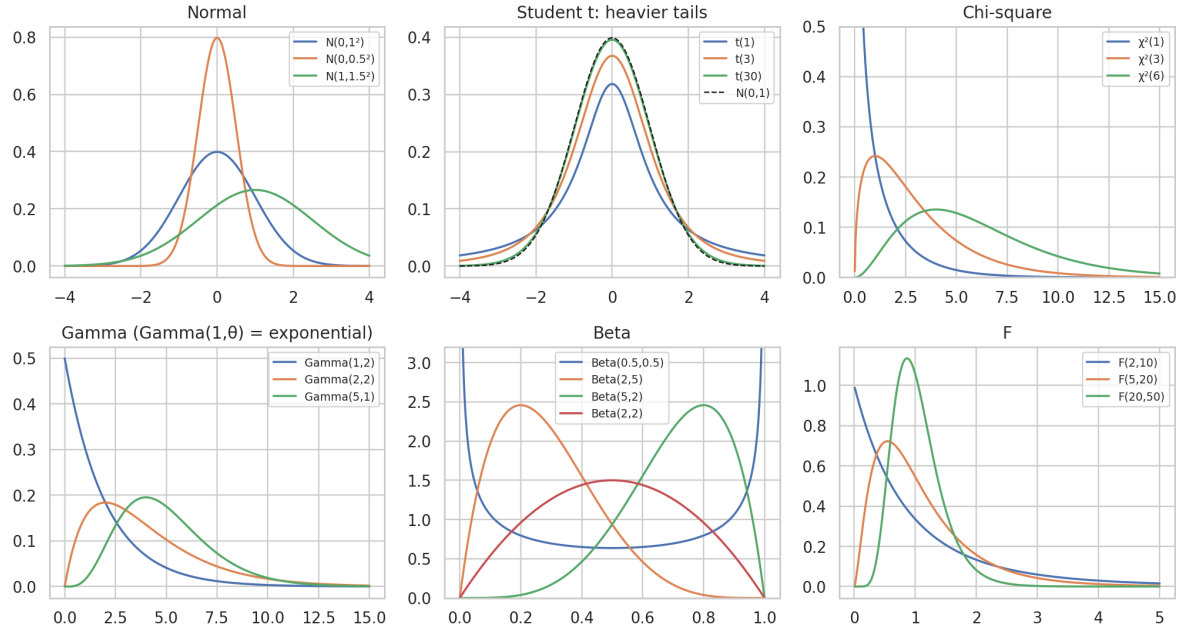
3.3 Continuous distributions

Distribution	Typical use	Mean / Variance	scipy (note the parametrisation)
Uniform(a, b)	Equal likelihood on an interval; random numbers	(a+b)/2 / (b−a)^2/12	`stats.uniform(loc=a, scale=b-a)`
Normal(μ, σ^2)	Measurement error, sums of many effects (CLT)	μ / σ^2	`stats.norm(loc=mu, scale=sigma)`
Exponential(λ)	Waiting time between Poisson events; memoryless	1/λ / 1/λ^2	`stats.expon(scale=1/lam)`
Gamma(k, θ)	Sum of k exponential waits; positive skewed data	kθ / kθ^2	`stats.gamma(a=k, scale=theta)`
Beta(α, β)	Proportions and probabilities on [0, 1]; Bayesian priors	α/(α+β)	`stats.beta(a, b)`
Lognormal	Incomes, prices: log is normal	exp(μ+σ^2/2)	`stats.lognorm(s=sigma, scale=np.exp(mu))`
Student t(ν)	Means with estimated SD; heavy tails	0 / ν/(ν−2)	`stats.t(df)`
Chi-square(k)	Sum of k squared normals; variances, contingency tables	k / 2k	`stats.chi2(df)`
F(d1, d2)	Ratio of two variances; ANOVA and regression F tests	d2/(d2−2)	`stats.f(dfn, dfd)`

IN [Python / Jupyter cell]

```
x = np.linspace(-4, 4, 400)
fig, ax = plt.subplots(2, 3, figsize=(12, 6.5))
ax = ax.ravel()
for mu, s in [(0, 1), (0, 0.5), (1, 1.5)]:
    ax[0].plot(x, stats.norm.pdf(x, mu, s), label=f"N({mu},{s}²)")
ax[0].set_title("Normal"); ax[0].legend(fontsize=8)
for df_ in [1, 3, 30]:
    ax[1].plot(x, stats.t.pdf(x, df_), label=f"t({df_})")
ax[1].plot(x, stats.norm.pdf(x), "k--", lw=1, label="N(0,1)")
ax[1].set_title("Student t: heavier tails"); ax[1].legend(fontsize=8)
xp = np.linspace(0.001, 15, 400)
for k in [1, 3, 6]:
    ax[2].plot(xp, stats.chi2.pdf(xp, k), label=f"χ²({k})")
```

```
ax[2].set_title("Chi-square"); ax[2].legend(fontsize=8); ax[2].set_ylim(0, .5)
for a, th in [(1, 2), (2, 2), (5, 1)]:
    ax[3].plot(xp, stats.gamma.pdf(xp, a, scale=th), label=f"Gamma({a},{th})")
ax[3].set_title("Gamma (Gamma(1,θ) = exponential)"); ax[3].legend(fontsize=8)
u = np.linspace(0, 1, 300)
for a, b in [(0.5, 0.5), (2, 5), (5, 2), (2, 2)]:
    ax[4].plot(u, stats.beta.pdf(u, a, b), label=f"Beta({a},{b})")
ax[4].set_title("Beta"); ax[4].legend(fontsize=8); ax[4].set_ylim(0, 3.2)
xf = np.linspace(0.01, 5, 400)
for d1, d2 in [(2, 10), (5, 20), (20, 50)]:
    ax[5].plot(xf, stats.f.pdf(xf, d1, d2), label=f"F({d1},{d2})")
ax[5].set_title("F"); ax[5].legend(fontsize=8)
plt.tight_layout(); plt.show()
```



IN [Python / Jupyter cell]

```
# Classic normal calculations: IQ ~ N(100, 15^2)
IQ = stats.norm(100, 15)
print("P(IQ > 130)           :", round(IQ.sf(130), 4))
print("P(85 < IQ < 115)      :", round(IQ.cdf(115) - IQ.cdf(85), 4), " (the 68% rule)")
print("top 2% cutoff         :", round(IQ.ppf(0.98), 2))
# The 68-95-99.7 rule, exactly
for k in (1, 2, 3):
    print(f"P(|Z| < {k}) = {stats.norm.cdf(k) - stats.norm.cdf(-k):.4f}")

# Exponential and memorylessness: P(T > s+t | T > s) = P(T > t)
T = stats.expon(scale=1 / 0.5)              # rate 0.5 per hour -> mean 2 hours
s, t = 3, 1
print("\nP(T > 4 | T > 3) =", round(T.sf(s + t) / T.sf(s), 4),
      "| P(T > 1) =", round(T.sf(t), 4))
```

OUT

```
P(IQ > 130)           : 0.0228
P(85 < IQ < 115)      : 0.6827  (the 68% rule)
top 2% cutoff         : 130.81
P(|Z| < 1) = 0.6827
P(|Z| < 2) = 0.9545
P(|Z| < 3) = 0.9973

P(T > 4 | T > 3) = 0.6065 | P(T > 1) = 0.6065
```

Critical values: the tables at the back of old textbooks

Every printed statistical table is a call to `ppf`. Being able to produce them on demand removes the need to memorise anything.

IN [Python / Jupyter cell]

```
alpha = 0.05
print("z  two-sided 5%     :", round(stats.norm.ppf(1 - alpha / 2), 4))
print("t  two-sided, df=10 :", round(stats.t.ppf(1 - alpha / 2, df=10), 4))
print("chi2 upper 5%, df=3 :", round(stats.chi2.ppf(1 - alpha, df=3), 4))
print("F upper 5%, (3, 20) :", round(stats.f.ppf(1 - alpha, 3, 20), 4))

# A mini t-table
dfs = [1, 2, 5, 10, 20, 30, 60, 120, np.inf]
tab = pd.DataFrame({f"{q}": [stats.t.ppf(q, d) for d in dfs] for q in (.9, .95, .975, .995)},
                   index=[str(d) for d in dfs])
tab.index.name = "df"
print("\nupper quantiles of t:\n", tab.round(3))
```

OUT

```
z  two-sided 5%     : 1.96
t  two-sided, df=10 : 2.2281
chi2 upper 5%, df=3 : 7.8147
F upper 5%, (3, 20) : 3.0984

upper quantiles of t:
        0.9   0.95   0.975   0.995
df
1     3.078  6.314  12.706  63.657
2     1.886  2.920   4.303   9.925
5     1.476  2.015   2.571   4.032
10    1.372  1.812   2.228   3.169
20    1.325  1.725   2.086   2.845
30    1.310  1.697   2.042   2.750
60    1.296  1.671   2.000   2.660
120   1.289  1.658   1.980   2.617
inf   1.282  1.645   1.960   2.576
```

3.4 Relationships between distributions

Distributions form a family tree, and understanding it explains where test statistics come from. If $Z_1, \ldots, Z_k$ are independent standard normals, $\Sigma Z_i^2 \sim \chi^2(k)$. A standard normal divided by the square root of an independent $\chi^2(\nu)/\nu$ is $t(\nu)$. A ratio of two independent scaled chi-squares is F. And $t(\nu)^2 \sim F(1, \nu)$, which is why a two-group t test and a two-group ANOVA give identical p-values.

IN [Python / Jupyter cell]

```
rng = np.random.default_rng(3)
N, k, nu = 200_000, 4, 7
Zs = rng.standard_normal((N, k))
chi = (Zs**2).sum(axis=1)
t_sim = rng.standard_normal(N) / np.sqrt(rng.chisquare(nu, N) / nu)
print("sum of 4 squared normals: mean", chi.mean().round(3), "(theory 4), var",
      chi.var().round(3), "(theory 8)")
print("simulated t(7) 97.5% quantile:", np.quantile(t_sim, .975).round(3),
      "| theory:", stats.t.ppf(.975, nu).round(3))
print("t(7)^2 95% quantile:", round(stats.t.ppf(.975, nu)**2, 4),
      "| F(1,7) 95% quantile:", round(stats.f.ppf(.95, 1, nu), 4))
```

OUT

```
sum of 4 squared normals: mean 4.003 (theory 4), var 7.986 (theory 8)
simulated t(7) 97.5% quantile: 2.367 | theory: 2.365
t(7)^2 95% quantile: 5.5914 | F(1,7) 95% quantile: 5.5914
```

3.5 Fitting a distribution to data

Given data, `dist.fit(data)` returns maximum likelihood estimates of the parameters (Chapter 13 explains the method). You can then judge the fit visually with a histogram plus fitted density and a Q-Q plot, and formally with a goodness-of-fit test. Compare candidate distributions using the log-likelihood or AIC.

IN [Python / Jupyter cell]

```
rng = np.random.default_rng(42)
study = np.clip(rng.gamma(shape=4, scale=1.5, size=300), 0.5, None)  # same as student data

results = {}
for name, dist in [("norm", stats.norm), ("gamma", stats.gamma),
                   ("lognorm", stats.lognorm), ("expon", stats.expon)]:
    params = dist.fit(study, floc=0) if name != "norm" else dist.fit(study)
    ll = np.sum(dist.logpdf(study, *params))
    k = len(params) - (0 if name == "norm" else 1)        # floc=0 was fixed, not estimated
    results[name] = {"params": np.round(params, 3), "loglik": round(ll, 2),
                     "AIC": round(2 * k - 2 * ll, 2)}
print(pd.DataFrame(results).T.sort_values("AIC"))
```

OUT

```
                      params  loglik      AIC
gamma    [4.277, 0.0, 1.414] -722.89  1449.78
lognorm  [0.513, 0.0, 5.357] -728.98  1461.96
norm           [6.049, 2.93] -748.15   1500.3
expon           [0.0, 6.049] -839.98  1681.95
```

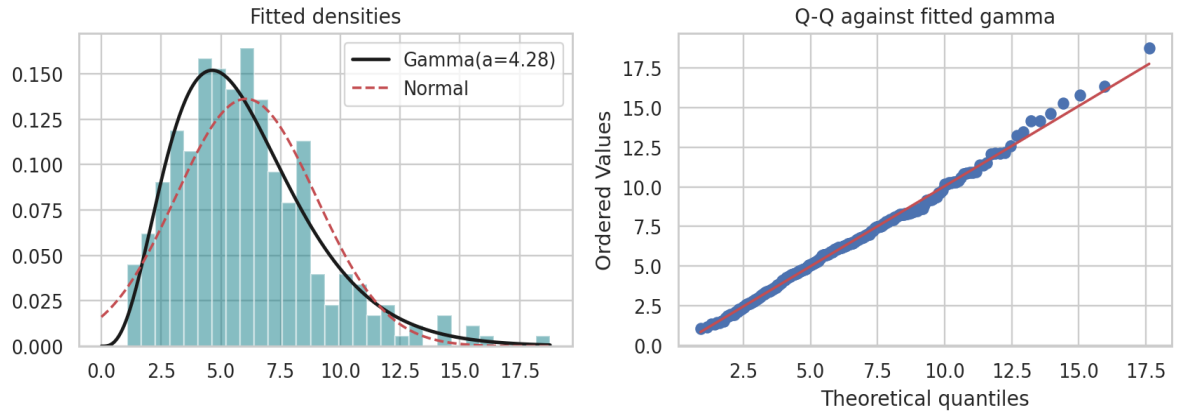
IN [Python / Jupyter cell]

```
a, loc, scale = stats.gamma.fit(study, floc=0)
xs = np.linspace(0, study.max(), 300)
fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
ax[0].hist(study, bins=30, density=True, alpha=.5, color="#0E7C86")
ax[0].plot(xs, stats.gamma.pdf(xs, a, loc, scale), "k", lw=2, label=f"Gamma(a={a:.2f})")
ax[0].plot(xs, stats.norm.pdf(xs, *stats.norm.fit(study)), "r--", label="Normal")
ax[0].legend(); ax[0].set_title("Fitted densities")
stats.probplot(study, dist=stats.gamma(a, loc, scale), plot=ax[1])
ax[1].set_title("Q-Q against fitted gamma")
plt.tight_layout(); plt.show()

# Formal test (parameters estimated, so use goodness_of_fit, which simulates the null)
res = stats.goodness_of_fit(stats.gamma, study, known_params={"loc": 0},
                            statistic="ad", random_state=0, n_mc_samples=499)
print(f"Anderson-Darling vs gamma: stat={res.statistic:.3f}, p={res.pvalue:.3f}")
```

OUT

```
Anderson-Darling vs gamma: stat=0.320, p=0.526
```



WATCH OUT
The plain Kolmogorov-Smirnov test `stats.kstest` assumes the parameters were known in advance. If you estimated them from the same data, its p-values are too large (too lenient). Use `stats.goodness_of_fit` (parametric bootstrap) or the Lilliefors correction in statsmodels.

3.6 The multivariate normal

The multivariate normal N(μ, Σ) is the foundation of regression, factor analysis and many clustering models. Its marginals are normal, any linear combination is normal, and zero correlation implies independence (a property special to the normal).

IN [Python / Jupyter cell]

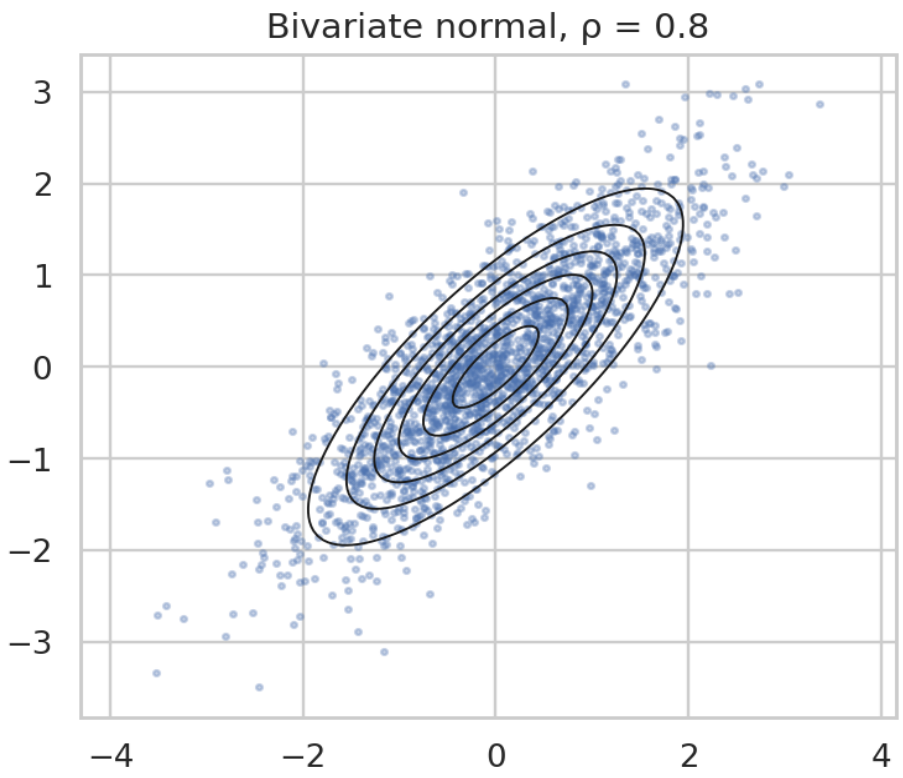
```
mu = np.array([0, 0])
Sigma = np.array([[1.0, 0.8], [0.8, 1.0]])
MVN = stats.multivariate_normal(mean=mu, cov=Sigma)
draws = MVN.rvs(size=2000, random_state=5)
print("sample correlation:", np.corrcoef(draws.T)[0, 1].round(3))

g = np.linspace(-3, 3, 120); X, Y = np.meshgrid(g, g)
dens = MVN.pdf(np.dstack([X, Y]))
plt.figure(figsize=(5, 4.3))
plt.scatter(draws[:, 0], draws[:, 1], s=4, alpha=.3)
plt.contour(X, Y, dens, levels=6, colors="k", linewidths=.8)
plt.title("Bivariate normal, ρ = 0.8"); plt.axis("equal"); plt.show()

# Conditional distribution: X2 | X1 = x1 is normal with these parameters
x1 = 1.5
cond_mean = mu[1] + Sigma[1, 0] / Sigma[0, 0] * (x1 - mu[0])
cond_var = Sigma[1, 1] - Sigma[1, 0]**2 / Sigma[0, 0]
print(f"E[X2 | X1=1.5] = {cond_mean:.2f}, Var = {cond_var:.2f}  <- this IS linear regression")
```

OUT

```
sample correlation: 0.808
E[X2 | X1=1.5] = 1.20, Var = 0.36  <- this IS linear regression
```



Chapter summary

Learn the SciPy interface once (pmf/pdf, cdf, sf, ppf, rvs, fit) and every distribution becomes available. Watch the loc/scale parametrisation and the discrete off-by-one in `sf`. Know the family tree (normal, chi-square, t, F) because every classical test statistic lives in it. Fit distributions by maximum likelihood, compare them with AIC, and check them with Q-Q plots.

PRACTICAL TIP

References: Casella & Berger, *Statistical Inference* (2nd ed.), chapters 3 to 5; Wasserman, *All of Statistics*, chapters 2 to 4; the SciPy documentation page 'Probability distributions' (scipy.stats tutorial).

CHAPTER 4

Introduction to Statistical Inference

Statistical inference is reasoning from a sample to the population it came from. The central difficulty is that a different sample would give a different answer. Inference quantifies that sample-to-sample variation through the **sampling distribution**, and every confidence interval and p-value in this guide is a statement about a sampling distribution.

4.1 Population, sample, parameter, statistic

Concept	Meaning	Notation
Population	The full set of units we want to learn about	size N (often infinite)
Parameter	A fixed, unknown number describing the population	μ, σ, p, β (Greek letters)
Sample	The units we actually observe, ideally a random sample	size n
Statistic / estimator	A function of the sample used to estimate a parameter	$\bar{x}$, s, $\hat{p}$, $\hat{\beta}$ (Latin or hats)
Sampling distribution	Distribution of a statistic over all possible samples	e.g. $\bar{x} \sim N(\mu, \sigma^2/n)$
Standard error (SE)	Standard deviation of the sampling distribution	$SE(\bar{x}) = \sigma/\sqrt{n}$

The two main tasks of inference are **estimation** (what is the value of the parameter, and how precisely do we know it?) and **hypothesis testing** (is the data consistent with a specific claim about the parameter?). Chapters 5 and 6 treat them in turn; this chapter builds the machinery they share.

4.2 Seeing a sampling distribution

We can create a population ourselves, draw thousands of samples from it, and look at the resulting distribution of the sample mean. This makes the abstract idea concrete.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt

rng = np.random.default_rng(11)
# A skewed "population" of 1 million household incomes (lognormal), in thousands
population = rng.lognormal(mean=3.5, sigma=0.6, size=1_000_000)
mu, sigma = population.mean(), population.std()
print(f"population mean = {mu:.2f}, SD = {sigma:.2f}, skew = {stats.skew(population):.2f}")

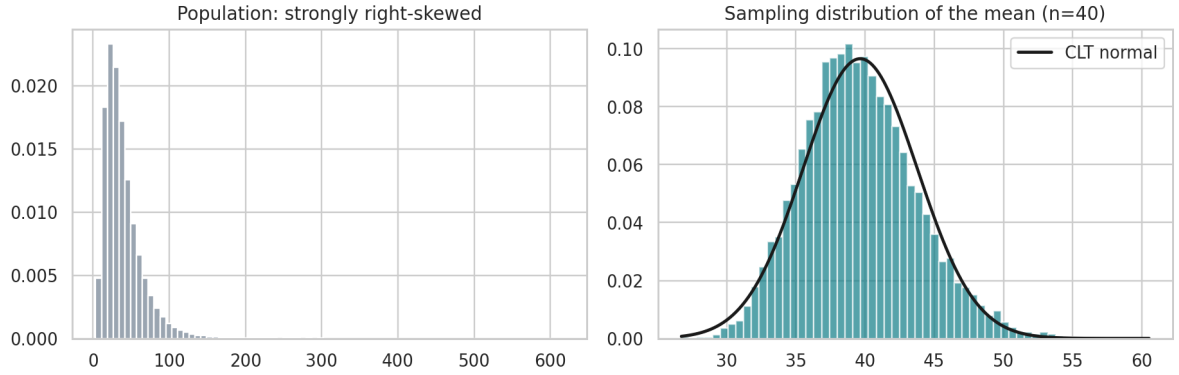
n, reps = 40, 10_000
idx = rng.integers(0, population.size, size=(reps, n))
means = population[idx].mean(axis=1)      # 10,000 sample means, each from n = 40
print(f"mean of sample means = {means.mean():.2f}  (theory: mu = {mu:.2f})")
print(f"SD of sample means   = {means.std():.3f}"
      f"  (theory: sigma/sqrt(n) = {sigma/np.sqrt(n):.3f})")
```

OUT

```
population mean = 39.65, SD = 26.12, skew = 2.28
mean of sample means = 39.60  (theory: mu = 39.65)
SD of sample means   = 4.111  (theory: sigma/sqrt(n) = 4.130)
```

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
ax[0].hist(population[:50_000], bins=80, density=True, color="#9AA5B1")
ax[0].set_title("Population: strongly right-skewed")
ax[1].hist(means, bins=60, density=True, color="#0E7C86", alpha=.7)
xs = np.linspace(means.min(), means.max(), 300)
ax[1].plot(xs, stats.norm.pdf(xs, mu, sigma / np.sqrt(n)), "k", lw=2, label="CLT normal")
ax[1].set_title(f"Sampling distribution of the mean (n={n})"); ax[1].legend()
plt.tight_layout(); plt.show()
```

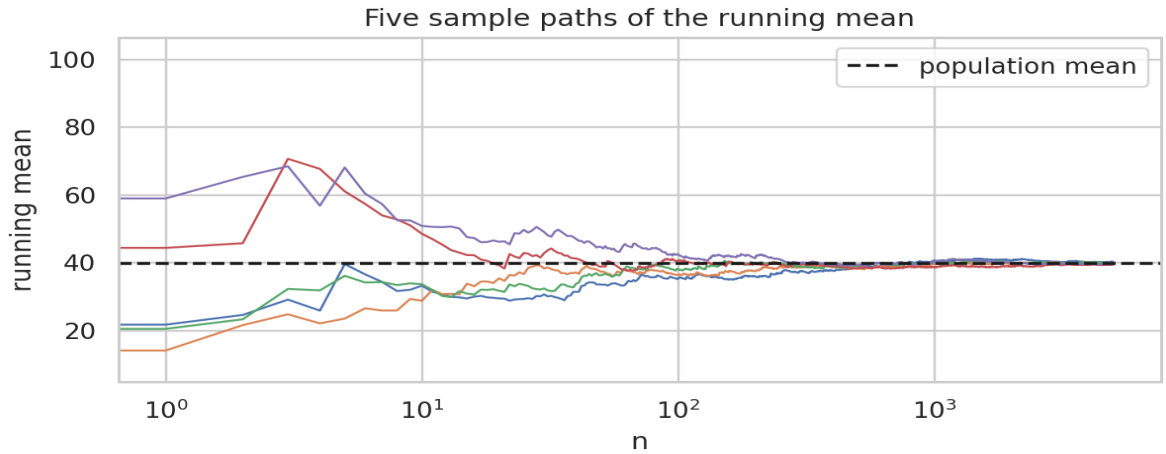


4.3 The law of large numbers

The **weak law of large numbers** says the sample mean converges in probability to μ as n grows. It is the reason that larger samples give better estimates, and the justification for Monte Carlo simulation throughout this guide. It says nothing about how fast, which is the job of the CLT and the standard error.

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(figsize=(8, 3.2))
for path in range(5):
    x = rng.choice(population, 5000)
    ax.plot(np.cumsum(x) / np.arange(1, 5001), lw=1)
ax.axhline(mu, color="k", ls="--", label="population mean")
ax.set_xscale("log"); ax.set_xlabel("n"); ax.set_ylabel("running mean")
ax.set_title("Five sample paths of the running mean"); ax.legend(); plt.show()
```



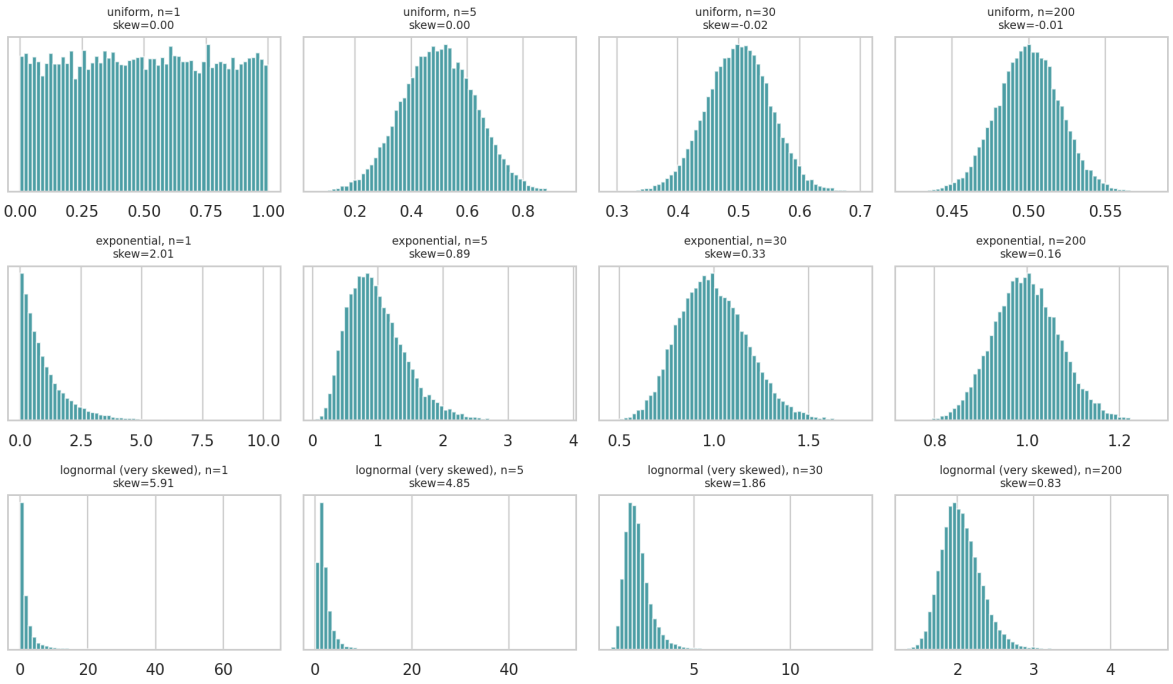
4.4 The central limit theorem

The **central limit theorem** (CLT) states that for independent draws from any population with finite variance, the standardised sample mean approaches a standard normal as n grows. This is why the normal distribution appears everywhere in inference, even when the data themselves are not normal.

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0,1) \qquad \text{so} \qquad \bar{X}_n \approx N\left(\mu, \frac{\sigma^2}{n}\right)$$

IN [Python / Jupyter cell]

```
# How large must n be? It depends on the skewness of the population.
pops = {"uniform": lambda m: rng.uniform(0, 1, m),
        "exponential": lambda m: rng.exponential(1, m),
        "lognormal (very skewed)": lambda m: rng.lognormal(0, 1.2, m)}
fig, ax = plt.subplots(3, 4, figsize=(12, 7))
for r, (name, draw) in enumerate(pops.items()):
    for c, n in enumerate([1, 5, 30, 200]):
        m = draw(20_000 * n).reshape(20_000, n).mean(axis=1)
        ax[r, c].hist(m, bins=60, density=True, color="#0E7C86", alpha=.75)
        ax[r, c].set_yticks([])
        ax[r, c].set_title(f"{name}, n={n}\nskew={stats.skew(m):.2f}", fontsize=8)
plt.tight_layout(); plt.show()
```



WATCH OUT

The often quoted 'n ≥ 30 is enough' is a rule of thumb that fails for heavily skewed populations, as the lognormal row shows. Check the skewness of your data; if it is extreme, use a transformation, a nonparametric method or the bootstrap.

4.5 Standard error and the t statistic

In practice σ is unknown, so we estimate the standard error by SE = s/√n. Replacing σ by s adds extra uncertainty, so the standardised mean no longer follows N(0, 1) but a **t distribution with n − 1 degrees of freedom** (exactly, if the data are normal; approximately otherwise). This discovery by W. S. Gosset ('Student') in 1908 is the basis of the t test.

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t_{n-1} \qquad \frac{(n-1)S^2}{\sigma^2} \sim \chi^2_{n-1}$$

IN [Python / Jupyter cell]

```
# Simulate T for small normal samples and compare with t(n-1) and N(0,1)
n, reps = 6, 100_000
samples = rng.normal(10, 3, size=(reps, n))
xbar, s = samples.mean(axis=1), samples.std(axis=1, ddof=1)
T = (xbar - 10) / (s / np.sqrt(n))
print("P(|T| > 1.96) simulated:", np.mean(np.abs(T) > 1.96).round(4))
print("  under t(5)           :", round(2 * stats.t.sf(1.96, n - 1), 4))
print("  under N(0,1)         :", round(2 * stats.norm.sf(1.96), 4),
      " <- using z would understate the tails")

# And (n-1)S^2/sigma^2 follows chi-square(n-1)
q = (n - 1) * s**2 / 9
print("mean, var of (n-1)S^2/sigma^2:", q.mean().round(3), q.var().round(3),
      "(theory", n - 1, ",", 2 * (n - 1), ")")
```

OUT

```
P(|T| > 1.96) simulated: 0.1052
  under t(5)           : 0.1073
  under N(0,1)         : 0.05  <- using z would understate the tails
mean, var of (n-1)S^2/sigma^2: 5.014 10.08 (theory 5 , 10 )
```

4.6 The bootstrap: a sampling distribution from one sample

Theory gives sampling distributions only for convenient statistics. The **bootstrap** (Efron, 1979) approximates the sampling distribution of *any* statistic by resampling the observed data with replacement, treating the sample as a stand-in for the population. It is one of the most useful tools in modern applied statistics.

IN [Python / Jupyter cell]

```
sample = rng.choice(population, 60)          # the ONE sample we actually have
B = 10_000
boot_idx = rng.integers(0, len(sample), size=(B, len(sample)))
boot_medians = np.median(sample[boot_idx], axis=1)
print(f"sample median        = {np.median(sample):.2f}")
print(f"bootstrap SE(median) = {boot_medians.std(ddof=1):.3f}")
lo, hi = np.percentile(boot_medians, [2.5, 97.5])
print(f"95% percentile bootstrap CI for the median: ({lo:.2f}, {hi:.2f})")
print(f"true population median: {np.median(population):.2f}")

# The same thing with SciPy's built-in (uses the more accurate BCa method by default)
res = stats.bootstrap((sample,), np.median, n_resamples=9999, random_state=1)
print("scipy BCa 95% CI:", np.round([res.confidence_interval.low,
                                     res.confidence_interval.high], 2))
```

OUT

```
sample median        = 31.44
bootstrap SE(median) = 3.832
95% percentile bootstrap CI for the median: (24.27, 38.92)
true population median: 33.10
scipy BCa 95% CI: [24.43 38.92]
```

4.7 Frequentist and Bayesian inference: a first look

In **frequentist** inference the parameter is fixed and the data are random; a 95% confidence interval is produced by a procedure that captures the true parameter in 95% of repeated samples. In **Bayesian** inference the parameter has a probability distribution that expresses uncertainty about it, and the data update a prior distribution into a posterior. Chapters 5 to 14 are mainly frequentist; Chapter 15 develops the Bayesian approach. For large samples and weak priors, the two typically give numerically similar answers with different interpretations.

Chapter summary

A statistic varies from sample to sample, and its sampling distribution describes that variation. The standard error is the SD of the sampling distribution and shrinks like $1/\sqrt{n}$. The LLN guarantees convergence; the CLT gives approximate normality of means; replacing σ with s gives the t distribution. When theory is unavailable, the bootstrap estimates the sampling distribution by resampling.

PRACTICAL TIP

References: Rice, *Mathematical Statistics and Data Analysis*, chapters 5 to 7; Efron & Tibshirani, *An Introduction to the Bootstrap*; Wasserman, *All of Statistics*, chapters 5 to 8.

CHAPTER 5

Estimation: Point Estimates and Confidence Intervals

An **estimator** is a rule that turns data into a guess of a parameter. A point estimate gives one number; an interval estimate gives a range with a stated confidence level. This chapter covers how to judge estimators (bias, variance, MSE, consistency, efficiency), the two classic construction methods (method of moments and maximum likelihood, previewed here and developed in Chapter 13), and how to build and correctly interpret confidence intervals.

5.1 Properties of estimators

$$\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta \qquad \text{MSE}(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2$$

Property	Definition	Example
Unbiased	E[θ] = θ: right on average	$\bar{x}$ for μ; s^2 (ddof=1) for σ^2
Consistent	θ → θ in probability as n → ∞	$\bar{x}$, s^2, most MLEs
Efficient	Smallest variance among unbiased estimators (attains the Cramér-Rao bound)	$\bar{x}$ is efficient for μ under normality
Robust	Insensitive to outliers or model misspecification	median, trimmed mean, MAD
Sufficient	Captures all information in the data about θ	Σx_i for a Poisson rate

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
rng = np.random.default_rng(5)

# Bias of the variance estimator: divide by n versus n - 1
sigma2, n, reps = 4.0, 8, 200_000
X = rng.normal(0, np.sqrt(sigma2), size=(reps, n))
v0, v1 = X.var(axis=1, ddof=0), X.var(axis=1, ddof=1)
print(f"E[var ddof=0] = {v0.mean():.3f}  (theory (n-1)/n * sigma^2 = {(n-1)/n*sigma2:.3f})")
print(f"E[var ddof=1] = {v1.mean():.3f}  (theory sigma^2 = {sigma2})")
mse = lambda est: np.mean((est - sigma2)**2)
print(f"MSE ddof=0 = {mse(v0):.3f} | MSE ddof=1 = {mse(v1):.3f}  <- biased one has LOWER MSE")
```

OUT

```
E[var ddof=0] = 3.498  (theory (n-1)/n * sigma^2 = 3.500)
E[var ddof=1] = 3.998  (theory sigma^2 = 4.0)
MSE ddof=0 = 3.752 | MSE ddof=1 = 4.572  <- biased one has LOWER MSE
```

KEY IDEA

Unbiasedness is not everything. The biased variance estimator (ddof=0) has a smaller MSE because it trades a little bias for less variance. This bias-variance trade-off reappears in ridge and lasso regression (Chapter 9) and in Bayesian shrinkage (Chapter 15).

IN [Python / Jupyter cell]

```
# Efficiency and robustness: mean versus median as estimators of the centre
def compare(draw, label, n=50, reps=50_000):
    S = draw((reps, n))
    return pd.Series({"Var(mean)": S.mean(axis=1).var(),
                      "Var(median)": np.median(S, axis=1).var()}, name=label)

tab = pd.concat([
    compare(lambda s: rng.normal(0, 1, s), "Normal"),
    compare(lambda s: rng.standard_t(3, s), "t(3), heavy tails"),
    compare(lambda s: rng.laplace(0, 1, s), "Laplace"),
], axis=1).T
tab["relative efficiency (mean/median)"] = tab["Var(median)"] / tab["Var(mean)"]
print(tab.round(4))
```

OUT

```
                   Var(mean)  Var(median)  relative efficiency (mean/median)
Normal                0.0200       0.0307                             1.5343
t(3), heavy tails     0.0591       0.0367                             0.6210
Laplace               0.0406       0.0244                             0.6004
```

Under normality the mean wins (the median needs about π/2 ≈ 1.57 times as much data for the same precision). Under heavy tails the median wins. Your choice of estimator should depend on what you believe about the tails.

5.2 Method of moments

The **method of moments** sets sample moments equal to theoretical moments and solves for the parameters. It is simple and often a good starting value for maximum likelihood. For a gamma distribution with shape k and scale θ, mean = kθ and variance = $k\theta^2$, so $\hat{\theta} = s^2/\bar{x}$ and $\hat{k} = \bar{x}^2/s^2$.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()

x = df["study_hours"].to_numpy()
k_mom = x.mean()**2 / x.var(ddof=1)
theta_mom = x.var(ddof=1) / x.mean()
```

```
k_mle, _, theta_mle = stats.gamma.fit(x, floc=0)
print(f"method of moments: shape={k_mom:.3f}, scale={theta_mom:.3f}")
print(f"maximum likelihood: shape={k_mle:.3f}, scale={theta_mle:.3f}")
print("true generating values: shape=4, scale=1.5 (before clipping)")
```

OUT

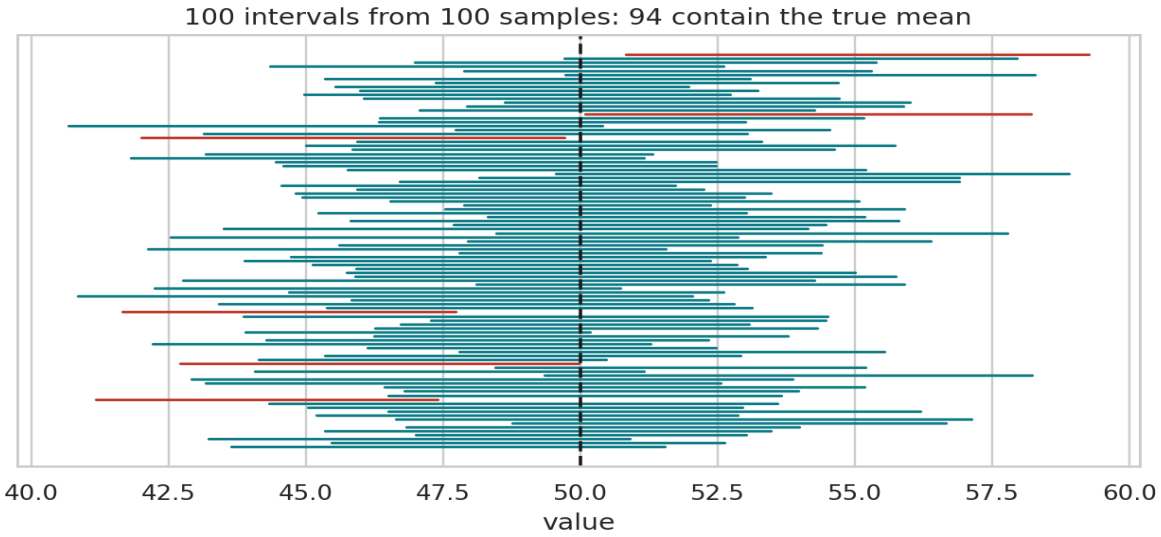
```
method of moments: shape=3.877, scale=1.503
maximum likelihood: shape=4.031, scale=1.446
true generating values: shape=4, scale=1.5 (before clipping)
```

5.3 Confidence intervals: the idea

A 95% confidence interval is produced by a method that, over repeated sampling, contains the true parameter 95% of the time. It is **not** correct to say 'there is a 95% probability that μ lies in this particular interval'; in the frequentist framework μ is fixed and the interval either contains it or not. (A Bayesian credible interval does allow that statement; see Chapter 15.) The simulation below shows the frequentist meaning directly.

IN [Python / Jupyter cell]

```
mu_true, sd_true, n = 50, 10, 25
fig, ax = plt.subplots(figsize=(9, 4.2))
covered = 0
for i in range(100):
    s = rng.normal(mu_true, sd_true, n)
    lo, hi = stats.t.interval(0.95, n - 1, loc=s.mean(), scale=stats.sem(s))
    ok = lo <= mu_true <= hi; covered += ok
    ax.plot([lo, hi], [i, i], color="#0E7C86" if ok else "#C0392B", lw=1.4)
ax.axvline(mu_true, color="k", ls="--")
ax.set_title(f"100 intervals from 100 samples: {covered} contain the true mean")
ax.set_xlabel("value"); ax.set_yticks([]); plt.show()
```



5.4 Confidence interval for a mean

For a mean with unknown σ, the interval is the estimate plus or minus a t critical value times the standard error. If σ were known (rare in practice), z would replace t.

$$\bar{x} \pm t_{n-1,\,1-\alpha/2}\,\frac{s}{\sqrt{n}}$$

IN [Python / Jupyter cell]

```
x = df["final_score"]
n, xbar, s = len(x), x.mean(), x.std(ddof=1)
se = s / np.sqrt(n)
tcrit = stats.t.ppf(0.975, df=n - 1)
print(f"by hand : {xbar:.3f} ± {tcrit:.3f} × {se:.3f} = "
      f"({xbar - tcrit*se:.3f}, {xbar + tcrit*se:.3f})")

lo, hi = stats.t.interval(0.95, df=n - 1, loc=xbar, scale=stats.sem(x))   # one line
print(f"scipy   : ({lo:.3f}, {hi:.3f})")
res = stats.ttest_1samp(x, popmean=0)                  # result objects also give CIs
ci = res.confidence_interval(0.95)
print(f"ttest_1samp: ({ci.low:.3f}, {ci.high:.3f})")

# Intervals by group
def mean_ci(s, conf=0.95):
    m, se = s.mean(), stats.sem(s)
    h = se * stats.t.ppf((1 + conf) / 2, len(s) - 1)
    return pd.Series({"n": len(s), "mean": m, "lower": m - h, "upper": m + h})
tab = df.groupby("method", observed=True)["final_score"].apply(mean_ci).unstack()
print(tab.round(2).astype({"n": int}))
```

OUT

```
by hand : 68.740 ± 1.968 × 0.584 = (67.592, 69.889)
scipy   : (67.592, 69.889)
ttest_1samp: (67.592, 69.889)
          n   mean  lower  upper
method
A       111  66.12  64.28  67.95
B       102  68.03  66.22  69.84
C        87  72.92  70.72  75.12
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import statsmodels.api as sm
from statsmodels.stats.weightstats import DescrStatsW

d = DescrStatsW(df["final_score"])
print(d.tconfint_mean(alpha=0.05))        # same interval as above
print(d.zconfint_mean(alpha=0.05))        # z version (large samples)
```

RUN THIS IN YOUR JUPYTER
Expected output: the t interval printed by SciPy above. statsmodels' DescrStatsW also supports weights, which is useful for survey data.

5.5 Confidence interval for a proportion

The textbook **Wald** interval $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$ is simple but performs badly for small n or p near 0 or 1: its real coverage can fall well below 95%. Prefer the **Wilson** score interval (good default) or the conservative exact **Clopper-Pearson** interval.

IN [Python / Jupyter cell]

```
k, n = int(df["passed"].sum()), len(df)
p_hat = k / n
z = stats.norm.ppf(0.975)
half = z * np.sqrt(p_hat * (1 - p_hat) / n)
wald = (p_hat - half, p_hat + half)
bt = stats.binomtest(k, n)
wilson = bt.proportion_ci(0.95, method="wilson")
exact = bt.proportion_ci(0.95, method="exact")          # Clopper-Pearson
print(f"p_hat = {p_hat:.4f}")
print("Wald          :", np.round(wald, 4))
print("Wilson        :", np.round([wilson.low, wilson.high], 4))
```

```
print("Clopper-Pearson:", np.round([exact.low, exact.high], 4))

# Coverage study for a small sample: why Wald is discouraged
p_true, n_small, reps = 0.08, 30, 20_000
ks = rng.binomial(n_small, p_true, reps)
ph = ks / n_small
hw = z * np.sqrt(ph * (1 - ph) / n_small)
wald_cover = np.mean((ph - hw <= p_true) & (p_true <= ph + hw))
# Wilson interval for every possible count 0..n, then look up each simulated count
wil = [stats.binomtest(kk, n_small).proportion_ci(method="wilson")
       for kk in range(n_small + 1)]
wl = np.array([c.low for c in wil])[ks]
wh = np.array([c.high for c in wil])[ks]
wilson_cover = np.mean((wl <= p_true) & (p_true <= wh))
print(f"\ncoverage with n=30, p=0.08 -> Wald: {wald_cover:.3f}, Wilson: {wilson_cover:.3f}")
```

OUT

```
p_hat = 0.7500
Wald          : [0.701 0.799]
Wilson        : [0.698  0.7956]
Clopper-Pearson: [0.697 0.798]

coverage with n=30, p=0.08 -> Wald: 0.910, Wilson: 0.972
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from statsmodels.stats.proportion import proportion_confint
proportion_confint(count=225, nobs=300, alpha=0.05, method="wilson")
proportion_confint(count=225, nobs=300, alpha=0.05, method="beta")   # Clopper-Pearson
```

RUN THIS IN YOUR JUPYTER

These reproduce the Wilson and Clopper-Pearson intervals printed above.

5.6 Confidence interval for a variance and SD

$$\left(\frac{(n-1)s^2}{\chi^2_{n-1,\,1-\alpha/2}},\ \frac{(n-1)s^2}{\chi^2_{n-1,\,\alpha/2}} \right)$$

IN [Python / Jupyter cell]

```
x = df["final_score"]; n = len(x); s2 = x.var(ddof=1)
lo = (n - 1) * s2 / stats.chi2.ppf(0.975, n - 1)
hi = (n - 1) * s2 / stats.chi2.ppf(0.025, n - 1)
print(f"variance: {s2:.2f}, 95% CI ({lo:.2f}, {hi:.2f})")
print(f"SD      : {np.sqrt(s2):.2f}, 95% CI ({np.sqrt(lo):.2f}, {np.sqrt(hi):.2f})")
```

OUT

```
variance: 102.18, 95% CI (87.59, 120.76)
SD      : 10.11, 95% CI (9.36, 10.99)
```

WATCH OUT

The chi-square interval for a variance is very sensitive to non-normality, much more so than the t interval for a mean. For non-normal data, use a bootstrap interval instead.

5.7 Confidence interval for a difference in means

For two independent groups, use the **Welch** interval, which does not assume equal variances and uses the Welch-Satterthwaite approximate degrees of freedom. The pooled (equal-variance) version is shown for completeness.

$$(\bar{x}_1 - \bar{x}_2) \pm t_\nu \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}, \quad \nu = \frac{(s_1^2/n_1 + s_2^2/n_2)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}}$$

IN [Python / Jupyter cell]

```
a = df.loc[df.method == "A", "final_score"]
c = df.loc[df.method == "C", "final_score"]
v1, v2, n1, n2 = a.var(), c.var(), len(a), len(c)
se = np.sqrt(v1 / n1 + v2 / n2)
nu = (v1 / n1 + v2 / n2)**2 / ((v1 / n1)**2 / (n1 - 1) + (v2 / n2)**2 / (n2 - 1))
diff = c.mean() - a.mean()
h = stats.t.ppf(0.975, nu) * se
print(f"C - A = {diff:.3f}, Welch df = {nu:.1f}, 95% CI ({diff - h:.3f}, {diff + h:.3f})")
res = stats.ttest_ind(c, a, equal_var=False)
print("scipy Welch CI:", np.round(res.confidence_interval(), 3), "| true effect = 7")
```

OUT

```
C - A = 6.807, Welch df = 179.6, 95% CI (3.961, 9.652)
scipy Welch CI: [3.961 9.652] | true effect = 7
```

5.8 Bootstrap confidence intervals

When no formula exists (median, correlation, ratio, a percentile) or assumptions are doubtful, bootstrap. SciPy offers three methods: **percentile** (simple), **basic**, and **BCa** (bias-corrected and accelerated, the most accurate default).

IN [Python / Jupyter cell]

```
def corr_stat(x, y):
    return stats.pearsonr(x, y)[0]
data = (df["study_hours"].to_numpy(), df["final_score"].to_numpy())
for m in ["percentile", "basic", "BCa"]:
    r = stats.bootstrap(data, corr_stat, paired=True, vectorized=False,
                        n_resamples=2000, method=m, random_state=0)
    print(f"{m:<10} 95% CI for r: ({r.confidence_interval.low:.3f}, "
          f"{r.confidence_interval.high:.3f})")
# Fisher z-transform: the classical analytic interval for a correlation
r = stats.pearsonr(*data)
print("analytic (Fisher z):", np.round(r.confidence_interval(0.95), 3))
```

OUT

```
percentile 95% CI for r: (0.579, 0.719)
basic      95% CI for r: (0.592, 0.732)
BCa        95% CI for r: (0.580, 0.720)
analytic (Fisher z): [0.586 0.716]
```

5.9 Sample size planning

To estimate a mean within a margin of error E with confidence $1 - \alpha$, you need roughly $n = (z\,\sigma / E)^2$; for a proportion, $n = z^2\, p(1-p)/E^2$, using $p = 0.5$ for the worst case. Halving the margin of error requires four times the sample size.

IN [Python / Jupyter cell]

```
import math
def n_for_mean(sigma, E, conf=0.95):
    z = stats.norm.ppf((1 + conf) / 2); return math.ceil((z * sigma / E)**2)
def n_for_prop(E, p=0.5, conf=0.95):
    z = stats.norm.ppf((1 + conf) / 2); return math.ceil(z**2 * p * (1 - p) / E**2)
print("mean, sigma=10, E=2      :", n_for_mean(10, 2))
print("mean, sigma=10, E=1      :", n_for_mean(10, 1), " <- 4x the sample")
print("poll, E=3 points, p=0.5  :", n_for_prop(0.03))
print("poll, E=3 points, 99%    :", n_for_prop(0.03, conf=0.99))
```

OUT

```
mean, sigma=10, E=2      : 97
mean, sigma=10, E=1      : 385  <- 4x the sample
poll, E=3 points, p=0.5  : 1068
poll, E=3 points, 99%    : 1844
```

Chapter summary

Judge estimators by bias, variance and MSE, not by unbiasedness alone. A confidence level describes the long-run behaviour of the procedure, not the probability for one interval. Use t intervals for means, Wilson or Clopper-Pearson for proportions, Welch for differences in means, and the bootstrap (BCa) when no reliable formula exists. Plan sample sizes in advance: precision improves only with the square root of n.

PRACTICAL TIP

References: Casella & Berger, chapters 7 and 9; Wasserman, *All of Statistics*, chapters 6 to 9; Agresti & Coull (1998), 'Approximate is better than exact for interval estimation of binomial proportions', *The American Statistician*.

CHAPTER 6

Hypothesis Testing

A hypothesis test asks whether the data are consistent with a specific claim about the population. The logic is a proof by contradiction with uncertainty: assume the claim (the null hypothesis) is true, work out how surprising the observed data would be under that assumption, and reject the claim if the data would be very surprising. This chapter builds the logic, then covers the tests you will use most, their assumptions, effect sizes, power and multiple testing.

6.1 The framework

Element	Meaning
Null hypothesis H_0	The default claim, usually 'no effect' or 'no difference' ($\mu = \mu_0$, $\mu_1 = \mu_2$).
Alternative H_1	What we conclude if H_0 is rejected. Two-sided ($\neq$) or one-sided ($<$ or $>$), chosen BEFORE seeing data.
Test statistic	A number measuring how far the data are from H_0, with a known distribution under H_0.
p-value	P(a test statistic at least as extreme as observed | H_0 true). Small p = data surprising under H_0.
Significance level α	The Type I error rate we accept, fixed in advance (commonly 0.05). Reject H_0 if $p \leq \alpha$.
Type I error	Rejecting a true H_0 (false positive). Probability α.
Type II error	Failing to reject a false H_0 (false negative). Probability β.
Power	$1 - \beta$: the probability of detecting an effect that is really there.

	H_0 true	H_0 false
Reject H_0	Type I error (α)	Correct (power, $1 - \beta$)
Do not reject H_0	Correct ($1 - \alpha$)	Type II error (β)

EXAM / VIVA ANGLE
'Fail to reject H_0' is not the same as 'accept H_0'. Absence of evidence is not evidence of absence: a non-significant result from a small study is often just low power. To claim that an effect is negligible, use an equivalence test (TOST) or report a confidence interval that excludes meaningful effects.

6.2 A p-value from first principles

A coin lands heads 61 times in 100 tosses. Is it fair? Under H_0: $p = 0.5$, simulate many experiments, and count how often the result is at least as extreme as 61 (in either direction).

IN [Python / Jupyter cell]

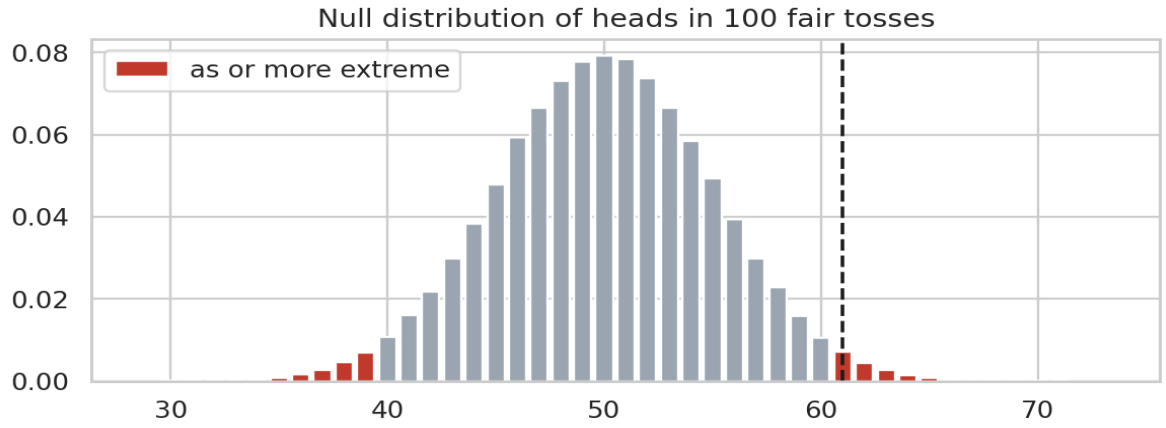
```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
rng = np.random.default_rng(6)

heads, n = 61, 100
sims = rng.binomial(n, 0.5, size=200_000)                    # world where H0 is true
p_sim = np.mean(np.abs(sims - 50) >= abs(heads - 50))        # two-sided extremeness
p_exact = stats.binomtest(heads, n, p=0.5, alternative="two-sided").pvalue
print(f"simulated p = {p_sim:.4f} | exact binomial p = {p_exact:.4f}")
```

```
plt.figure(figsize=(8, 3))
vals, cnt = np.unique(sims, return_counts=True)
extreme = np.abs(vals - 50) >= 11
plt.bar(vals[~extreme], cnt[~extreme] / len(sims), color="#9AA5B1")
plt.bar(vals[extreme], cnt[extreme] / len(sims), color="#C0392B", label="as or more extreme")
plt.axvline(heads, color="k", ls="--"); plt.legend()
plt.title("Null distribution of heads in 100 fair tosses"); plt.show()
```

OUT

```
simulated p = 0.0346 | exact binomial p = 0.0352
```



6.3 One-sample t test

Tests whether a population mean equals a hypothesised value μ_0. Assumptions: independent observations and approximately normal data (or large n, by the CLT).

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \sim t_{n-1} \text{ under } H_0$$

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()
```

```
import scipy.stats as stats
# Is the average final score different from 70, a historical benchmark?
x = df["final_score"]
t_hand = (x.mean() - 70) / (x.std(ddof=1) / np.sqrt(len(x)))
p_hand = 2 * stats.t.sf(abs(t_hand), df=len(x) - 1)
res = stats.ttest_1samp(x, popmean=70)
print(f"by hand: t = {t_hand:.4f}, p = {p_hand:.4f}")
print(f"scipy  : t = {res.statistic:.4f}, p = {res.pvalue:.4f}, df = {res.df}")
print("95% CI for mu:", np.round(res.confidence_interval(), 3))

# One-sided: is it LESS than 70?
print("one-sided p (mean < 70):",
      round(stats.ttest_1samp(x, 70, alternative="less").pvalue, 4))
```

OUT

```
by hand: t = -2.1584, p = 0.0317
scipy  : t = -2.1584, p = 0.0317, df = 299
95% CI for mu: [67.592 69.889]
one-sided p (mean < 70): 0.0158
```

Interpretation: the mean score is about 68.7, and the 95% CI excludes 70, so at $\alpha = 0.05$ we reject H_0: $\mu = 70$. Note the duality: a two-sided test at level α rejects exactly when the $(1 - \alpha)$ confidence interval excludes μ_0.

6.4 Two-sample t tests

For two independent groups, **Welch's t test** (unequal variances) should be your default. It loses almost nothing when variances are equal and protects you when they are not. Student's pooled t test assumes equal variances. Never choose between them using a preliminary variance test: that two-stage procedure distorts the error rate.

IN [Python / Jupyter cell]

```
a = df.loc[df.method == "A", "final_score"]
b = df.loc[df.method == "B", "final_score"]
welch = stats.ttest_ind(b, a, equal_var=False)
student = stats.ttest_ind(b, a, equal_var=True)
print(f"Welch  : t = {welch.statistic:.3f}, df = {welch.df:.1f}, p = {welch.pvalue:.4f}")
print(f"Student: t = {student.statistic:.3f}, df = {student.df:.0f}, p = {student.pvalue:.4f}")

# Effect size: Cohen's d (pooled SD) and Hedges' g (small-sample corrected)
def cohens_d(x, y):
    nx, ny = len(x), len(y)
    sp = np.sqrt(((nx - 1) * x.var() + (ny - 1) * y.var()) / (nx + ny - 2))
    return (x.mean() - y.mean()) / sp
d = cohens_d(b, a)
g = d * (1 - 3 / (4 * (len(a) + len(b)) - 9))
print(f"Cohen's d = {d:.3f}, Hedges' g = {g:.3f}  (0.2 small, 0.5 medium, 0.8 large)")
```

OUT

```
Welch  : t = 1.473, df = 210.8, p = 0.1423
Student: t = 1.469, df = 211, p = 0.1432
Cohen's d = 0.202, Hedges' g = 0.201  (0.2 small, 0.5 medium, 0.8 large)
```

Paired t test

When the same units are measured twice (before and after, left and right), the observations are not independent. Analyse the **differences**: a paired t test is a one-sample t test on $d_i = \text{after}_i - \text{before}_i$. Pairing removes between-subject variability and can increase power dramatically.

IN [Python / Jupyter cell]

```
# 40 students take a mock test before and after a revision workshop (true gain = 2 points)
n = 40
ability = rng.normal(65, 10, n)                 # large between-student variation
before = ability + rng.normal(0, 3, n)
after = ability + 2 + rng.normal(0, 3, n)
paired = stats.ttest_rel(after, before)
wrong = stats.ttest_ind(after, before)          # WRONG: ignores the pairing
print(f"paired t test     : t = {paired.statistic:.3f}, p = {paired.pvalue:.5f}")
print(f"independent t test: t = {wrong.statistic:.3f}, p = {wrong.pvalue:.4f}"
      "  <- loses the effect")
print(f"same as 1-sample on differences: p = {stats.ttest_1samp(after - before, 0).pvalue:.5f}")
```

OUT

```
paired t test     : t = 2.083, p = 0.04390
independent t test: t = 0.644, p = 0.5213  <- loses the effect
same as 1-sample on differences: p = 0.04390
```

6.5 Tests for proportions and categorical data

One and two proportions

IN [Python / Jupyter cell]

```
# One proportion: is the pass rate different from 70%?
k, n = int(df.passed.sum()), len(df)
print("exact binomial test p =", round(stats.binomtest(k, n, p=0.70).pvalue, 4))

# Two proportions: pass rate in method A versus method B (z test, pooled under H0)
ka, na = df.loc[df.method == "A", "passed"].agg(["sum", "count"])
kb, nb = df.loc[df.method == "B", "passed"].agg(["sum", "count"])
pa, pb, pp = ka / na, kb / nb, (ka + kb) / (na + nb)
z = (pa - pb) / np.sqrt(pp * (1 - pp) * (1 / na + 1 / nb))
print(f"pA = {pa:.3f}, pB = {pb:.3f}, z = {z:.3f}, p = {2 * stats.norm.sf(abs(z)):.4f}")
```

OUT

```
exact binomial test p = 0.0589
pA = 0.802, pB = 0.696, z = 1.783, p = 0.0746
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from statsmodels.stats.proportion import proportions_ztest
count = np.array([89, 71]); nobs = np.array([111, 102])
z, p = proportions_ztest(count, nobs)        # matches the hand-computed z and p above
```

RUN THIS IN YOUR JUPYTER

Gives the same z and p as the manual two-proportion test.

Chi-square test of independence

For two categorical variables, compare observed counts O with the counts E expected under independence (row total × column total / grand total). The test requires expected counts of about 5 or more in each cell; otherwise use Fisher's exact test (2 × 2 tables). Report **Cramér's V** as the effect size.

$$\chi^2 = \sum_{\text{cells}} \frac{(O-E)^2}{E} \sim \chi^2_{(r-1)(c-1)} \qquad V = \sqrt{\frac{\chi^2}{n\,(\min(r,c)-1)}}$$

IN [Python / Jupyter cell]

```
table = pd.crosstab(df["method"], df["passed"])
chi2, p, dof, expected = stats.chi2_contingency(table)
print("observed:\n", table)
print("expected under independence:\n", pd.DataFrame(expected, index=table.index,
                                                     columns=table.columns).round(1))
V = np.sqrt(chi2 / (table.values.sum() * (min(table.shape) - 1)))
print(f"chi2 = {chi2:.3f}, df = {dof}, p = {p:.4f}, Cramer's V = {V:.3f}")
print("scipy Cramer's V:", round(stats.contingency.association(table, method="cramer"), 3))

# Standardised residuals show WHICH cells drive the result (|r| > 2 is notable)
print("standardised residuals:\n", ((table - expected) / np.sqrt(expected)).round(2))
```

OUT

```
observed:
 passed   0   1
method
A       22  89
B       31  71
C       22  65
expected under independence:
 passed     0     1
method
A       27.8  83.2
B       25.5  76.5
C       21.8  65.2
chi2 = 3.174, df = 2, p = 0.2045, Cramer's V = 0.103
scipy Cramer's V: 0.103
standardised residuals:
 passed     0     1
method
A      -1.09  0.63
B       1.09 -0.63
C       0.05 -0.03
```

IN [Python / Jupyter cell]

```
# Fisher's exact test for a small 2x2 table
small = np.array([[8, 2], [1, 5]])
odds, p_f = stats.fisher_exact(small)
print(f"Fisher exact: odds ratio = {odds:.2f}, p = {p_f:.4f}")

# Chi-square goodness of fit: is a die fair? (observed counts from 120 rolls)
obs = np.array([25, 17, 15, 23, 24, 16])
res = stats.chisquare(obs, f_exp=np.full(6, obs.sum() / 6))
print(f"goodness of fit: chi2 = {res.statistic:.3f}, p = {res.pvalue:.4f}")
```

OUT

```
Fisher exact: odds ratio = 20.00, p = 0.0350
goodness of fit: chi2 = 5.000, p = 0.4159
```

6.6 Checking assumptions: normality and equal variances

Test	H_0	scipy	Notes
Shapiro-Wilk	Data are normal	`stats.shapiro`	Most powerful general normality test; n up to ~5000.
D'Agostino-Pearson	Normal (skew and kurtosis)	`stats.normaltest`	Needs n ≥ 20; tells you why via skew/kurtosis.
Anderson-Darling	Specified distribution	`stats.anderson`	Weights the tails; gives critical values.
Levene / Brown-Forsythe	Equal variances	`stats.levene(center='median')`	Robust to non-normality; preferred.

Test	H_0	scipy	Notes
Bartlett	Equal variances	`stats.bartlett`	Sensitive to non-normality; use only if data normal.

IN [Python / Jupyter cell]

```
for col in ["final_score", "study_hours"]:
    sw = stats.shapiro(df[col]); nt = stats.normaltest(df[col])
    print(f"{col:<12} Shapiro W={sw.statistic:.4f} p={sw.pvalue:.4f} | "
          f"D'Agostino p={nt.pvalue:.4f}")
ad = stats.anderson(df["final_score"], dist="norm")
print("Anderson-Darling stat:", round(ad.statistic, 3),
      "| 5% critical value:", ad.critical_values[2])

groups = [g["final_score"].to_numpy() for _, g in df.groupby("method", observed=True)]
lev = stats.levene(*groups, center="median"); bart = stats.bartlett(*groups)
print(f"Levene (median-centred): W = {lev.statistic:.3f}, p = {lev.pvalue:.4f}")
print(f"Bartlett               : T = {bart.statistic:.3f}, p = {bart.pvalue:.4f}")
```

OUT

```
final_score  Shapiro W=0.9857 p=0.0046 | D'Agostino p=0.0048
study_hours  Shapiro W=0.9265 p=0.0000 | D'Agostino p=0.0000
Anderson-Darling stat: 0.971 | 5% critical value: 0.75
Levene (median-centred): W = 0.467, p = 0.6276
Bartlett               : T = 1.147, p = 0.5634
```

WATCH OUT

With large samples, normality tests reject for trivial departures that do not matter; with small samples, they lack the power to detect serious ones. Treat them as a supplement to Q-Q plots, never as a substitute.

6.7 Nonparametric tests

Nonparametric (rank-based) tests drop the normality assumption. They test slightly different hypotheses: the Mann-Whitney U test asks whether a random observation from one group tends to be larger than one from the other ($P(X > Y) = 0.5$ under H_0), which equals a test of medians only when the two distributions have the same shape.

Parametric test	Nonparametric counterpart	scipy
One-sample / paired t	Wilcoxon signed-rank (or the sign test)	`stats.wilcoxon`
Two-sample t	Mann-Whitney U (Wilcoxon rank-sum)	`stats.mannwhitneyu`
One-way ANOVA	Kruskal-Wallis H	`stats.kruskal`
Repeated-measures ANOVA	Friedman	`stats.friedmanchisquare`
Pearson correlation	Spearman ρ / Kendall τ	`stats.spearmanr / kendalltau`

IN [Python / Jupyter cell]

```
mw = stats.mannwhitneyu(b, a, alternative="two-sided")
wx = stats.wilcoxon(after, before)
print(f"Mann-Whitney B vs A : U = {mw.statistic:.0f}, p = {mw.pvalue:.4f}")
print(f"Wilcoxon paired     : W = {wx.statistic:.0f}, p = {wx.pvalue:.4f}")
# Sign test = binomial test on the number of positive differences
diffs = after - before
print("sign test p         :", round(stats.binomtest(int((diffs > 0).sum()),
                                                     int((diffs != 0).sum())).pvalue, 5))
# Rank-biserial correlation: effect size for Mann-Whitney
U = stats.mannwhitneyu(b, a).statistic
print("rank-biserial r     :", round(2 * U / (len(a) * len(b)) - 1, 3))
```

OUT

```
Mann-Whitney B vs A : U = 6339, p = 0.1316
Wilcoxon paired     : W = 250, p = 0.0310
sign test p         : 0.01659
rank-biserial r     : 0.12
```

Permutation tests: the most general test of all

If H_0 says group labels do not matter, then shuffling the labels should not change the statistic's distribution. A **permutation test** computes the statistic under many random relabellings and compares the observed value with this null distribution. It works for any statistic and needs only exchangeability.

IN [Python / Jupyter cell]

```
def diff_in_means(x, y, axis):
    return np.mean(x, axis=axis) - np.mean(y, axis=axis)
res = stats.permutation_test((b.to_numpy(), a.to_numpy()), diff_in_means,
                             permutation_type="independent", n_resamples=20_000,
                             vectorized=True, random_state=0)
print(f"observed diff = {res.statistic:.3f}, permutation p = {res.pvalue:.4f}")
print(f"(compare Welch p = {welch.pvalue:.4f})")
```

OUT

```
observed diff = 1.915, permutation p = 0.1470
(compare Welch p = 0.1423)
```

6.8 Power and sample size

Power depends on four quantities: the effect size, the sample size, α, and the noise (SD). Fix three and you can solve for the fourth. Power analysis belongs *before* data collection; computing 'observed power' after a non-significant result is uninformative (it is a function of the p-value).

IN [Python / Jupyter cell]

```
def power_two_sample(d, n_per_group, alpha=0.05):
    """Exact power of the two-sided two-sample t test via the noncentral t."""
    df_ = 2 * n_per_group - 2
    nc = d * np.sqrt(n_per_group / 2)                # noncentrality parameter
    tcrit = stats.t.ppf(1 - alpha / 2, df_)
    return stats.nct.sf(tcrit, df_, nc) + stats.nct.cdf(-tcrit, df_, nc)

def n_for_power(d, power=0.8, alpha=0.05):
    n = 2
    while power_two_sample(d, n, alpha) < power: n += 1
    return n

for d in (0.2, 0.5, 0.8):
    print(f"d = {d}: n per group for 80% power = {n_for_power(d)}")

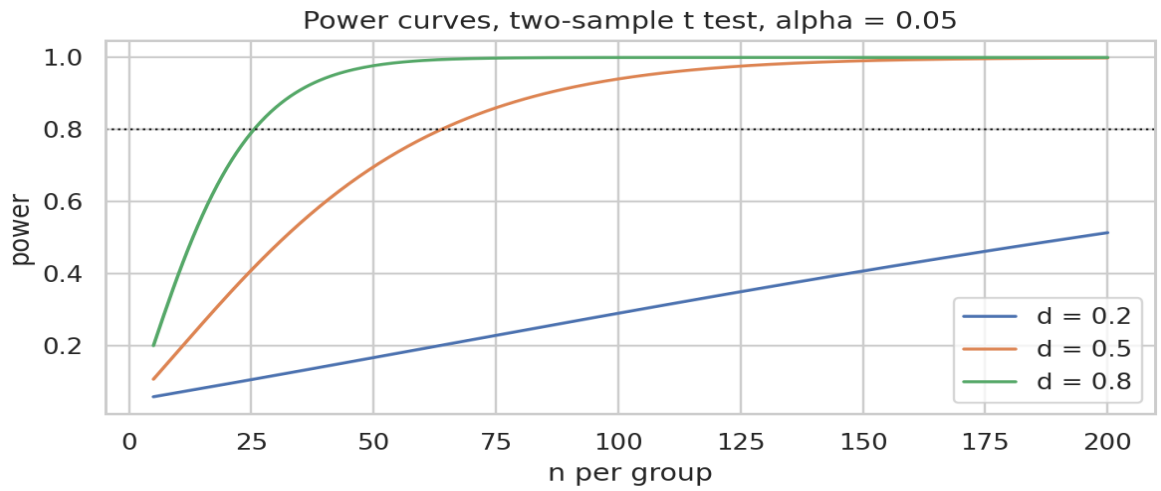
# Check one case by simulation
d, n_g, hits = 0.5, 64, 0
for _ in range(5000):
    x1 = rng.normal(0, 1, n_g); x2 = rng.normal(d, 1, n_g)
    hits += stats.ttest_ind(x1, x2).pvalue < 0.05
print(f"simulated power, d=0.5, n=64: {hits / 5000:.3f}"
      f" | exact: {power_two_sample(.5, 64):.3f}")
```

OUT

```
d = 0.2: n per group for 80% power = 394
d = 0.5: n per group for 80% power = 64
d = 0.8: n per group for 80% power = 26
simulated power, d=0.5, n=64: 0.795 | exact: 0.801
```

IN [Python / Jupyter cell]

```
ns = np.arange(5, 201)
plt.figure(figsize=(8, 3.4))
for d in (0.2, 0.5, 0.8):
    plt.plot(ns, [power_two_sample(d, n) for n in ns], label=f"d = {d}")
plt.axhline(0.8, color="k", ls=":", lw=1)
plt.xlabel("n per group"); plt.ylabel("power"); plt.legend()
plt.title("Power curves, two-sample t test, alpha = 0.05"); plt.show()
```



IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from statsmodels.stats.power import TTestIndPower
analysis = TTestIndPower()
analysis.solve_power(effect_size=0.5, alpha=0.05, power=0.8)    # -> about 63.8 per group
analysis.power(effect_size=0.5, nobs1=64, alpha=0.05)           # -> about 0.80
```

RUN THIS IN YOUR JUPYTER

Matches the noncentral-t function above: about 64 per group for d = 0.5.

6.9 Multiple testing

Run 20 independent tests at α = 0.05 on pure noise and the chance of at least one 'significant' result is $1 - 0.95^{20} \approx 64\%$. Corrections control either the **family-wise error rate** (FWER: probability of any false positive; Bonferroni, Holm) or the **false discovery rate** (FDR: expected share of false positives among discoveries; Benjamini-Hochberg). Use FWER control for a few confirmatory tests and FDR for large screening problems such as genomics.

IN [Python / Jupyter cell]

```
m, m_true = 100, 10                          # 100 tests, 10 real effects
pvals = []
for j in range(m):
    effect = 0.8 if j < m_true else 0.0
    pvals.append(stats.ttest_ind(rng.normal(effect, 1, 30), rng.normal(0, 1, 30)).pvalue)
pvals = np.array(pvals); truth = np.arange(m) < m_true

def holm(p, alpha=0.05):
    order = np.argsort(p); rej = np.zeros(len(p), bool)
    for rank, i in enumerate(order):
        if p[i] <= alpha / (len(p) - rank): rej[i] = True
        else: break
    return rej

rules = {"uncorrected": pvals <= 0.05,
```

```
        "Bonferroni": pvals <= 0.05 / m,
        "Holm": holm(pvals),
        "Benjamini-Hochberg": stats.false_discovery_control(pvals, method="bh") <= 0.05}
for name, rej in rules.items():
    print(f"{name:<20} discoveries={rej.sum():>3}  true={np.sum(rej & truth):>2}  "
          f"false={np.sum(rej & ~truth):>2}")
```

OUT

```
uncorrected          discoveries= 14  true= 7  false= 7
Bonferroni           discoveries=  2  true= 2  false= 0
Holm                 discoveries=  2  true= 2  false= 0
Benjamini-Hochberg   discoveries=  3  true= 3  false= 0
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from statsmodels.stats.multitest import multipletests
reject, p_adj, _, _ = multipletests(pvals, alpha=0.05, method="holm")      # or "fdr_bh"
```

RUN THIS IN YOUR JUPYTER

Methods include 'bonferroni', 'holm', 'fdr_bh' (Benjamini-Hochberg) and 'fdr_by'. Results match the executed cell above.

6.10 Reporting and interpreting tests well

The American Statistical Association's 2016 statement on p-values makes six points worth memorising: p-values indicate incompatibility of data with a model; they do not measure the probability that the hypothesis is true; decisions should not rest on p crossing 0.05 alone; proper inference needs full reporting and transparency; a p-value does not measure the size or importance of an effect; and by itself a p-value is not a good measure of evidence. In practice: report the estimate, its confidence interval, the effect size and the exact p-value, and state the assumptions you checked.

The cell below generates a complete, publication-style sentence directly from the results, which is a good habit: the numbers in your report can never drift from the numbers in your code.

IN [Python / Jupyter cell]

```
ci = welch.confidence_interval(0.95)
print(f"Students taught with method B scored on average {b.mean() - a.mean():.1f} points "
      f"higher than those taught with method A\n(95% CI {ci.low:.1f} to {ci.high:.1f}; "
      f"Welch t({welch.df:.1f}) = {welch.statistic:.2f}, p = {welch.pvalue:.3f}; "
      f"Hedges' g = {g:.2f}).\nThe data are consistent with no difference and with a "
      f"small benefit of B.")
```

OUT

```
Students taught with method B scored on average 1.9 points higher than those taught with method A
(95% CI -0.6 to 4.5; Welch t(210.8) = 1.47, p = 0.142; Hedges' g = 0.20).
The data are consistent with no difference and with a small benefit of B.
```

Chapter summary

State H_0 and H_1 before seeing the data; compute a test statistic; compare it with its null distribution. Use Welch by default for two groups, paired tests for paired data, chi-square or Fisher for categorical data, and rank or permutation tests when assumptions are doubtful. Always pair p-values with effect sizes and confidence intervals, plan power in advance, and correct for multiple comparisons.

PRACTICAL TIP

References: Casella & Berger, chapter 8; Wasserman, *All of Statistics*, chapter 10; Wasserstein & Lazar (2016), 'The ASA statement on p-values', *The American Statistician*; Cohen, *Statistical Power Analysis for the Behavioral Sciences*; Agresti, *Categorical Data Analysis*.

CHAPTER 7

Analysis of Variance (ANOVA)

ANOVA, developed by R. A. Fisher in the 1920s for agricultural experiments, compares the means of three or more groups with a single test. Running all pairwise t tests instead would inflate the Type I error rate (Chapter 6.9). The key insight is in the name: we decide whether means differ by comparing two estimates of *variance*, one from between the groups and one from within them.

7.1 The idea: partitioning variability

Total variability around the grand mean splits exactly into variability **between** group means (signal) and variability **within** groups (noise). If H_0: $\mu_1 = \mu_2 = \ldots = \mu_k$ is true, both mean squares estimate the same σ^2, so their ratio F should be near 1. A large F means group means differ more than noise alone would explain.

$$SS_T = \sum_{i,j}(y_{ij} - \bar{y})^2 \;=\; SS_B + SS_W \;=\; \sum_j n_j(\bar{y}_j - \bar{y})^2 + \sum_{i,j}(y_{ij} - \bar{y}_j)^2$$

$$F = \frac{MS_B}{MS_W} = \frac{SS_B/(k-1)}{SS_W/(N-k)} \;\sim\; F_{k-1,\,N-k} \text{ under } H_0$$

Source	SS	df	MS	F
Between groups (treatment)	SS_B	$k - 1$	$SS_B/(k - 1)$	MS_B/MS_W
Within groups (error)	SS_W	$N - k$	$SS_W/(N - k)$	
Total	SS_T	$N - 1$		

7.2 One-way ANOVA from scratch and with SciPy

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})
```

```
df = make_students()

import scipy.stats as stats
import matplotlib.pyplot as plt

def one_way_anova(data, group, y):
    grand = data[y].mean()
    g = data.groupby(group, observed=True)[y]
    n_j, m_j = g.size(), g.mean()
    ss_b = np.sum(n_j * (m_j - grand)**2)
    ss_w = np.sum((data[y] - g.transform("mean"))**2)
    k, N = len(n_j), len(data)
    df_b, df_w = k - 1, N - k
    F = (ss_b / df_b) / (ss_w / df_w)
    p = stats.f.sf(F, df_b, df_w)
    table = pd.DataFrame({"SS": [ss_b, ss_w, ss_b + ss_w], "df": [df_b, df_w, N - 1],
                          "MS": [ss_b / df_b, ss_w / df_w, np.nan],
                          "F": [F, np.nan, np.nan], "p": [p, np.nan, np.nan]},
                         index=[group, "Residual", "Total"])
    return table

tab = one_way_anova(df, "method", "final_score")
print(tab.to_string(na_rep="", float_format=lambda v: f"{v:.5g}"))

groups = [g.to_numpy() for _, g in df.groupby("method", observed=True)["final_score"]]
res = stats.f_oneway(*groups)
print(f"\nscipy f_oneway: F = {res.statistic:.4f}, p = {res.pvalue:.3e}")
```

OUT

```
             SS   df     MS      F          p
method   2337.5    2 1168.7 12.303 7.3596e-06
Residual  28215  297 94.999
Total     30552  299

scipy f_oneway: F = 12.3026, p = 7.360e-06
```

Interpretation: F(2, 297) is large and p is tiny, so at least one teaching method has a different mean score. ANOVA does not tell us *which* methods differ; that is the job of post-hoc tests (7.5).

Effect sizes

$\eta^2 = SS_B/SS_T$ is the share of variance explained by the grouping, but it is biased upward. ω^2 corrects this bias and is preferred for reporting. Rough guide: 0.01 small, 0.06 medium, 0.14 large.

$$\eta^2 = \frac{SS_B}{SS_T} \qquad \omega^2 = \frac{SS_B - (k-1)MS_W}{SS_T + MS_W}$$

IN [Python / Jupyter cell]

```
ss_b, ss_w, ss_t = tab.loc["method", "SS"], tab.loc["Residual", "SS"], tab.loc["Total", "SS"]
ms_w, k = tab.loc["Residual", "MS"], 3
eta2 = ss_b / ss_t
omega2 = (ss_b - (k - 1) * ms_w) / (ss_t + ms_w)
print(f"eta^2 = {eta2:.4f}, omega^2 = {omega2:.4f}")

# With two groups, ANOVA F equals the pooled t statistic squared
a, c = groups[0], groups[2]
t = stats.ttest_ind(a, c).statistic; F2 = stats.f_oneway(a, c).statistic
print(f"t^2 = {t**2:.4f}  equals  F = {F2:.4f}")
```

OUT

```
eta^2 = 0.0765, omega^2 = 0.0701
t^2 = 22.5957  equals  F = 22.5957
```

7.3 Assumptions and diagnostics

One-way ANOVA assumes (1) independent observations, (2) normally distributed residuals within each group, and (3) equal variances across groups (homoscedasticity). ANOVA is fairly robust to non-normality with balanced groups of moderate size, but unequal variances with unequal group sizes can badly distort the F test. Check the residuals, not the raw outcome.

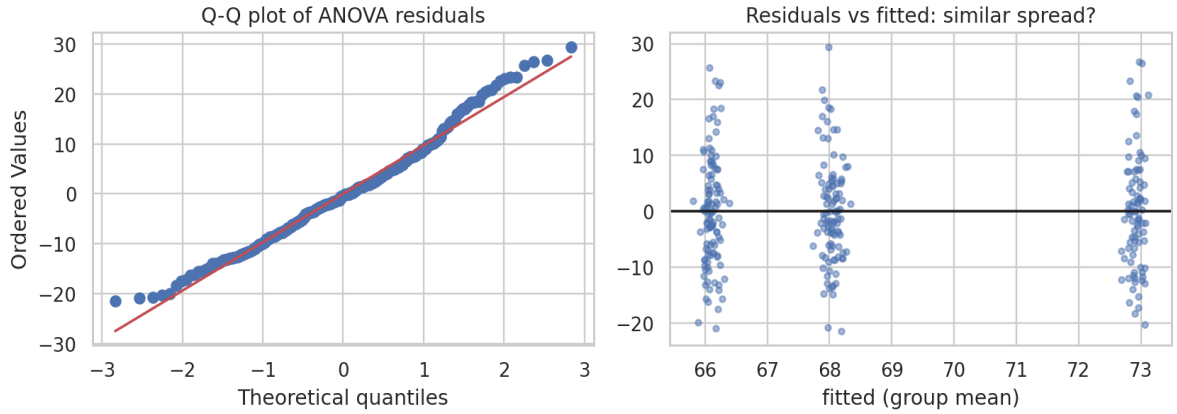
IN [Python / Jupyter cell]

```
resid = df["final_score"] - df.groupby("method", observed=True)["final_score"].transform("mean")
print("Shapiro on residuals :", round(stats.shapiro(resid).pvalue, 4))
print("Levene (Brown-Forsythe):", round(stats.levene(*groups, center="median").pvalue, 4))
print("group SDs            :", [round(float(g.std(ddof=1)), 2) for g in groups])

fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
stats.probplot(resid, plot=ax[0]); ax[0].set_title("Q-Q plot of ANOVA residuals")
fitted = df.groupby("method", observed=True)["final_score"].transform("mean")
ax[1].scatter(fitted + np.random.default_rng(0).normal(0, .1, len(df)), resid, alpha=.5, s=12)
ax[1].axhline(0, color="k"); ax[1].set_xlabel("fitted (group mean)")
ax[1].set_title("Residuals vs fitted: similar spread?"); plt.tight_layout(); plt.show()
```

OUT

```
Shapiro on residuals : 0.0019
Levene (Brown-Forsythe): 0.6276
group SDs            : [9.74, 9.24, 10.32]
```



Variances look equal (Levene p is large), but Shapiro-Wilk rejects normality of the residuals. The reason is instructive: these residuals still contain the effect of the right-skewed study hours variable, which the one-way model ignores. Once study hours enter the model (ANCOVA, section 7.7, and regression, Chapter 9), the residuals become close to normal. Non-normal residuals are often a sign of a missing predictor, not a reason to abandon the model.

Welch's ANOVA for unequal variances

When variances differ, use **Welch's ANOVA**, which weights each group by n_j/s_j^2. Just as with the two-sample case, many statisticians recommend it as the default. SciPy 1.15+ provides it via `equal_var=False`; the formula is also implemented below so you can see what it does.

IN [Python / Jupyter cell]

```
def welch_anova(*gs):
    k = len(gs)
    n = np.array([len(g) for g in gs]); m = np.array([np.mean(g) for g in gs])
    v = np.array([np.var(g, ddof=1) for g in gs])
    w = n / v; mw = np.sum(w * m) / np.sum(w)
    A = np.sum(w * (m - mw)**2) / (k - 1)
    lam = np.sum((1 - w / w.sum())**2 / (n - 1))
    B = 1 + 2 * (k - 2) / (k**2 - 1) * lam
    F = A / B; df1, df2 = k - 1, (k**2 - 1) / (3 * lam)
```

```
    return F, df1, df2, stats.f.sf(F, df1, df2)

F, d1, d2, p = welch_anova(*groups)
print(f"Welch ANOVA (manual): F({d1}, {d2:.1f}) = {F:.3f}, p = {p:.2e}")
res_w = stats.f_oneway(*groups, equal_var=False)
print(f"scipy equal_var=False: F = {res_w.statistic:.3f}, p = {res_w.pvalue:.2e}")
```

OUT

```
Welch ANOVA (manual): F(2, 191.6) = 11.392, p = 2.12e-05
scipy equal_var=False: F = 11.392, p = 2.12e-05
```

7.4 Nonparametric alternative: Kruskal-Wallis

IN [Python / Jupyter cell]

```
H = stats.kruskal(*groups)
eps2 = (H.statistic - len(groups) + 1) / (len(df) - len(groups))   # epsilon-squared effect size
print(f"Kruskal-Wallis H = {H.statistic:.3f}, p = {H.pvalue:.2e}, epsilon^2 = {eps2:.3f}")
```

OUT

```
Kruskal-Wallis H = 21.976, p = 1.69e-05, epsilon^2 = 0.067
```

7.5 Post-hoc comparisons

After a significant F, compare pairs while controlling the family-wise error rate. **Tukey's HSD** is the standard choice for all pairwise comparisons with equal variances. **Games-Howell** is its unequal-variance counterpart. **Dunnett's test** compares every group with a single control and is more powerful when that is the question. Bonferroni-adjusted t tests are simple but conservative.

IN [Python / Jupyter cell]

```
labels = ["A", "B", "C"]
tk = stats.tukey_hsd(*groups)
print(tk)                                          # pairwise differences, p-values, CIs
ci = tk.confidence_interval(0.95)
for i in range(3):
    for j in range(i + 1, 3):
        print(f"{labels[j]} - {labels[i]}: diff = {-tk.statistic[i, j]:6.3f}, "
              f"95% CI ({-ci.high[i, j]:6.3f}, {-ci.low[i, j]:6.3f}), "
              f"p = {tk.pvalue[i, j]:.4f}")
```

OUT

```
Pairwise Group Comparisons (95.0% Confidence Interval)
Comparison  Statistic  p-value  Lower CI  Upper CI
 (0 - 1)     -1.915     0.326    -5.064     1.234
 (0 - 2)     -6.807     0.000   -10.094    -3.519
 (1 - 0)      1.915     0.326    -1.234     5.064
 (1 - 2)     -4.891     0.002    -8.242    -1.541
 (2 - 0)      6.807     0.000     3.519    10.094
 (2 - 1)      4.891     0.002     1.541     8.242

B - A: diff =  1.915, 95% CI (-1.234,  5.064), p = 0.3255
C - A: diff =  6.807, 95% CI ( 3.519, 10.094), p = 0.0000
C - B: diff =  4.891, 95% CI ( 1.541,  8.242), p = 0.0019
```

IN [Python / Jupyter cell]

```
# Dunnett: B and C each compared with method A as the control
dn = stats.dunnett(groups[1], groups[2], control=groups[0])
for lab, s, p in zip(["B vs A", "C vs A"], dn.statistic, dn.pvalue):
    print(f"Dunnett {lab}: t = {s:.3f}, p = {p:.4f}")
```

OUT

```
Dunnett B vs A: t = 1.433, p = 0.2644
Dunnett C vs A: t = 4.877, p = 0.0000
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import pingouin as pg
pg.welch_anova(data=df, dv="final_score", between="method")
pg.pairwise_gameshowell(data=df, dv="final_score", between="method")

from statsmodels.stats.multicomp import pairwise_tukeyhsd
print(pairwise_tukeyhsd(df["final_score"], df["method"], alpha=0.05))
```

RUN THIS IN YOUR JUPYTER
pingouin gives Welch ANOVA and Games-Howell in tidy tables; statsmodels' pairwise_tukeyhsd reproduces the SciPy Tukey results above.

Planned contrasts

If you have a specific hypothesis before seeing data, test it directly with a **contrast** (weights summing to zero) instead of running all pairs. Example: is the average of the new methods B and C better than the old method A? Contrast weights (−1, ½, ½).

$$\hat{L} = \sum_j c_j \bar{y}_j, \qquad SE(\hat{L}) = \sqrt{MS_W \sum_j c_j^2 / n_j}, \qquad t = \hat{L}/SE \sim t_{N-k}$$

IN [Python / Jupyter cell]

```
c_w = np.array([-1, 0.5, 0.5])
means = np.array([g.mean() for g in groups]); ns = np.array([len(g) for g in groups])
L = c_w @ means
se = np.sqrt(ms_w * np.sum(c_w**2 / ns))
t_c = L / se
print(f"contrast estimate = {L:.3f}, t({len(df) - 3}) = {t_c:.3f}, "
      f"p = {2 * stats.t.sf(abs(t_c), len(df) - 3):.2e}")
```

OUT

```
contrast estimate = 4.361, t(297) = 3.737, p = 2.23e-04
```

7.6 Two-way ANOVA and interactions

Two-way ANOVA studies two factors at once and, crucially, their **interaction**: whether the effect of one factor depends on the level of the other. We use a classic balanced factorial experiment: crop yield under 3 fertilisers × 2 irrigation regimes, 10 plots per cell, generated with a real interaction (fertiliser F3 works especially well with irrigation).

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijk}, \qquad \varepsilon_{ijk} \sim N(0, \sigma^2)$$

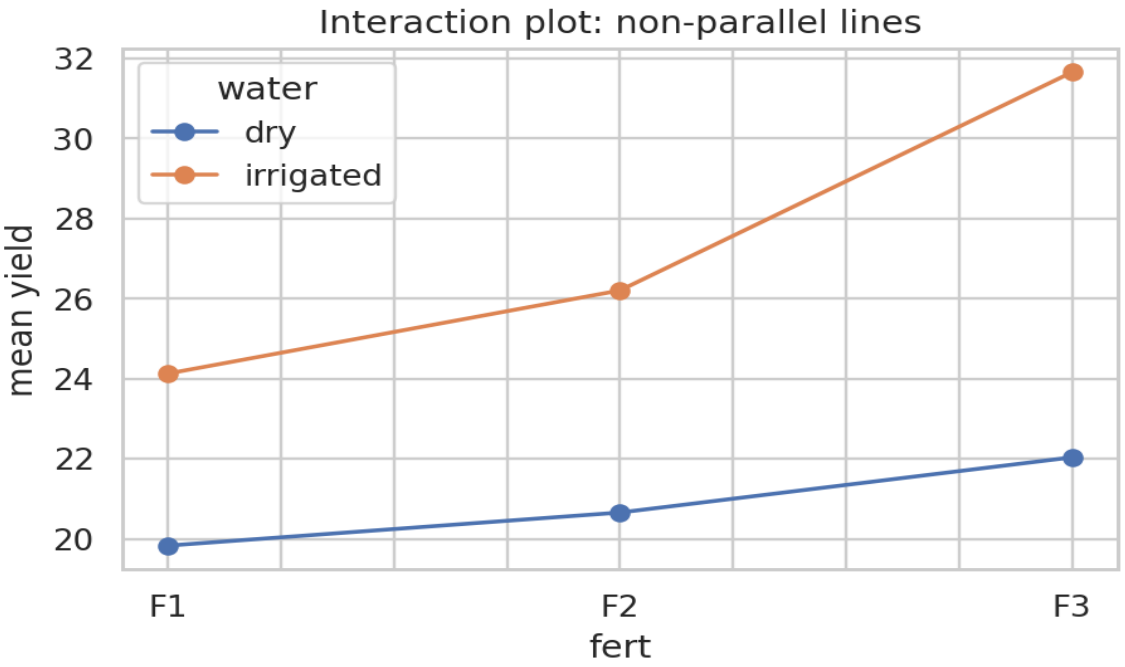
IN [Python / Jupyter cell]

```
rng = np.random.default_rng(77)
rows = []
cell_means = {("F1", "dry"): 20, ("F1", "irrigated"): 24,
              ("F2", "dry"): 22, ("F2", "irrigated"): 26,
              ("F3", "dry"): 23, ("F3", "irrigated"): 31}      # F3 x irrigated: interaction
for (f, w), mu in cell_means.items():
    for y in rng.normal(mu, 2.5, 10):
        rows.append({"fert": f, "water": w, "yield_": y})
crop = pd.DataFrame(rows)
print(crop.pivot_table(index="fert", columns="water", values="yield_", aggfunc="mean").round(2))

means = crop.groupby(["fert", "water"])["yield_"].mean().unstack()
means.plot(marker="o", figsize=(6, 3.6), title="Interaction plot: non-parallel lines")
plt.ylabel("mean yield"); plt.show()
```

OUT

```
water    dry  irrigated
fert
F1     19.83      24.13
F2     20.66      26.21
F3     22.04      31.66
```



Parallel lines would mean no interaction. Here the lines diverge (they would be parallel if irrigation added the same amount for every fertiliser), which the formal test should confirm. We compute the two-way ANOVA table from scratch using the general linear model: each sum of squares is the drop in residual sum of squares when a term is added to the model. This 'model comparison' view is exactly how statsmodels computes ANOVA tables, and it extends to unbalanced data.

IN [Python / Jupyter cell]

```
def design(data, terms):
    """Build a dummy-coded design matrix for main effects and interactions."""
    X = [np.ones(len(data))]
    for t in terms:
        if "*" in t:                              # interaction of two factors
            f1, f2 = t.split("*")
            d1 = pd.get_dummies(data[f1], drop_first=True, dtype=float)
            d2 = pd.get_dummies(data[f2], drop_first=True, dtype=float)
            for c1 in d1:
                for c2 in d2: X.append((d1[c1] * d2[c2]).to_numpy())
        else:
            X += [col.to_numpy() for _, col in
                  pd.get_dummies(data[t], drop_first=True, dtype=float).items()]
    return np.column_stack(X)

def rss(data, y, terms):
    X = design(data, terms)
    beta, *_ = np.linalg.lstsq(X, data[y].to_numpy(), rcond=None)
    r = data[y].to_numpy() - X @ beta
    return r @ r, X.shape[1]

y = "yield_"
rss_full, p_full = rss(crop, y, ["fert", "water", "fert*water"])
df_res = len(crop) - p_full; ms_res = rss_full / df_res
# Type II sums of squares: each main effect adjusted for the other main effect
```

```
comparisons = {"fert": (["water"], ["fert", "water"]),
               "water": (["fert"], ["fert", "water"]),
               "fert:water": (["fert", "water"], ["fert", "water", "fert*water"])}
out = {}
for term, (reduced, bigger) in comparisons.items():
    r0, p0 = rss(crop, y, reduced); r1, p1 = rss(crop, y, bigger)
    ss, dfn = r0 - r1, p1 - p0
    F = (ss / dfn) / ms_res
    out[term] = [ss, dfn, F, stats.f.sf(F, dfn, df_res)]
out["Residual"] = [rss_full, df_res, np.nan, np.nan]
aov = pd.DataFrame(out, index=["sum_sq", "df", "F", "PR(>F)"]).T
print(aov.round(4).to_string(na_rep=""))
```

OUT

```
              sum_sq    df        F  PR(>F)
fert        249.7949   2.0  19.3886  0.0000
water       631.5916   1.0  98.0457  0.0000
fert:water   77.5206   2.0   6.0170  0.0044
Residual    347.8577  54.0
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

model = smf.ols("yield_ ~ C(fert) * C(water)", data=crop).fit()
print(anova_lm(model, typ=2))        # Type II table: matches the executed table above
print(model.summary())               # the same model viewed as a regression

# One-way ANOVA via statsmodels, for comparison with section 7.2
print(anova_lm(smf.ols("final_score ~ C(method)", data=df).fit()))
```

RUN THIS IN YOUR JUPYTER

anova_lm(typ=2) returns the same sum_sq, df, F and PR(>F) as the from-scratch Type II table. Use typ=3 with sum-to-zero coding, C(fert, Sum) * C(water, Sum), if you need Type III.

EXAM / VIVA ANGLE

Type I, II or III sums of squares? With balanced data they coincide. With unbalanced data: Type I is sequential and depends on term order; Type II tests each main effect adjusted for the others and is the most powerful choice when there is no interaction; Type III tests each effect adjusted for all others including interactions and requires sum-to-zero contrasts. When an interaction is significant, interpret simple effects (the effect of one factor at each level of the other) rather than main effects.

IN [Python / Jupyter cell]

```
# Simple effects: the irrigation effect within each fertiliser
for f, g in crop.groupby("fert"):
    r = stats.ttest_ind(g.loc[g.water == "irrigated", y], g.loc[g.water == "dry", y])
    diff = g.loc[g.water == "irrigated", y].mean() - g.loc[g.water == "dry", y].mean()
    print(f"{f}: irrigation effect = {diff:5.2f}, t = {r.statistic:5.2f}, p = {r.pvalue:.2e}")
```

OUT

```
F1: irrigation effect =  4.30, t =  3.15, p = 5.54e-03
F2: irrigation effect =  5.55, t =  4.94, p = 1.07e-04
F3: irrigation effect =  9.62, t = 11.18, p = 1.57e-09
```

7.7 ANCOVA: adjusting for a covariate

Analysis of covariance adds a continuous covariate to ANOVA. Two benefits: it removes covariate-related noise (more power) and it adjusts group comparisons for imbalance in the covariate. Here we compare teaching methods after adjusting for study hours, prior GPA, sleep and attendance. The method effect estimates should move toward the true values (B = 4, C = 7).

IN [Python / Jupyter cell]

```
def ols_fit(X, y):
    beta, *_ = np.linalg.lstsq(X, y, rcond=None); r = y - X @ beta
    return beta, r @ r

yv = df["final_score"].to_numpy()
dums = pd.get_dummies(df["method"], drop_first=True, dtype=float).to_numpy()
covs = df[["study_hours", "sleep_hours", "attendance", "prior_gpa"]].to_numpy()
one = np.ones((len(df), 1))
b_full, rss_f = ols_fit(np.hstack([one, dums, covs]), yv)
b_red, rss_r = ols_fit(np.hstack([one, covs]), yv)
df_res = len(df) - (1 + 2 + 4)
F = ((rss_r - rss_f) / 2) / (rss_f / df_res)
print(f"ANCOVA F test for method (adjusted): F(2, {df_res}) = {F:.2f}, "
      f"p = {stats.f.sf(F, 2, df_res):.2e}")
print(f"adjusted effects: B - A = {b_full[1]:.2f}, C - A = {b_full[2]:.2f}  (truth 4 and 7)")
raw = df.groupby("method", observed=True)["final_score"].mean()
print(f"unadjusted:       B - A = {raw['B'] - raw['A']:.2f}, C - A = {raw['C'] - raw['A']:.2f}")
```

OUT

```
ANCOVA F test for method (adjusted): F(2, 293) = 38.30, p = 1.67e-15
adjusted effects: B - A = 3.67, C - A = 7.66  (truth 4 and 7)
unadjusted:       B - A = 1.92, C - A = 6.81
```

Adjustment brought the estimates closer to the truth, because method A students happened to study slightly more (Chapter 1.6), which inflated A's raw mean. Residual error also shrinks because the covariates explain much of the variation.

7.8 Repeated measures

When the same subjects are measured under every condition, use repeated-measures ANOVA (or a mixed model). The nonparametric counterpart is the Friedman test.

IN [Python / Jupyter cell]

```
# 15 subjects, each tested under 3 conditions
subj = rng.normal(0, 4, 15)[:, None]
scores = 50 + subj + np.array([0, 1.5, 3.0]) + rng.normal(0, 1.5, (15, 3))
fr = stats.friedmanchisquare(scores[:, 0], scores[:, 1], scores[:, 2])
print(f"Friedman chi2 = {fr.statistic:.3f}, p = {fr.pvalue:.4f}")
long = pd.DataFrame({"subject": np.repeat(np.arange(15), 3),
                     "cond": np.tile(["c1", "c2", "c3"], 15), "score": scores.ravel()})
print(long.head(6))
```

OUT

```
Friedman chi2 = 22.533, p = 0.0000
   subject cond    score
0        0   c1  46.7840
1        0   c2  49.9069
2        0   c3  52.0303
3        1   c1  49.7868
4        1   c2  50.9223
5        1   c3  54.8499
```

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from statsmodels.stats.anova import AnovaRM
print(AnovaRM(data=long, depvar="score", subject="subject", within=["cond"]).fit())

# A linear mixed model handles missing data and unbalanced designs better
mixed = smf.mixedlm("score ~ C(cond)", data=long, groups=long["subject"]).fit()
print(mixed.summary())
```

RUN THIS IN YOUR JUPYTER

AnovaRM requires a complete, balanced long-format table (one row per subject per condition).

Chapter summary

ANOVA compares between-group and within-group variance with an F test. Report η^2 or ω^2, check residual normality and equal variances, and switch to Welch's ANOVA or Kruskal-Wallis when assumptions fail. Follow a significant F with Tukey, Games-Howell or Dunnett, or better, test planned contrasts. In factorial designs, examine interactions first; with unbalanced data, choose Type II or III sums of squares deliberately. ANOVA is a special case of the linear model, the subject of the next two chapters.

PRACTICAL TIP

References: Montgomery, *Design and Analysis of Experiments* (chapters 3 to 5); Maxwell, Delaney & Kelley, *Designing Experiments and Analyzing Data*; Kutner et al., *Applied Linear Statistical Models*, parts IV and V.

CHAPTER 8

Simple Linear Regression

Regression models how the mean of a response Y changes with a predictor X. Simple linear regression (one predictor, straight line) is the gateway to almost all of statistical modelling: multiple regression, ANOVA, logistic regression and many time series models are extensions of the ideas in this chapter. Master the estimation, the inference, and above all the diagnostics here, and the rest becomes much easier.

8.1 The model and its assumptions

$$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad \varepsilon_i \overset{iid}{\sim} N(0, \sigma^2)$$

β_0 is the intercept (mean of Y when x = 0), β_1 the slope (change in the mean of Y for a one-unit increase in x) and σ^2 the error variance. The assumptions are remembered as **LINE**: **L**inearity of the mean function, **I**ndependence of errors, **N**ormality of errors, and **E**qual variance (homoscedasticity). Only linearity and independence are needed for unbiased estimates; normality and equal variance matter for exact standard errors, tests and prediction intervals.

8.2 Least squares estimation

Ordinary least squares (OLS) chooses the line that minimises the sum of squared residuals. Setting the derivatives to zero gives closed-form estimates. Under the normal model, OLS is also the maximum likelihood estimator (Chapter 13), and by the Gauss-Markov theorem it is the best linear unbiased estimator even without normality.

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = r\frac{s_y}{s_x} \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

$$\hat{\sigma}^2 = s^2 = \frac{SSE}{n-2} = \frac{\sum (y_i - \hat{y}_i)^2}{n-2}$$

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
```

```
                        "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                        "attendance": attend.round(1), "prior_gpa": prior.round(2),
                        "final_score": score.round(1), "passed": passed})

df = make_students()

import scipy.stats as stats
import matplotlib.pyplot as plt

x = df["study_hours"].to_numpy(); y = df["final_score"].to_numpy(); n = len(x)
Sxx = np.sum((x - x.mean())**2)
Sxy = np.sum((x - x.mean()) * (y - y.mean()))
b1 = Sxy / Sxx
b0 = y.mean() - b1 * x.mean()
yhat = b0 + b1 * x
resid = y - yhat
s2 = np.sum(resid**2) / (n - 2)
print(f"by hand : intercept = {b0:.4f}, slope = {b1:.4f}, sigma_hat = {np.sqrt(s2):.4f}")

# Three library routes to the same line
print("np.polyfit      :", np.round(np.polyfit(x, y, deg=1)[::-1], 4))
lr = stats.linregress(x, y)
print(f"stats.linregress: intercept = {lr.intercept:.4f}, slope = {lr.slope:.4f}, "
      f"r = {lr.rvalue:.4f}, p = {lr.pvalue:.2e}")
from sklearn.linear_model import LinearRegression
sk = LinearRegression().fit(x.reshape(-1, 1), y)       # sklearn needs a 2-D X
print(f"sklearn         : intercept = {sk.intercept_:.4f}, slope = {sk.coef_[0]:.4f}")
```

OUT

```
by hand : intercept = 55.6903, slope = 2.2396, sigma_hat = 7.6453
np.polyfit      : [55.6903  2.2396]
stats.linregress: intercept = 55.6903, slope = 2.2396, r = 0.6557, p = 3.05e-38
sklearn         : intercept = 55.6903, slope = 2.2396
```

Interpretation: each extra hour of weekly study is associated with about 2.2 more points on the final exam, on average. The true value used to generate the data was 2.5; the estimate is within sampling error (see the confidence interval below). The intercept, the predicted score for zero study hours, is an extrapolation since nobody in the data studies less than 0.5 hours, so it should not be over-interpreted.

8.3 Inference for the coefficients

$$SE(\hat\beta_1) = \frac{s}{\sqrt{S_{xx}}}, \quad SE(\hat\beta_0) = s\sqrt{\frac{1}{n} + \frac{\bar{x}^2}{S_{xx}}}, \quad t = \frac{\hat\beta_1 - 0}{SE(\hat\beta_1)} \sim t_{n-2}$$

IN [Python / Jupyter cell]

```
s = np.sqrt(s2)
se_b1 = s / np.sqrt(Sxx)
se_b0 = s * np.sqrt(1 / n + x.mean()**2 / Sxx)
tcrit = stats.t.ppf(0.975, n - 2)
coef = pd.DataFrame({
    "coef": [b0, b1], "std_err": [se_b0, se_b1],
    "t": [b0 / se_b0, b1 / se_b1],
    "p": [2 * stats.t.sf(abs(b0 / se_b0), n - 2), 2 * stats.t.sf(abs(b1 / se_b1), n - 2)],
    "ci_low": [b0 - tcrit * se_b0, b1 - tcrit * se_b1],
    "ci_high": [b0 + tcrit * se_b0, b1 + tcrit * se_b1]}, index=["Intercept", "study_hours"])
print(coef.to_string(float_format=lambda v: f"{v:.4g}"))
print("\nlinregress SE(slope):", round(lr.stderr, 4),
      "| SE(intercept):", round(lr.intercept_stderr, 4))
```

OUT

```
              coef  std_err     t         p  ci_low  ci_high
Intercept    55.69   0.9761 57.05 1.99e-162   53.77    57.61
study_hours  2.24   0.1494 14.99  3.05e-38   1.946    2.534
```

```
linregress SE(slope): 0.1494 | SE(intercept): 0.9761
```

8.4 Goodness of fit: R² and the ANOVA decomposition

The total variation in y splits into the part explained by the regression (SSR) and the residual part (SSE). R^2 = SSR/SST is the proportion of variance explained; in simple regression, $R^2 = r^2$. The regression F test of H_0: $\beta_1 = 0$ is equivalent to the t test ($F = t^2$).

$$SST = SSR + SSE, \qquad R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}, \qquad F = \frac{SSR/1}{SSE/(n-2)}$$

IN [Python / Jupyter cell]

```
SST = np.sum((y - y.mean())**2); SSE = np.sum(resid**2); SSR = SST - SSE
R2 = SSR / SST; F = (SSR / 1) / (SSE / (n - 2))
print(f"R^2 = {R2:.4f} (r^2 = {lr.rvalue**2:.4f}) | F = {F:.3f} = t^2 = {(b1 / se_b1)**2:.3f}")
print(f"residual standard error = {s:.3f} points  <- typical size of a prediction error")
```

OUT

```
R^2 = 0.4299 (r^2 = 0.4299) | F = 224.702 = t^2 = 224.702
residual standard error = 7.645 points  <- typical size of a prediction error
```

KEY IDEA

R^2 = 0.43 means study hours explain 43% of the variation in scores; the other 57% comes from sleep, prior GPA, method and noise. A low R^2 does not mean the slope is wrong or unimportant, and a high R^2 does not mean the model is correct or causal.

8.5 Confidence and prediction intervals

Two different questions: (1) what is the **mean** score of all students who study x_0 hours (confidence interval for E[Y | x_0]), and (2) what score will **one new** student who studies x_0 hours get (prediction interval)? The prediction interval adds the irreducible individual noise σ^2, so it is always wider, and it does not shrink to zero as n grows.

$$\hat{y}_0 \pm t_{n-2}\, s\sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}} \quad \text{(mean)} \qquad \hat{y}_0 \pm t_{n-2}\, s\sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}} \quad \text{(new)}$$

IN [Python / Jupyter cell]

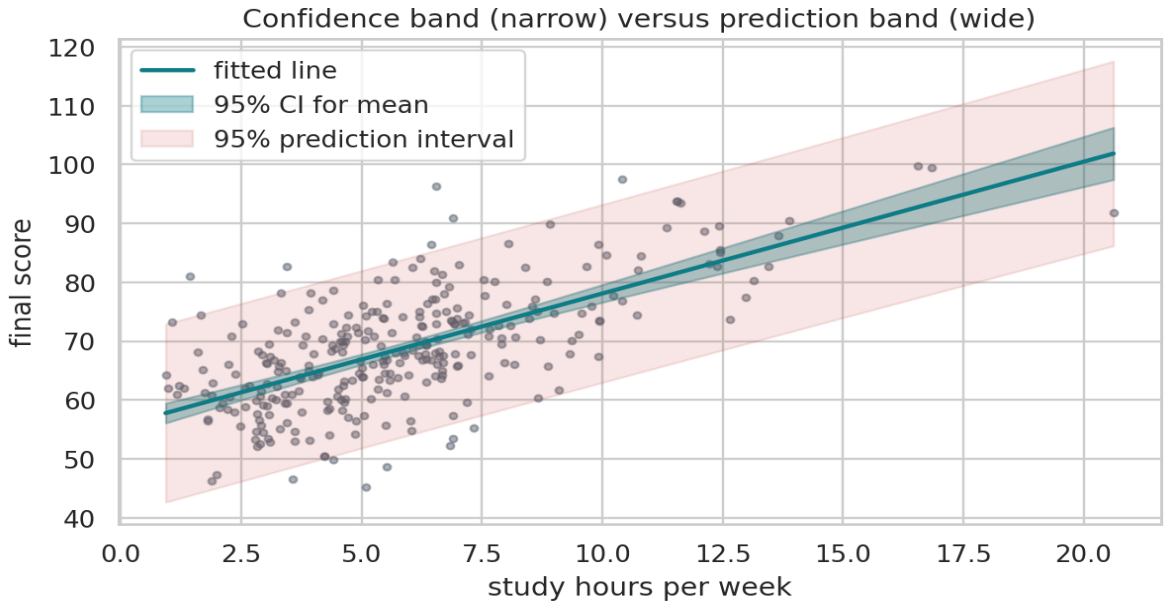
```
def intervals(x0):
    y0 = b0 + b1 * x0
    se_mean = s * np.sqrt(1 / n + (x0 - x.mean())**2 / Sxx)
    se_pred = s * np.sqrt(1 + 1 / n + (x0 - x.mean())**2 / Sxx)
    return (y0, y0 - tcrit * se_mean, y0 + tcrit * se_mean,
            y0 - tcrit * se_pred, y0 + tcrit * se_pred)

for x0 in (2, 6, 15):
    y0, cl, ch, pl, ph = intervals(x0)
    print(f"x0={x0:>2}: fit={y0:6.2f} | mean CI ({cl:6.2f}, {ch:6.2f}) | "
          f"prediction ({pl:6.2f}, {ph:6.2f})")

grid = np.linspace(x.min(), x.max(), 200)
_, cl, ch, pl, ph = intervals(grid)
plt.figure(figsize=(8, 4.2))
plt.scatter(x, y, s=10, alpha=.5, color="#5B6573")
plt.plot(grid, b0 + b1 * grid, color="#0E7C86", lw=2, label="fitted line")
plt.fill_between(grid, cl, ch, alpha=.35, color="#0E7C86", label="95% CI for mean")
plt.fill_between(grid, pl, ph, alpha=.12, color="#C0392B", label="95% prediction interval")
plt.xlabel("study hours per week"); plt.ylabel("final score"); plt.legend()
plt.title("Confidence band (narrow) versus prediction band (wide)"); plt.show()
```

OUT

```
x0= 2: fit= 60.17 | mean CI ( 58.75,  61.59) | prediction ( 45.06,  75.28)
x0= 6: fit= 69.13 | mean CI ( 68.26,  70.00) | prediction ( 54.06,  84.20)
x0=15: fit= 89.28 | mean CI ( 86.45,  92.12) | prediction ( 73.97, 104.59)
```



Both bands are narrowest at $\bar{x}$ and widen toward the extremes, because uncertainty in the slope matters more far from the centre of the data. Extrapolating beyond the range of x is risky for a second reason: the linear form may simply not hold there.

8.6 Regression diagnostics

Regression output is only as trustworthy as its assumptions. The four standard diagnostic plots check them visually. Look for: a curve in residuals versus fitted (non-linearity), a funnel (heteroscedasticity), departures from the line in the Q-Q plot (non-normality), and points with high leverage and large residuals (influence).

IN [Python / Jupyter cell]

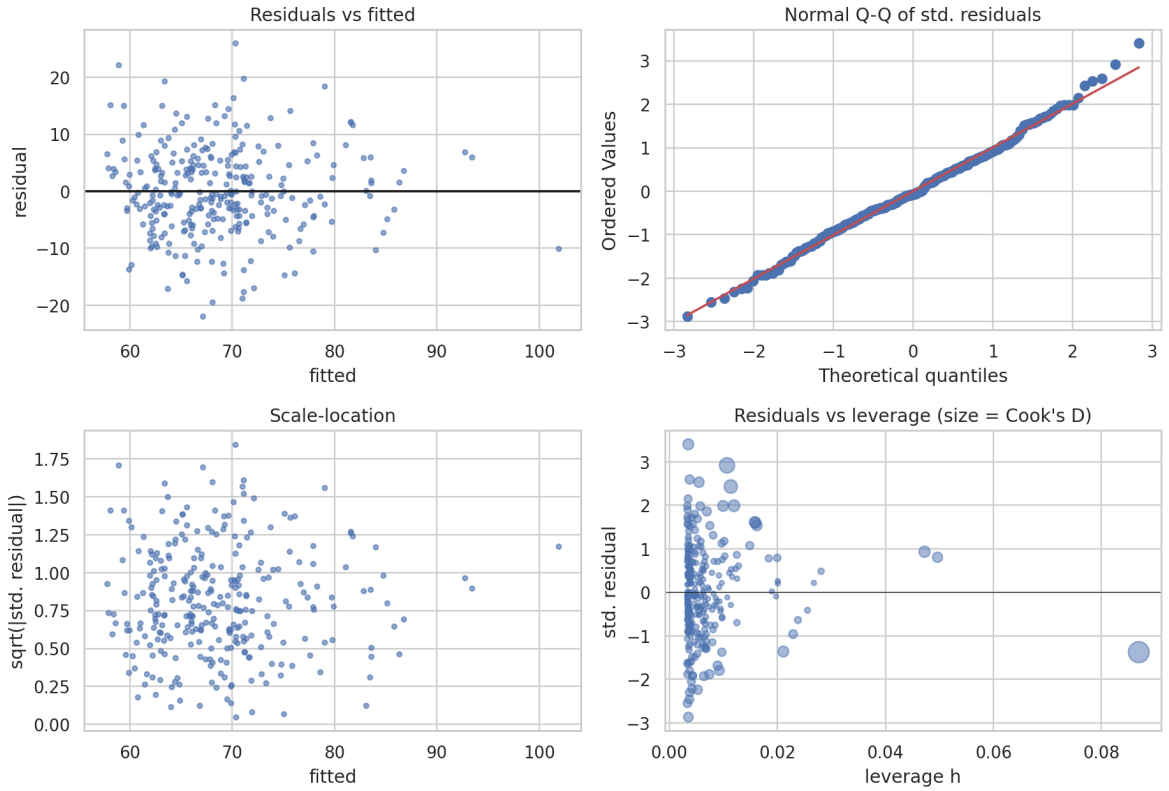
```
# Leverage, studentized residuals, Cook's distance (closed forms for simple regression)
h = 1 / n + (x - x.mean())**2 / Sxx                 # leverage: distance of x_i from x-bar
r_std = resid / (s * np.sqrt(1 - h))                # internally studentized residuals
cooks = r_std**2 / 2 * h / (1 - h)                  # p = 2 parameters

fig, ax = plt.subplots(2, 2, figsize=(11, 7.5))
ax[0, 0].scatter(yhat, resid, s=10, alpha=.6); ax[0, 0].axhline(0, color="k")
ax[0, 0].set(title="Residuals vs fitted", xlabel="fitted", ylabel="residual")
stats.probplot(r_std, plot=ax[0, 1]); ax[0, 1].set_title("Normal Q-Q of std. residuals")
ax[1, 0].scatter(yhat, np.sqrt(np.abs(r_std)), s=10, alpha=.6)
ax[1, 0].set(title="Scale-location", xlabel="fitted", ylabel="sqrt(|std. residual|)")
ax[1, 1].scatter(h, r_std, s=10 + 2000 * cooks, alpha=.5)
ax[1, 1].axhline(0, color="k", lw=.5)
ax[1, 1].set(title="Residuals vs leverage (size = Cook's D)", xlabel="leverage h",
             ylabel="std. residual")
plt.tight_layout(); plt.show()

print("high leverage (h > 4/n):", np.sum(h > 4 / n),
      "| |std resid| > 3:", np.sum(np.abs(r_std) > 3),
      "| Cook's D > 4/n:", np.sum(cooks > 4 / n))
print("most influential obs:", np.argsort(cooks)[-3:][::-1],
      "with D =", np.sort(cooks)[-3:][::-1].round(4))
```

OUT

```
high leverage (h > 4/n): 19 | |std resid| > 3: 1 | Cook's D > 4/n: 15
most influential obs: [208  16 243] with D = [0.0904 0.046  0.034 ]
```



Diagnostic	What it detects	Rule of thumb
Residuals vs fitted	Non-linearity; missing predictors	Should be a random cloud around 0
Q-Q of residuals	Non-normal errors, outliers	Points on the line
Scale-location	Heteroscedasticity	Flat trend; no funnel
Leverage h_{ii}	Unusual x values	h > 2p/n or 4/n is high
Studentized residual	Outliers in y	|r| > 3 suspicious
Cook's distance	Influence on the whole fit	D > 4/n worth a look; D > 1 serious

Formal tests: Breusch-Pagan and Durbin-Watson

IN [Python / Jupyter cell]

```
# Breusch-Pagan (Koenker version): regress squared residuals on x; LM = n R^2 ~ chi2(1)
e2 = resid**2
lr_aux = stats.linregress(x, e2)
LM = n * lr_aux.rvalue**2
print(f"Breusch-Pagan LM = {LM:.3f}, p = {stats.chi2.sf(LM, 1):.4f}  (H0: constant variance)")

# Durbin-Watson: autocorrelation of residuals in ORDERED data (about 2 = none)
dw = np.sum(np.diff(resid)**2) / np.sum(resid**2)
print(f"Durbin-Watson = {dw:.3f}")
print(f"Shapiro-Wilk on residuals p = {stats.shapiro(resid).pvalue:.4f}")
```

OUT

```
Breusch-Pagan LM = 0.025, p = 0.8743  (H0: constant variance)
Durbin-Watson = 2.038
Shapiro-Wilk on residuals p = 0.5059
```

8.7 Influence in action

A single high-leverage point can drag the line. Adding one student who studied 30 hours but scored 40 shows why diagnostics matter, and why robust alternatives exist.

IN [Python / Jupyter cell]

```
xo = np.append(x, 30); yo = np.append(y, 40)
fit_o = stats.linregress(xo, yo)
print(f"slope without the point: {b1:.3f} | with it: {fit_o.slope:.3f}")

# A robust alternative: Theil-Sen (median of pairwise slopes) resists outliers
ts = stats.theilslopes(yo, xo)
print(f"Theil-Sen slope with the point: {ts.slope:.3f} (95% CI {ts.low_slope:.3f} to "
      f"{ts.high_slope:.3f})")
from sklearn.linear_model import HuberRegressor
hub = HuberRegressor().fit(xo.reshape(-1, 1), yo)
print(f"Huber regression slope        : {hub.coef_[0]:.3f}")
```

OUT

```
slope without the point: 2.240 | with it: 1.616
Theil-Sen slope with the point: 2.208 (95% CI 1.897 to 2.522)
Huber regression slope        : 2.157
```

8.8 Transformations and interpreting log models

When the relationship is curved or the variance grows with the mean, transform. The interpretation of the slope changes with the transformation, and examiners ask about it constantly.

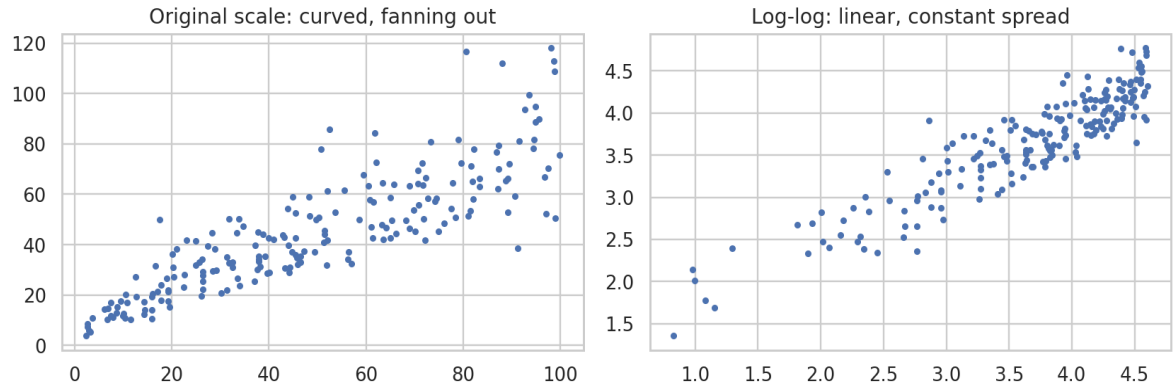
Model	Equation	Interpretation of β_1
Level-level	$y = \beta_0 + \beta_1 x$	+1 unit in x → $+\beta_1$ units in y
Log-level	$\ln y = \beta_0 + \beta_1 x$	+1 unit in x → y × $e^{\beta 1}$ (≈ $100\beta_1$% change)
Level-log	$y = \beta_0 + \beta_1 \ln x$	+1% in x → $+\beta_1/100$ units in y
Log-log	$\ln y = \beta_0 + \beta_1 \ln x$	+1% in x → $+\beta_1$% in y (elasticity)

IN [Python / Jupyter cell]

```
# Example: a power law y = 3 x^0.7 with multiplicative noise
rng = np.random.default_rng(8)
xp = rng.uniform(1, 100, 200)
yp = 3 * xp**0.7 * np.exp(rng.normal(0, .25, 200))
lin = stats.linregress(xp, yp); loglog = stats.linregress(np.log(xp), np.log(yp))
print(f"level-level R^2 = {lin.rvalue**2:.3f} | log-log R^2 = {loglog.rvalue**2:.3f}")
print(f"estimated elasticity = {loglog.slope:.3f} (true 0.7); "
      f"scale = {np.exp(loglog.intercept):.3f} (true 3)")
fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
ax[0].scatter(xp, yp, s=8); ax[0].set_title("Original scale: curved, fanning out")
ax[1].scatter(np.log(xp), np.log(yp), s=8); ax[1].set_title("Log-log: linear, constant spread")
plt.tight_layout(); plt.show()
```

OUT

```
level-level R^2 = 0.734 | log-log R^2 = 0.835
estimated elasticity = 0.696 (true 0.7); scale = 3.014 (true 3)
```



WATCH OUT
Back-transforming predictions from a log model gives the **median**, not the mean, of y. For the mean, multiply by $\exp(s^2/2)$ (the lognormal correction) or use Duan's smearing estimator.

8.9 Regression with statsmodels

For everyday statistical work, statsmodels' formula interface gives the full inference table in one line. The formula syntax `'y ~ x'` (borrowed from R via the patsy/formulaic libraries) automatically adds an intercept.

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import statsmodels.formula.api as smf
model = smf.ols("final_score ~ study_hours", data=df).fit()
print(model.summary())          # coefficients, SEs, t, p, CIs, R2, F, AIC/BIC, diagnostics

model.params, model.bse, model.conf_int(), model.rsquared   # individual pieces
pred = model.get_prediction(pd.DataFrame({"study_hours": [2, 6, 15]}))
print(pred.summary_frame(alpha=0.05))   # mean_ci_* and obs_ci_* (prediction) columns

infl = model.get_influence()
infl.summary_frame()[["hat_diag", "student_resid", "cooks_d"]].sort_values("cooks_d").tail()

from statsmodels.stats.diagnostic import het_breuschpagan
het_breuschpagan(model.resid, model.model.exog)   # (LM, LM p, F, F p)
```

RUN THIS IN YOUR JUPYTER
The coefficient table, intervals and diagnostics match the executed from-scratch results in sections 8.2 to 8.6 (intercept and slope, SEs, $R^2 = 0.43$, the mean and prediction intervals, leverage and Cook's distance).

8.10 Correlation, causation and regression to the mean

A regression slope measures association. Interpreting it causally requires that x was randomly assigned or that all confounders are controlled, which is the motivation for multiple regression (next chapter). A related trap is **regression to the mean**: because the slope in standard units is r, which is less than 1 in absolute value, extreme scorers on a first test tend to be less extreme on a second, even with no intervention at all. Many 'treatment effects' on extreme groups are just this artefact.

IN [Python / Jupyter cell]

```
ability = rng.normal(70, 8, 5000)
test1 = ability + rng.normal(0, 6, 5000); test2 = ability + rng.normal(0, 6, 5000)
top = test1 > np.quantile(test1, 0.9)
print(f"top 10% on test 1: mean test1 = {test1[top].mean():.1f}, "
      f"mean test2 = {test2[top].mean():.1f}  <- 'decline' with no cause")
```

OUT

```
top 10% on test 1: mean test1 = 87.4, mean test2 = 81.9  <- 'decline' with no cause
```

Chapter summary

OLS estimates the line that minimises squared residuals; the slope is $r \cdot s_y/s_x$. Inference uses t_{n-2}. R^2 measures explained variance, not correctness. Distinguish confidence intervals for the mean response from wider prediction intervals for individuals. Always run the four diagnostic plots and check leverage and Cook's distance; use transformations for curvature and non-constant variance, and robust regression when outliers are genuine.

PRACTICAL TIP

References: Kutner, Nachtsheim, Neter & Li, *Applied Linear Statistical Models* (5th ed.), chapters 1 to 4 and 10; Montgomery, Peck & Vining, *Introduction to Linear Regression Analysis*; James, Witten, Hastie, Tibshirani & Taylor, *An Introduction to Statistical Learning with Applications in Python* (ISLP), chapter 3.

CHAPTER 9

Multiple Linear Regression

Multiple regression models the mean of Y as a linear function of several predictors at once. Its central power is **adjustment**: each coefficient estimates the association between one predictor and Y *holding the other predictors fixed*. This chapter develops the matrix form, builds a complete OLS engine from scratch (so every number in a statsmodels summary is demystified), then covers categorical predictors, interactions, multicollinearity, diagnostics, robust standard errors, model selection and regularisation.

9.1 The model in matrix form

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim N(\mathbf{0}, \sigma^2 \mathbf{I}_n)$$

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \qquad \widehat{\mathrm{Var}}(\hat{\boldsymbol{\beta}}) = s^2 (\mathbf{X}^\top \mathbf{X})^{-1}, \qquad s^2 = \frac{\mathbf{e}^\top \mathbf{e}}{n - p}$$

X is the n × p design matrix (a column of ones for the intercept, then one column per predictor), so p counts the intercept. The **hat matrix** $H = X(X^TX)^{-1}X^T$ maps y to fitted values $\hat{y} = Hy$; its diagonal entries are the leverages. In code, never invert X^TX explicitly for estimation; use a QR-based solver such as `np.linalg.lstsq`, which is numerically stable.

9.2 An OLS engine from scratch

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()

import scipy.stats as stats
import matplotlib.pyplot as plt

class OLS:
    """Ordinary least squares with a statsmodels-style summary."""
```

```
    def __init__(self, X, y, names):
        self.X, self.y, self.names = np.asarray(X, float), np.asarray(y, float), names
        self.n, self.p = self.X.shape

    def fit(self, cov="classical"):
        X, y, n, p = self.X, self.y, self.n, self.p
        self.beta, *_ = np.linalg.lstsq(X, y, rcond=None)
        self.fitted = X @ self.beta
        self.resid = y - self.fitted
        self.df_resid = n - p
        self.sse = self.resid @ self.resid
        self.s2 = self.sse / self.df_resid
        XtX_inv = np.linalg.inv(X.T @ X)
        if cov == "classical":
            self.cov = self.s2 * XtX_inv
        elif cov == "HC3":                                  # heteroscedasticity-robust
            h = np.einsum("ij,jk,ik->i", X, XtX_inv, X)  # leverages
            u2 = (self.resid / (1 - h))**2
            self.cov = XtX_inv @ (X.T * u2) @ X @ XtX_inv
        self.se = np.sqrt(np.diag(self.cov))
        self.t = self.beta / self.se
        self.pval = 2 * stats.t.sf(np.abs(self.t), self.df_resid)
        sst = np.sum((y - y.mean())**2)
        self.r2 = 1 - self.sse / sst
        self.adj_r2 = 1 - (1 - self.r2) * (n - 1) / self.df_resid
        self.F = ((sst - self.sse) / (p - 1)) / self.s2
        self.F_p = stats.f.sf(self.F, p - 1, self.df_resid)
        self.loglik = -n / 2 * (np.log(2 * np.pi * self.sse / n) + 1)
        self.aic = -2 * self.loglik + 2 * p
        self.bic = -2 * self.loglik + np.log(n) * p
        return self

    def summary(self, alpha=0.05):
        tc = stats.t.ppf(1 - alpha / 2, self.df_resid)
        tab = pd.DataFrame({"coef": self.beta, "std_err": self.se, "t": self.t,
                            "P>|t|": self.pval, "[0.025": self.beta - tc * self.se,
                            "0.975]": self.beta + tc * self.se}, index=self.names)
        print(tab.to_string(float_format=lambda v: f"{v:.4f}"))
        print(f"n = {self.n}, R2 = {self.r2:.4f}, adj R2 = {self.adj_r2:.4f}, "
              f"F({self.p - 1}, {self.df_resid}) = {self.F:.2f} (p = {self.F_p:.2e})")
        print(f"residual SE = {np.sqrt(self.s2):.4f}, log-lik = {self.loglik:.2f}, "
              f"AIC = {self.aic:.2f}, BIC = {self.bic:.2f}")
```

IN [Python / Jupyter cell]

```
def build_X(data, numeric, categorical=(), intercept=True):
    parts, names = [], []
    if intercept:
        parts.append(np.ones((len(data), 1))); names.append("Intercept")
    if numeric:
        parts.append(data[list(numeric)].to_numpy(float)); names += list(numeric)
    for c in categorical:                # dummy coding, first level = reference
        d = pd.get_dummies(data[c], prefix=c, drop_first=True, dtype=float)
        parts.append(d.to_numpy()); names += list(d.columns)
    return np.hstack(parts), names

num_vars = ["study_hours", "sleep_hours", "attendance", "prior_gpa"]
X, names = build_X(df, num_vars, ["method"])
full = OLS(X, df["final_score"], names).fit()
full.summary()
```

OUT

```
              coef  std_err       t  P>|t|  [0.025  0.975]
Intercept   12.0539   4.1385  2.9126 0.0039  3.9089 20.1988
study_hours  2.2973   0.1198 19.1686 0.0000  2.0614  2.5332
sleep_hours  1.1949   0.3153  3.7900 0.0002  0.5744  1.8154
attendance   0.1660   0.0272  6.1106 0.0000  0.1125  0.2194
prior_gpa    6.1269   0.7947  7.7099 0.0000  4.5629  7.6909
method_B     3.6721   0.8427  4.3573 0.0000  2.0135  5.3307
```

```
method_C     7.6572   0.8758  8.7435 0.0000  5.9336  9.3807
n = 300, R2 = 0.6438, adj R2 = 0.6365, F(6, 293) = 88.25 (p = 9.75e-63)
residual SE = 6.0947, log-lik = -964.37, AIC = 1942.73, BIC = 1968.66
```

Term	True value	What the estimate means
study_hours	2.5	Each extra weekly study hour adds about 2.5 points, holding sleep, attendance, GPA and method fixed.
sleep_hours	1.5	Each extra nightly hour of sleep adds about 1.5 points, other things equal.
attendance	0.15	Each extra percentage point of attendance adds about 0.15 points.
prior_gpa	6	One GPA point higher is associated with about 6 more points.
method_B, method_C	4, 7	Mean difference from the reference method A, adjusted for the covariates.

All five estimates land close to the truth, and every true value lies inside its 95% confidence interval. The residual SE estimates σ (true value 6). Compare the simple regression of Chapter 8 (R^2 0.43) with R^2 here: adding the other predictors explains much more of the variation.

IN [Python / Jupyter cell]

```python
# Residuals are now close to normal (compare Chapter 7.3), because the model is right
print("Shapiro-Wilk on full-model residuals: p =", round(stats.shapiro(full.resid).pvalue, 4))
```

OUT

```
Shapiro-Wilk on full-model residuals: p = 0.8336
```

9.3 'Holding other variables constant' and omitted variable bias

Leaving out a predictor that is correlated with an included one biases the included coefficient. The bias equals (effect of the omitted variable) × (slope of the omitted variable on the included one). The demonstration below creates a confounder deliberately.

IN [Python / Jupyter cell]

```python
rng = np.random.default_rng(9)
n = 2000
motivation = rng.normal(0, 1, n)                          # unobserved confounder
tutoring = 2 + 0.8 * motivation + rng.normal(0, 1, n)     # motivated students seek tutoring
grade = 50 + 1.0 * tutoring + 5.0 * motivation + rng.normal(0, 3, n)

short = OLS(np.column_stack([np.ones(n), tutoring]), grade, ["Int", "tutoring"]).fit()
long_ = OLS(np.column_stack([np.ones(n), tutoring, motivation]), grade,
            ["Int", "tutoring", "motivation"]).fit()
delta = np.polyfit(tutoring, motivation, 1)[0]            # slope of omitted on included
print(f"true effect of tutoring  : 1.000")
print(f"omitting motivation      : {short.beta[1]:.3f}")
print(f"controlling for it       : {long_.beta[1]:.3f}")
print(f"predicted bias = 5.0 x {delta:.3f} = {5 * delta:.3f}; "
      f"observed bias = {short.beta[1] - 1:.3f}")
```

OUT

```
true effect of tutoring  : 1.000
omitting motivation      : 3.401
controlling for it       : 0.987
predicted bias = 5.0 x 0.478 = 2.389; observed bias = 2.401
```

WATCH OUT
Regression adjusts only for variables you include and measure well. Controlling for a **mediator** (a variable on the causal path) or a **collider** (caused by both X and Y) can create bias rather than remove it. Causal claims need a causal model, typically drawn as a directed acyclic graph (DAG).

9.4 Testing groups of coefficients: the nested F test

To test whether several coefficients are jointly zero (for example, all method dummies), compare a reduced model with the full model. This general linear hypothesis test underlies ANOVA tables, and is the right way to test a categorical predictor with more than two levels.

$$F = \frac{(SSE_R - SSE_F)/q}{SSE_F/(n - p_F)} \sim F_{q,\ n-p_F}$$

IN [Python / Jupyter cell]

```
def nested_F(reduced, full):
    q = full.p - reduced.p
    F = ((reduced.sse - full.sse) / q) / (full.sse / full.df_resid)
    return F, q, full.df_resid, stats.f.sf(F, q, full.df_resid)

X_r, names_r = build_X(df, num_vars)                  # no method dummies
red = OLS(X_r, df["final_score"], names_r).fit()
F, q, d2, p = nested_F(red, full)
print(f"H0: method has no effect (2 coefficients = 0): F({q}, {d2}) = {F:.2f}, p = {p:.2e}")
print(f"R2 reduced = {red.r2:.4f} -> full = {full.r2:.4f}")
```

OUT

```
H0: method has no effect (2 coefficients = 0): F(2, 293) = 38.30, p = 1.67e-15
R2 reduced = 0.5506 -> full = 0.6438
```

9.5 Interactions

An interaction lets the slope of one predictor depend on another. With x_1x_2 in the model, the effect of x_1 is $\beta_1 + \beta_3x_2$. Centre continuous predictors before forming products: it makes the main-effect coefficients interpretable (effects at the mean of the other variable) and reduces collinearity between the main effects and the product.

$$E[Y] = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_3x_1x_2 \quad \Rightarrow \quad \frac{\partial E[Y]}{\partial x_1} = \beta_1 + \beta_3x_2$$

IN [Python / Jupyter cell]

```
d = df.copy()
d["study_c"] = d.study_hours - d.study_hours.mean()
d["gpa_c"] = d.prior_gpa - d.prior_gpa.mean()
d["study_x_gpa"] = d.study_c * d.gpa_c
Xi, ni = build_X(d, ["study_c", "gpa_c", "study_x_gpa", "sleep_hours", "attendance"],
                 ["method"])
inter = OLS(Xi, d.final_score, ni).fit()
inter.summary()
print("\n(The data were generated with NO interaction; the test should not reject.)")
```

OUT

```
               coef  std_err       t  P>|t|  [0.025  0.975]
Intercept   43.9204   3.1241 14.0585 0.0000 37.7718 50.0691
study_c      2.2987   0.1199 19.1686 0.0000  2.0627  2.5347
gpa_c        6.1098   0.7954  7.6819 0.0000  4.5444  7.6751
study_x_gpa  0.2166   0.2588  0.8371 0.4032 -0.2927  0.7260
sleep_hours  1.2043   0.3156  3.8155 0.0002  0.5831  1.8255
attendance   0.1654   0.0272  6.0836 0.0000  0.1119  0.2189
method_B     3.6709   0.8432  4.3536 0.0000  2.0114  5.3304
```

```
method_C     7.6608   0.8762  8.7431 0.0000  5.9363  9.3853
n = 300, R2 = 0.6446, adj R2 = 0.6361, F(7, 292) = 75.67 (p = 6.77e-62)
residual SE = 6.0978, log-lik = -964.01, AIC = 1944.01, BIC = 1973.64

(The data were generated with NO interaction; the test should not reject.)
```

9.6 Polynomial terms for curvature

IN [Python / Jupyter cell]

```python
# A curved relationship: y depends on x quadratically
xq = rng.uniform(0, 10, 300)
yq = 5 + 4 * xq - 0.35 * xq**2 + rng.normal(0, 2, 300)
lin = OLS(np.column_stack([np.ones(300), xq]), yq, ["Int", "x"]).fit()
quad = OLS(np.column_stack([np.ones(300), xq, xq**2]), yq, ["Int", "x", "x^2"]).fit()
print(f"linear   : R2 = {lin.r2:.3f}, AIC = {lin.aic:.1f}")
print(f"quadratic: R2 = {quad.r2:.3f}, AIC = {quad.aic:.1f}, coefs = {quad.beta.round(3)}")
print(f"turning point at x = {-quad.beta[1] / (2 * quad.beta[2]):.2f} (true {4 / 0.7:.2f})")
```

OUT

```
linear   : R2 = 0.153, AIC = 1577.1
quadratic: R2 = 0.711, AIC = 1256.6, coefs = [ 5.087  4.008 -0.354]
turning point at x = 5.67 (true 5.71)
```

9.7 Multicollinearity

When predictors are highly correlated, the data cannot separate their individual effects: standard errors inflate, coefficients become unstable and may even flip sign, although predictions can remain fine. The **variance inflation factor** $\mathrm{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 comes from regressing x_j on the other predictors, measures how much the variance of β_j is inflated. VIF above 5 is worth a look; above 10 is serious.

IN [Python / Jupyter cell]

```python
def vif(data, cols):
    out = {}
    for c in cols:
        others = [o for o in cols if o != c]
        Xo = np.column_stack([np.ones(len(data)), data[others].to_numpy(float)])
        r2 = OLS(Xo, data[c], ["Int"] + others).fit().r2
        out[c] = 1 / (1 - r2)
    return pd.Series(out, name="VIF")

print("student predictors:\n", vif(df, num_vars).round(3))

# Create a collinear predictor: 'revision_hours' almost equal to study_hours
d2 = df.copy()
d2["revision_hours"] = d2.study_hours * 0.9 + rng.normal(0, 0.3, len(d2))
print("\nwith a near-duplicate predictor:\n",
      vif(d2, num_vars + ["revision_hours"]).round(2))
Xc, nc = build_X(d2, num_vars + ["revision_hours"])
OLS(Xc, d2.final_score, nc).fit().summary()
```

OUT

```
student predictors:
 study_hours    1.001
sleep_hours    1.001
attendance     1.009
prior_gpa      1.009
Name: VIF, dtype: float64

with a near-duplicate predictor:
 study_hours       83.16
sleep_hours        1.02
attendance         1.01
```

```
prior_gpa          1.01
revision_hours    83.27
Name: VIF, dtype: float64
                  coef  std_err       t  P>|t|  [0.025  0.975]
Intercept      17.2203   4.5810  3.7590 0.0002  8.2045 26.2360
study_hours     3.1325   1.2167  2.5745 0.0105  0.7379  5.5271
sleep_hours     0.9591   0.3552  2.7002 0.0073  0.2600  1.6581
attendance      0.1664   0.0304  5.4746 0.0000  0.1066  0.2262
prior_gpa       6.1775   0.8897  6.9433 0.0000  4.4265  7.9285
revision_hours -0.9737   1.3352 -0.7293 0.4664 -3.6014  1.6540
n = 300, R2 = 0.5514, adj R2 = 0.5438, F(5, 294) = 72.29 (p = 3.68e-49)
residual SE = 6.8274, log-lik = -998.93, AIC = 2009.87, BIC = 2032.09
```

The study_hours SE has exploded and the two near-duplicate predictors now split the effect arbitrarily, yet R^2 barely changed. Remedies: drop one of the variables, combine them (an average or a principal component, Chapter 11), collect more varied data, or use ridge regression (9.11).

9.8 Diagnostics and robust standard errors

Diagnostics are as in Chapter 8, with two additions. **Partial regression (added-variable) plots** show the relationship between y and one predictor after removing the linear effect of all the others. **Heteroscedasticity-consistent (HC) standard errors**, also called sandwich or White standard errors, remain valid when the error variance is not constant. HC3 is the recommended small-sample version.

$$\widehat{\mathrm{Var}}_{HC3}(\hat\beta) = (X^\top X)^{-1} X^\top \,\mathrm{diag}\!\left(\frac{e_i^2}{(1-h_{ii})^2}\right) X\,(X^\top X)^{-1}$$

IN [Python / Jupyter cell]

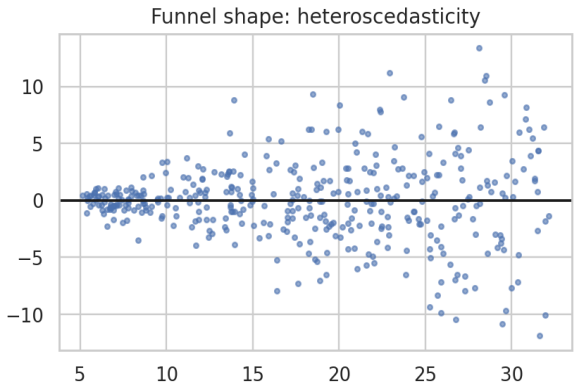
```
# Heteroscedastic data: noise SD grows with x
xh = rng.uniform(1, 10, 400)
yh = 2 + 3 * xh + rng.normal(0, 0.6 * xh)
Xh = np.column_stack([np.ones(400), xh])
classical = OLS(Xh, yh, ["Int", "x"]).fit()
robust = OLS(Xh, yh, ["Int", "x"]).fit(cov="HC3")
print(f"slope = {classical.beta[1]:.4f}")
print(f"classical SE = {classical.se[1]:.4f} | HC3 robust SE = {robust.se[1]:.4f}")

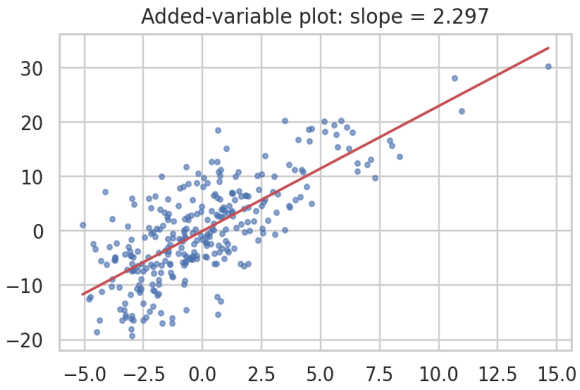
# Breusch-Pagan on these residuals
aux = OLS(Xh, classical.resid**2, ["Int", "x"]).fit()
LM = len(yh) * aux.r2
print(f"Breusch-Pagan LM = {LM:.2f}, p = {stats.chi2.sf(LM, 1):.2e}")

fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
ax[0].scatter(classical.fitted, classical.resid, s=8, alpha=.6); ax[0].axhline(0, color="k")
ax[0].set_title("Funnel shape: heteroscedasticity")
# Added-variable plot for study_hours in the student model
others = [c for c in num_vars if c != "study_hours"]
Xo, no = build_X(df, others, ["method"])
ey = OLS(Xo, df.final_score, no).fit().resid          # y purged of other predictors
ex = OLS(Xo, df.study_hours, no).fit().resid          # x purged of other predictors
ax[1].scatter(ex, ey, s=8, alpha=.6)
ax[1].plot(np.sort(ex), np.sort(ex) * np.polyfit(ex, ey, 1)[0], "r")
ax[1].set_title(f"Added-variable plot: slope = {np.polyfit(ex, ey, 1)[0]:.3f}")
plt.tight_layout(); plt.show()
```

OUT

```
slope = 3.0077
classical SE = 0.0700 | HC3 robust SE = 0.0761
Breusch-Pagan LM = 70.78, p = 3.99e-17
```





EXAM / VIVA ANGLE
The slope in the added-variable plot equals the multiple regression coefficient of study_hours exactly. This is the **Frisch-Waugh-Lovell theorem**, and it is the cleanest way to explain what 'holding other variables constant' means.

9.9 Standardised coefficients

Raw coefficients depend on units, so they cannot be compared directly. Standardising every variable (z-scores) gives coefficients in SD units: the change in Y, in SDs, for a one-SD change in x_j. Useful for rough importance comparisons among continuous predictors.

IN [Python / Jupyter cell]

```
z = df[num_vars + ["final_score"]].apply(lambda s: (s - s.mean()) / s.std())
Xz = np.column_stack([np.ones(len(z)), z[num_vars]])
beta_std = OLS(Xz, z.final_score, ["Int"] + num_vars).fit().beta[1:]
print(pd.Series(beta_std, index=num_vars).round(3).sort_values(ascending=False))
```

OUT

```
study_hours    0.659
prior_gpa      0.272
attendance     0.214
sleep_hours    0.110
dtype: float64
```

9.10 Model selection

With many candidate predictors, we need principled selection. **Adjusted R^2** penalises extra terms mildly. **AIC** (Akaike) targets predictive accuracy; **BIC** (Schwarz) penalises complexity more heavily and targets the 'true' model. **Cross-validation** estimates out-of-sample error directly. Avoid automated stepwise selection based on p-values for inference: the resulting p-values and CIs are invalid because the same data chose the model.

$$AIC = -2\ln\hat{L} + 2p \qquad BIC = -2\ln\hat{L} + p\ln n \qquad \bar{R}^2 = 1 - (1-R^2)\frac{n-1}{n-p}$$

IN [Python / Jupyter cell]

```
from itertools import combinations
from sklearn.datasets import load_diabetes
diab = load_diabetes(as_frame=True, scaled=False).frame
preds = [c for c in diab.columns if c != "target"]
print(diab.head(3))

# Best-subset search over all 2^10 - 1 = 1023 models, scored by AIC and BIC
rows = []
```

```
for k in range(1, len(preds) + 1):
    for combo in combinations(preds, k):
        Xs = np.column_stack([np.ones(len(diab)), diab[list(combo)]])
        m = OLS(Xs, diab.target, ["Int", *combo]).fit()
        rows.append({"vars": combo, "k": k, "adjR2": m.adj_r2, "AIC": m.aic, "BIC": m.bic})
res = pd.DataFrame(rows)
for crit in ["AIC", "BIC"]:
    best = res.loc[res[crit].idxmin()]
    print(f"\nbest by {crit}: {best['vars']}"
          f" ({crit} = {best[crit]:.1f}, adjR2 = {best.adjR2:.4f})")
best_per_k = res.loc[res.groupby("k")["BIC"].idxmin(), ["k", "adjR2", "AIC", "BIC"]]
print("\n", best_per_k.round(3).to_string(index=False))
```

OUT

```
    age  sex   bmi     bp     s1     s2    s3   s4      s5    s6  target
0  59.0  2.0  32.1  101.0  157.0   93.2  38.0  4.0  4.8598  87.0   151.0
1  48.0  1.0  21.6   87.0  183.0  103.2  70.0  3.0  3.8918  69.0    75.0
2  72.0  2.0  30.5   93.0  156.0   93.6  41.0  4.0  4.6728  85.0   141.0

best by AIC: ('sex', 'bmi', 'bp', 's1', 's2', 's5') (AIC = 4788.6, adjR2 = 0.5082)

best by BIC: ('sex', 'bmi', 'bp', 's3', 's5') (BIC = 4816.8, adjR2 = 0.5030)

  k  adjR2      AIC      BIC
 1  0.342 4912.038 4920.221
 2  0.457 4828.398 4840.672
 3  0.477 4813.226 4829.591
 4  0.487 4804.963 4825.419
 5  0.503 4792.264 4816.811
 6  0.508 4788.603 4817.243
 7  0.508 4789.320 4822.051
 8  0.509 4790.241 4827.062
 9  0.508 4792.015 4832.928
10  0.507 4793.986 4838.990
```

Cross-validation

IN [Python / Jupyter cell]

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, KFold

cv = KFold(n_splits=10, shuffle=True, random_state=0)
def cv_rmse(cols):
    sc = cross_val_score(LinearRegression(), diab[list(cols)], diab.target,
                         cv=cv, scoring="neg_root_mean_squared_error")
    return -sc.mean()
best_aic = res.loc[res.AIC.idxmin(), "vars"]; best_bic = res.loc[res.BIC.idxmin(), "vars"]
print(f"10-fold CV RMSE, all 10 predictors : {cv_rmse(preds):.2f}")
print(f"10-fold CV RMSE, AIC-best subset   : {cv_rmse(best_aic):.2f}")
print(f"10-fold CV RMSE, BIC-best subset   : {cv_rmse(best_bic):.2f}")
print(f"10-fold CV RMSE, bmi only          : {cv_rmse(['bmi']):.2f}")
```

OUT

```
10-fold CV RMSE, all 10 predictors : 54.46
10-fold CV RMSE, AIC-best subset   : 54.19
10-fold CV RMSE, BIC-best subset   : 54.40
10-fold CV RMSE, bmi only          : 62.35
```

9.11 Regularisation: ridge, lasso and elastic net

Regularised regression adds a penalty on coefficient size to the least-squares criterion. **Ridge** (L2 penalty) shrinks all coefficients toward zero and handles collinearity well. **Lasso** (L1 penalty) can set coefficients exactly to zero, performing variable selection. **Elastic net** mixes both. The penalty strength λ (called `alpha` in scikit-learn) is chosen by cross-validation. Always standardise predictors first, since the penalty treats all coefficients on the same scale.

$$\hat{\beta}^{ridge} = \arg\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_2^2 \qquad \hat{\beta}^{lasso} = \arg\min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \lambda \|\beta\|_1$$

IN [Python / Jupyter cell]

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV, lasso_path

Xd, yd = diab[preds], diab.target
ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 50))).fit(Xd, yd)
lasso = make_pipeline(StandardScaler(), LassoCV(cv=cv, random_state=0)).fit(Xd, yd)
enet = make_pipeline(StandardScaler(),
                     ElasticNetCV(l1_ratio=[.2, .5, .8, 1], cv=cv, random_state=0)).fit(Xd, yd)
ols_std = make_pipeline(StandardScaler(), LinearRegression()).fit(Xd, yd)
coefs = pd.DataFrame({"OLS": ols_std[-1].coef_, "ridge": ridge[-1].coef_,
                      "lasso": lasso[-1].coef_, "enet": enet[-1].coef_}, index=preds)
print(coefs.round(2))
print(f"\nchosen penalties: ridge={ridge[-1].alpha_:.3f}, lasso={lasso[-1].alpha_:.3f}, "
      f"enet={enet[-1].alpha_:.3f} (l1_ratio={enet[-1].l1_ratio_})")
for name, m in [("OLS", LinearRegression()), ("ridge", ridge[-1]), ("lasso", lasso[-1])]:
    pipe = make_pipeline(StandardScaler(), m)
    sc = cross_val_score(pipe, Xd, yd, cv=cv, scoring="neg_root_mean_squared_error")
    print(f"CV RMSE {name:<6}: {-sc.mean():.2f}")
```

OUT

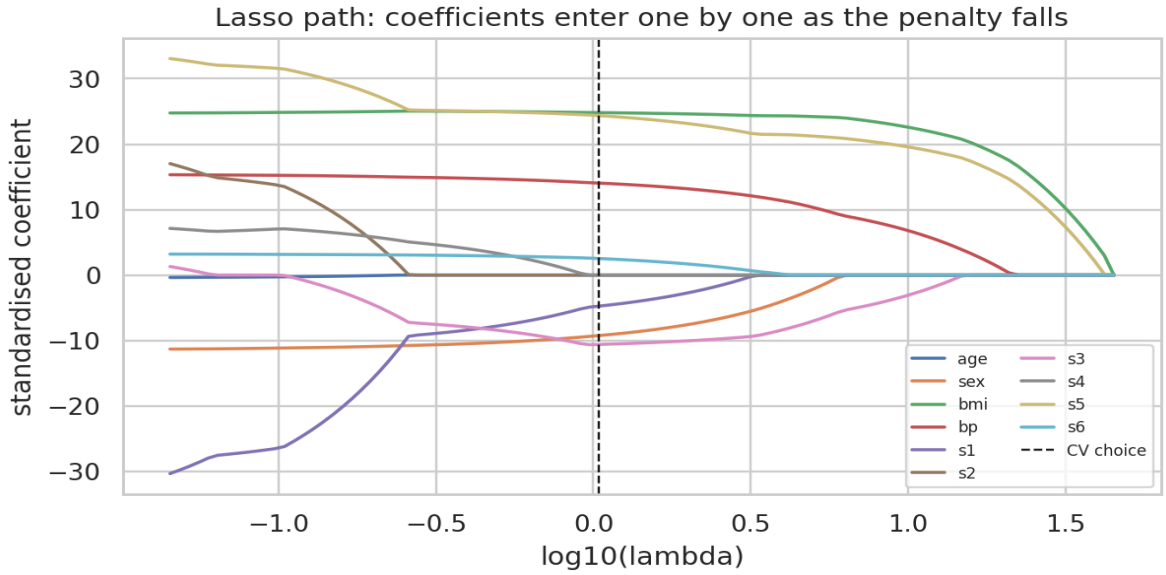
```
       OLS  ridge  lasso   enet
age  -0.48  -0.40  -0.00  -0.00
sex -11.41 -11.27  -9.24  -9.24
bmi  24.73  24.79  24.82  24.82
bp   15.43  15.33  14.05  14.05
s1  -37.68 -25.02  -4.75  -4.75
s2   22.68  12.64  -0.00  -0.00
s3    4.81  -0.75 -10.60 -10.60
s4    8.42   6.94   0.00   0.00
s5   35.73  30.89  24.37  24.37
s6    3.22   3.31   2.52   2.52

chosen penalties: ridge=2.024, lasso=1.043, enet=1.043 (l1_ratio=1.0)
CV RMSE OLS   : 54.46
CV RMSE ridge : 54.65
CV RMSE lasso : 54.68
```

Notice what lasso did: it set age, s2 and s4 to zero and stabilised the wildly offsetting s1/s2 pair (the blood serum measures are strongly correlated). Its cross-validated error is essentially the same as OLS here, because with n = 442 and only 10 predictors there is little overfitting to cure. Regularisation pays off most when p is large relative to n or predictors are highly collinear; its other benefit is a simpler, more stable model.

IN [Python / Jupyter cell]

```
Xs = StandardScaler().fit_transform(Xd)
alphas, paths, _ = lasso_path(Xs, yd - yd.mean())
plt.figure(figsize=(8, 4))
for j, name in enumerate(preds):
    plt.plot(np.log10(alphas), paths[j], label=name)
plt.axvline(np.log10(lasso[-1].alpha_), color="k", ls="--", lw=1, label="CV choice")
plt.xlabel("log10(lambda)"); plt.ylabel("standardised coefficient")
plt.title("Lasso path: coefficients enter one by one as the penalty falls")
plt.legend(fontsize=7, ncol=2); plt.show()
```



9.12 A predictive workflow with a held-out test set

IN [Python / Jupyter cell]

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X_tr, X_te, y_tr, y_te = train_test_split(Xd, yd, test_size=0.25, random_state=1)
model = make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=0)).fit(X_tr, y_tr)
pred = model.predict(X_te)
print(f"test RMSE = {mean_squared_error(y_te, pred) ** 0.5:.2f}, "
      f"MAE = {mean_absolute_error(y_te, pred):.2f}, R2 = {r2_score(y_te, pred):.3f}")
```

OUT

```
test RMSE = 54.32, MAE = 42.47, R2 = 0.435
```

KEY IDEA

Two different goals need two different workflows. For **inference** (which variables matter, by how much, with what uncertainty), specify the model from subject knowledge, fit it once, and report coefficients with CIs. For **prediction**, choose among models by cross-validated error and judge the winner on a test set that played no part in the choice.

9.13 The statsmodels formula interface

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.anova import anova_lm

# Formulas: C() marks categorical, * gives main effects + interaction, I() protects maths
m_full = smf.ols("final_score ~ study_hours + sleep_hours + attendance + prior_gpa"
                 " + C(method)", data=df).fit()
print(m_full.summary())                       # matches the from-scratch OLS in 9.2

m_ref = smf.ols("final_score ~ study_hours + C(method, Treatment(reference='C'))",
                data=df).fit()                # change the reference level
m_int = smf.ols("final_score ~ study_hours * prior_gpa + C(method)", data=df).fit()
m_poly = smf.ols("y ~ x + I(x**2)", data=pd.DataFrame({"x": xq, "y": yq})).fit()
m_log = smf.ols("np.log(final_score) ~ study_hours", data=df).fit()
```

```
m_red = smf.ols("final_score ~ study_hours + sleep_hours + attendance + prior_gpa",
                data=df).fit()
print(anova_lm(m_red, m_full))                # nested F test (9.4)
print(m_full.compare_f_test(m_red))           # (F, p, df_diff)

m_hc3 = m_full.get_robustcov_results(cov_type="HC3")   # or .fit(cov_type="HC3")
print(m_hc3.summary())

X = sm.add_constant(df[["study_hours", "sleep_hours", "attendance", "prior_gpa"]])
print({c: variance_inflation_factor(X.values, i) for i, c in enumerate(X.columns)})

fig = sm.graphics.plot_partregress_grid(m_full)          # added-variable plots
fig = sm.graphics.influence_plot(m_full, criterion="cooks")
```

RUN THIS IN YOUR JUPYTER

Every quantity here was computed from scratch above: coefficients and summary statistics (9.2), nested F (9.4), HC3 standard errors (9.8), VIFs (9.7) and added-variable plots (9.8). Use the outputs above to check your own run.

Chapter summary

Multiple regression estimates partial effects via $\beta = (X^TX)^{-1}X^Ty$. Coefficients are interpreted holding other predictors fixed; omitted confounders bias them. Test groups of terms with nested F tests, model curvature with polynomials and effect modification with (centred) interactions. Check VIFs for collinearity, use residual and added-variable plots, and switch to HC3 standard errors under heteroscedasticity. For prediction, select models by cross-validation and consider ridge or lasso; for inference, specify the model in advance.

PRACTICAL TIP

References: Kutner et al., *Applied Linear Statistical Models*, chapters 6 to 11; Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning*, chapter 3; ISLP chapters 3, 5 and 6; Wooldridge, *Introductory Econometrics* (omitted variable bias, robust SEs); Fox, *Applied Regression Analysis and Generalized Linear Models*.

CHAPTER 10

Time Series Analysis and Forecasting

A time series is a sequence of observations ordered in time. The ordering breaks the independence assumption behind everything so far: today's value is usually related to yesterday's. Time series analysis exploits that dependence to understand structure (trend, seasonality, cycles) and to forecast. This chapter follows the workflow of Hyndman and Athanasopoulos's *Forecasting: Principles and Practice*: visualise, decompose, check stationarity, model the autocorrelation, forecast, and evaluate honestly on held-out data.

10.1 Time series in pandas

pandas is built around a `DatetimeIndex`. With it you can slice by date strings, resample to other frequencies, compute rolling statistics and create lags in one line each.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_sales(seed=7):
    """Monthly retail sales, Jan 2012 to Dec 2025: trend + yearly seasonality
    + AR(1) noise. Multiplicative-looking seasonality grows with the level."""
    rng = np.random.default_rng(seed)
    idx = pd.date_range("2012-01-01", "2025-12-01", freq="MS")
    t = np.arange(len(idx))
    trend = 200 + 2.2 * t
    season = 1 + 0.18 * np.sin(2 * np.pi * (t % 12) / 12 - 1.2) \
               + 0.08 * np.cos(4 * np.pi * (t % 12) / 12)
    e = np.zeros(len(t))
    for i in range(1, len(t)):
        e[i] = 0.55 * e[i - 1] + rng.normal(0, 9)
    y = trend * season + e
    return pd.Series(y.round(1), index=idx, name="sales")

sales = make_sales()

import scipy.stats as stats
import matplotlib.pyplot as plt

print(sales["2025"].head(3))                    # slice by year (string indexing)
print("\nfrequency:", sales.index.freqstr)
q = sales.resample("QE").sum()                  # monthly -> quarterly totals
a = sales.resample("YE").mean()                 # annual mean
print("\nquarterly totals:\n", q.tail(4).round(1))
feat = pd.DataFrame({"sales": sales,
                     "lag1": sales.shift(1),            # last month
                     "lag12": sales.shift(12),          # same month last year
                     "roll12_mean": sales.rolling(12).mean(),
                     "yoy_growth_%": sales.pct_change(12) * 100})
print("\n", feat.dropna().tail(4).round(2))
```

OUT

```
2025-01-01    489.7
2025-02-01    490.2
2025-03-01    494.8
Freq: MS, Name: sales, dtype: float64

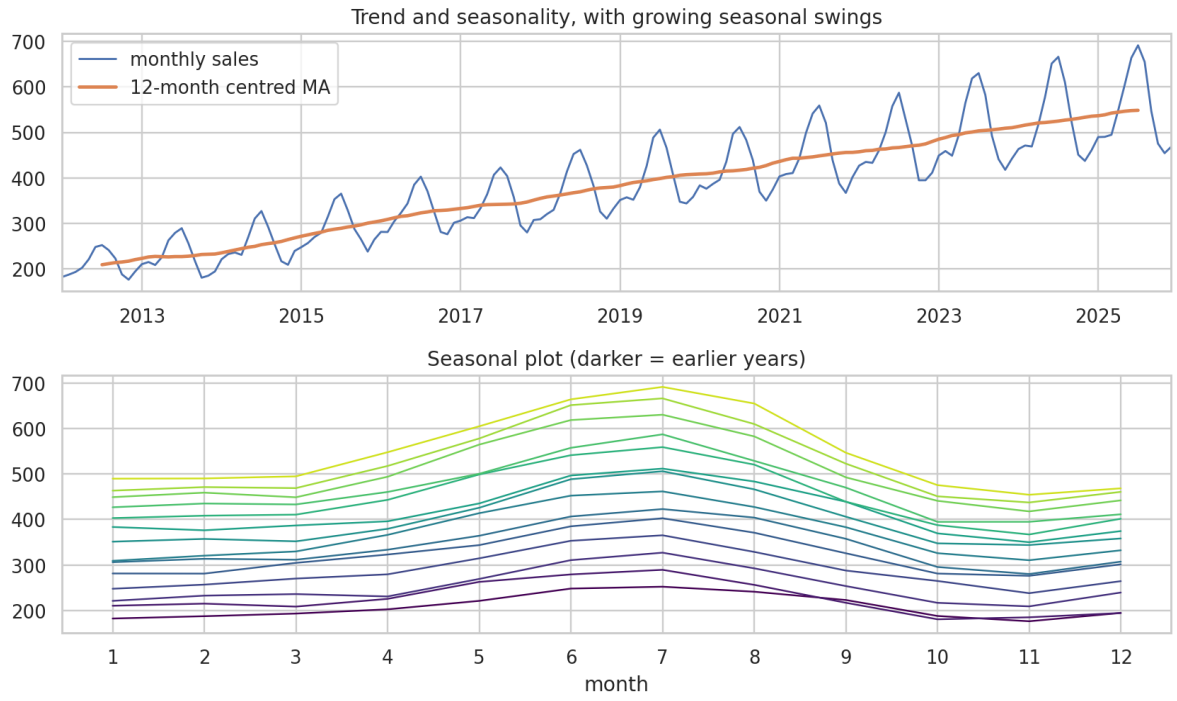
frequency: MS
```

```
quarterly totals:
 2025-03-31    1474.7
2025-06-30    1816.6
2025-09-30    1892.9
2025-12-31    1398.3
Freq: QE-DEC, Name: sales, dtype: float64

             sales   lag1  lag12  roll12_mean  yoy_growth_%
2025-09-01  546.7  654.9  522.7       544.43          4.59
2025-10-01  475.5  546.7  450.9       546.48          5.46
2025-11-01  454.4  475.5  437.4       547.90          3.89
2025-12-01  468.4  454.4  460.7       548.54          1.67
```

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(2, 1, figsize=(10, 6))
sales.plot(ax=ax[0], lw=1.2, label="monthly sales")
sales.rolling(12, center=True).mean().plot(ax=ax[0], lw=2, label="12-month centred MA")
ax[0].legend(); ax[0].set_title("Trend and seasonality, with growing seasonal swings")
# Seasonal plot: one line per year
tab = pd.DataFrame({"year": sales.index.year, "month": sales.index.month, "y": sales.values})
for yr, g in tab.groupby("year"):
    ax[1].plot(g.month, g.y, color=plt.cm.viridis((yr - 2012) / 14), lw=1)
ax[1].set_xticks(range(1, 13)); ax[1].set_xlabel("month")
ax[1].set_title("Seasonal plot (darker = earlier years)")
plt.tight_layout(); plt.show()
```



10.2 Components and classical decomposition

A series is commonly viewed as trend-cycle T, seasonal S and remainder R. In an **additive** decomposition y = T + S + R the seasonal swings are constant in size; in a **multiplicative** one y = T × S × R they grow with the level, as here. Taking logs turns multiplicative into additive, which is why log transforms are so common in forecasting.

The classical algorithm: (1) estimate the trend with a centred 2 × 12 moving average; (2) detrend (y/T for multiplicative); (3) average the detrended values for each month to get seasonal indices, normalised to average 1; (4) the remainder is what is left.

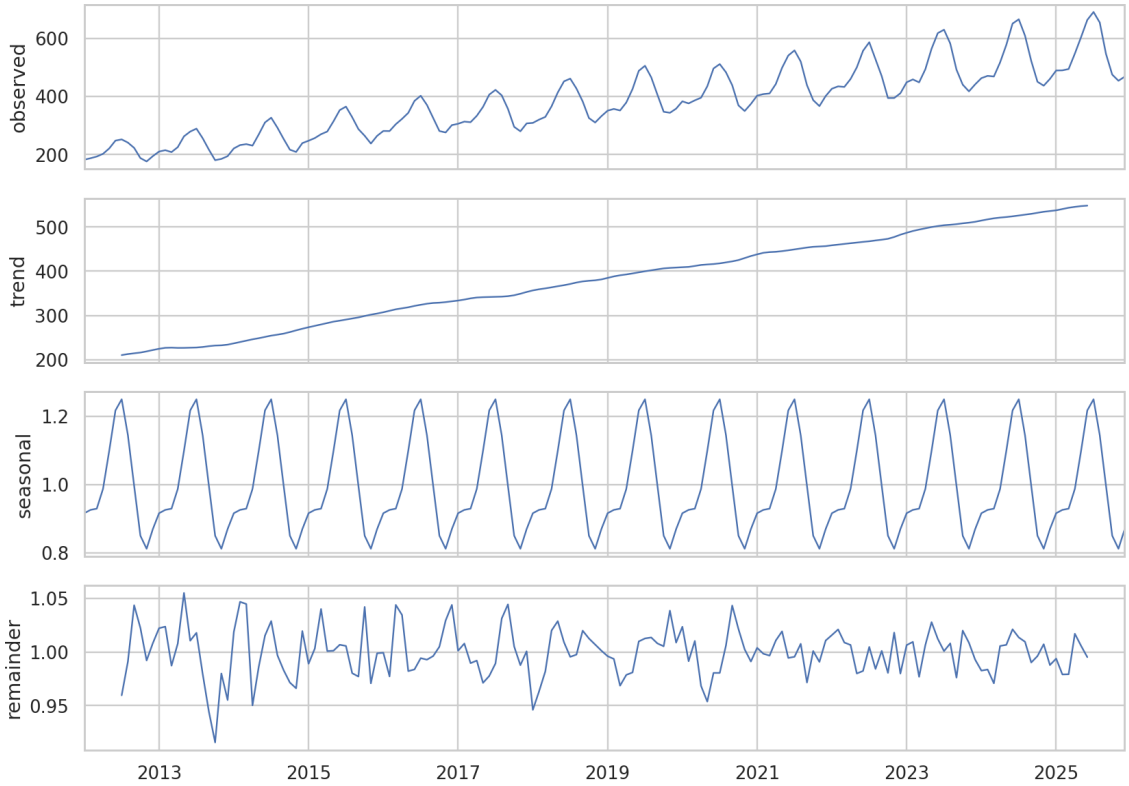
IN [Python / Jupyter cell]

```
def classical_decompose(y, m=12, model="multiplicative"):
    # centred 2 x m moving average (for even m)
    trend = y.rolling(m, center=True).mean().rolling(2).mean().shift(-1)
    detr = y / trend if model == "multiplicative" else y - trend
    month = np.arange(len(y)) % m
    idx = pd.Series(detr.values).groupby(month).mean()
    idx = idx / idx.mean() if model == "multiplicative" else idx - idx.mean()
    seasonal = pd.Series(idx.values[month], index=y.index)
    resid = y / (trend * seasonal) if model == "multiplicative" else y - trend - seasonal
    return trend, seasonal, resid

T, S, R = classical_decompose(sales)
print("seasonal indices (Jan..Dec):", S.iloc[:12].round(3).tolist())
fig, ax = plt.subplots(4, 1, figsize=(10, 7), sharex=True)
for a_, s_, t_ in zip(ax, [sales, T, S, R], ["observed", "trend", "seasonal", "remainder"]):
    s_.plot(ax=a_, lw=1); a_.set_ylabel(t_)
plt.tight_layout(); plt.show()
```

OUT

```
seasonal indices (Jan..Dec): [0.917, 0.926, 0.93, 0.987, 1.099, 1.217, 1.25, 1.144, 0.997, 0.851, 0.813, 0.87]
```



A seasonal index of 1.17 for a month means sales then run about 17% above the trend. The remainder should look like noise; any visible pattern means structure is left to model.

10.3 Stationarity

A series is (weakly) **stationary** if its mean, variance and autocorrelations do not change over time. Trend and seasonality violate stationarity. ARMA models assume it, so we transform first: logs stabilise variance, **differencing** ($y_t - y_{t-1}$) removes trend, and **seasonal differencing** ($y_t - y_{t-12}$) removes seasonality.

The **augmented Dickey-Fuller (ADF)** test checks H_0: the series has a unit root (non-stationary). It regresses Δy_t on y_{t-1} and lagged differences; the t statistic of y_{t-1} follows a non-standard Dickey-Fuller distribution, so it is compared with special critical values (about

−3.47, −2.88 and −2.58 at 1%, 5% and 10% for a model with a constant and n near 170). The **KPSS** test reverses the hypotheses (H_0: stationary); using both is good practice.

$$\Delta y_t = \alpha + \gamma\, y_{t-1} + \sum_{j=1}^{k} \phi_j \Delta y_{t-j} + e_t, \qquad H_0 : \gamma = 0 \text{ (unit root)}$$

IN [Python / Jupyter cell]

```
def adf_stat(y, k=2):
    """ADF t statistic (constant, k lagged differences), computed by OLS."""
    y = np.asarray(y, float); dy = np.diff(y)
    rows = range(k, len(dy))
    X = np.column_stack([np.ones(len(rows)), y[k:-1]] +
                        [dy[k - j:len(dy) - j] for j in range(1, k + 1)])
    Y = dy[k:]
    beta, *_ = np.linalg.lstsq(X, Y, rcond=None)
    e = Y - X @ beta; s2 = e @ e / (len(Y) - X.shape[1])
    se = np.sqrt(s2 * np.linalg.inv(X.T @ X)[1, 1])
    return beta[1] / se

ly = np.log(sales)
series = [("log sales", ly), ("diff(log)", ly.diff()), ("seasonal diff(log)", ly.diff(12)),
          ("diff + seasonal diff", ly.diff(12).diff())]
for name, s in series:
    print(f"{name:<22} ADF t = {adf_stat(s.dropna()):7.3f}   (reject unit root if < -2.88)")
```

OUT

```
log sales              ADF t =  -2.521   (reject unit root if < -2.88)
diff(log)              ADF t =  -9.803   (reject unit root if < -2.88)
seasonal diff(log)     ADF t =  -3.933   (reject unit root if < -2.88)
diff + seasonal diff   ADF t =  -9.525   (reject unit root if < -2.88)
```

10.4 Autocorrelation: ACF and PACF

The **autocorrelation function** ρ(k) is the correlation between y_t and y_{t-k}. The **partial autocorrelation** φ_{kk} is the correlation at lag k after removing the effect of lags 1 to k − 1; it equals the last coefficient in an AR(k) regression. Approximate 95% bounds for white noise are ±1.96/√n. The patterns identify ARMA models, as in the table below.

Process	ACF	PACF
White noise	No significant spikes	No significant spikes
AR(p)	Decays gradually (exponential or damped sine)	Cuts off after lag p
MA(q)	Cuts off after lag q	Decays gradually
ARMA(p, q)	Decays after lag q	Decays after lag p
Non-stationary (trend)	Decays very slowly from near 1	Large spike at lag 1
Seasonal (period m)	Spikes at m, 2m, ...	Spike(s) at m

IN [Python / Jupyter cell]

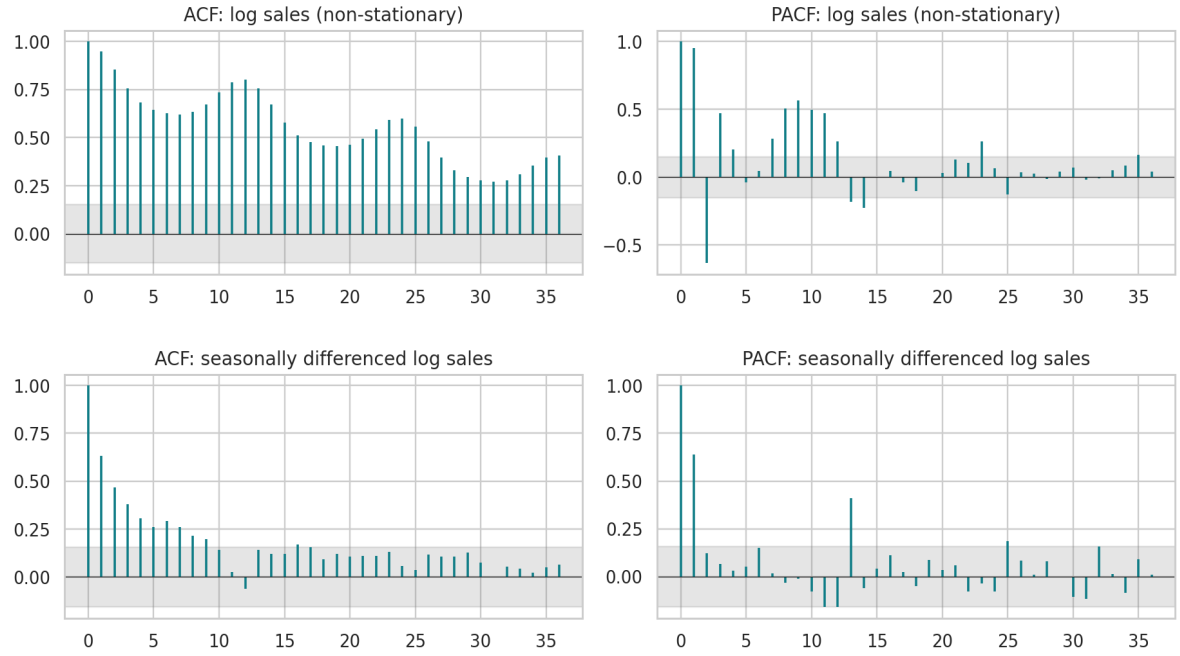
```
def acf(x, nlags):
    x = np.asarray(x, float) - np.mean(x); d = x @ x
    return np.array([1.0] + [x[k:] @ x[:-k] / d for k in range(1, nlags + 1)])

def pacf(x, nlags):
    """PACF by OLS: coefficient on the k-th lag in an AR(k) regression."""
    x = np.asarray(x, float); out = [1.0]
    for k in range(1, nlags + 1):
        lagged = [x[k - j:len(x) - j] for j in range(1, k + 1)]
```

```
        X = np.column_stack([np.ones(len(x) - k)] + lagged)
        beta, *_ = np.linalg.lstsq(X, x[k:], rcond=None); out.append(beta[-1])
    return np.array(out)

def acf_pacf_plot(x, nlags=36, title=""):
    n = len(x); bound = 1.96 / np.sqrt(n)
    fig, ax = plt.subplots(1, 2, figsize=(11, 3))
    for a_, vals, nm in [(ax[0], acf(x, nlags), "ACF"), (ax[1], pacf(x, nlags), "PACF")]:
        a_.vlines(range(nlags + 1), 0, vals, color="#0E7C86"); a_.axhline(0, color="k", lw=.5)
        a_.axhspan(-bound, bound, color="grey", alpha=.2); a_.set_title(f"{nm}: {title}")
    plt.tight_layout(); plt.show()

acf_pacf_plot(ly, title="log sales (non-stationary)")
z = ly.diff(12).dropna()                          # seasonally differenced log sales
acf_pacf_plot(z, title="seasonally differenced log sales")
```



Log sales show the textbook non-stationary signature: an ACF that decays very slowly. After seasonal differencing (which here equals year-on-year log growth), the ACF decays geometrically and the PACF has a dominant spike at lag 1 and a clear one at lag 12. That suggests an AR(1) term plus a seasonal term, i.e. a model in the SARIMA family.

The Ljung-Box test for remaining autocorrelation

$$Q = n(n+2)\sum_{k=1}^{h}\frac{\hat{\rho}_k^2}{n-k} \;\sim\; \chi^2_{h-\#\text{params}} \text{ under } H_0 \text{ (white noise)}$$

IN [Python / Jupyter cell]

```
def ljung_box(x, h=24, fitted_params=0):
    n = len(x); r = acf(x, h)[1:]
    Q = n * (n + 2) * np.sum(r**2 / (n - np.arange(1, h + 1)))
    return Q, stats.chi2.sf(Q, h - fitted_params)

rng = np.random.default_rng(0)
print("white noise          : Q = %.2f, p = %.3f" % ljung_box(rng.normal(size=160)))
print("seasonal diff log    : Q = %.2f, p = %.2e" % ljung_box(z))
```

OUT

```
white noise          : Q = 26.60, p = 0.323
seasonal diff log    : Q = 223.88, p = 2.33e-34
```

10.5 Train/test split and benchmark forecasts

Forecasts must be evaluated on data the model never saw. We hold out the last 24 months (2024 to 2025). Every sophisticated model must beat simple **benchmarks**: the naive forecast (last value), the seasonal naive forecast (same month last year), and drift (a line from the first to the last observation). Surprisingly often, they are hard to beat.

$$MAE = \frac{1}{h}\sum|e_t| \qquad RMSE = \sqrt{\frac{1}{h}\sum e_t^2} \qquad MAPE = \frac{100}{h}\sum\left|\frac{e_t}{y_t}\right| \qquad MASE = \frac{MAE}{\frac{1}{n-m}\sum|y_t - y_{t-m}|}$$

IN [Python / Jupyter cell]

```
train, test = sales[:"2023-12"], sales["2024-01":]
h, m = len(test), 12

def scores(name, fc):
    e = test.values - np.asarray(fc)
    scale = np.mean(np.abs(train.values[m:] - train.values[:-m]))   # seasonal naive in-sample
    return {"model": name, "MAE": np.mean(np.abs(e)), "RMSE": np.sqrt(np.mean(e**2)),
            "MAPE%": 100 * np.mean(np.abs(e / test.values)), "MASE": np.mean(np.abs(e)) / scale}

fc = {}
fc["mean"] = np.repeat(train.mean(), h)
fc["naive"] = np.repeat(train.iloc[-1], h)
fc["seasonal naive"] = np.tile(train.iloc[-m:].values, h // m)
slope = (train.iloc[-1] - train.iloc[0]) / (len(train) - 1)
fc["drift"] = train.iloc[-1] + slope * np.arange(1, h + 1)
board = pd.DataFrame([scores(k, v) for k, v in fc.items()]).set_index("model")
print(board.round(2))
```

OUT

```
                   MAE    RMSE  MAPE%  MASE
model
mean            182.74  199.63  32.65  6.78
naive            95.07  124.19  15.98  3.53
seasonal naive   33.35   36.96   6.09  1.24
drift            79.49  107.73  13.26  2.95
```

10.6 Exponential smoothing (Holt-Winters)

Exponential smoothing forecasts with weighted averages of past observations, weights decaying exponentially with age. **Simple** exponential smoothing tracks a level; **Holt's** method adds a trend; **Holt-Winters** adds seasonality. The smoothing parameters α, β, γ in (0, 1) are chosen by minimising one-step-ahead squared errors. We fit additive Holt-Winters to log sales, which corresponds to multiplicative seasonality on the original scale.

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}) \qquad b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}$$

$$s_t = \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m} \qquad \hat{y}_{t+h|t} = \ell_t + h\,b_t + s_{t+h-m(\lfloor (h-1)/m \rfloor + 1)}$$

IN [Python / Jupyter cell]

```
from scipy.optimize import minimize

def hw_filter(y, alpha, beta, gamma, m=12):
    y = np.asarray(y, float)
    l = y[:m].mean(); b = (y[m:2 * m].mean() - y[:m].mean()) / m
    s = list(y[:m] - l)
    fitted = np.full(len(y), np.nan)
    for t in range(m, len(y)):
        fitted[t] = l + b + s[t - m]
        l_new = alpha * (y[t] - s[t - m]) + (1 - alpha) * (l + b)
        b = beta * (l_new - l) + (1 - beta) * b
```

```
        s.append(gamma * (y[t] - l_new) + (1 - gamma) * s[t - m]); l = l_new
    return fitted, l, b, np.array(s)

def hw_fit(y, m=12):
    sse = lambda p: np.nansum((np.asarray(y) - hw_filter(y, *p, m)[0])**2)
    res = minimize(sse, x0=[0.3, 0.05, 0.1], bounds=[(1e-4, 1)] * 3, method="L-BFGS-B")
    return res.x

def hw_forecast(y, params, h, m=12):
    _, l, b, s = hw_filter(y, *params, m)
    return np.array([l + k * b + s[len(s) - m + (k - 1) % m] for k in range(1, h + 1)])

ly_train = np.log(train)
params = hw_fit(ly_train)
print("alpha, beta, gamma =", params.round(4))
fc["Holt-Winters (log)"] = np.exp(hw_forecast(ly_train, params, h))
```

OUT

```
alpha, beta, gamma = [5.898e-01 1.000e-04 5.544e-01]
```

10.7 ARIMA and seasonal ARIMA

An **ARIMA(p, d, q)** model applies d differences, then models the result as autoregressive (AR, p lags of the series) plus moving average (MA, q lags of past errors). **SARIMA(p, d, q)(P, D, Q)$_m$** adds seasonal versions of each at lag m. In backshift notation, with $By_t = y_{t-1}$:

$$\phi(B)\,\Phi(B^m)\,(1-B)^d(1-B^m)^D\,y_t = c + \theta(B)\,\Theta(B^m)\,\varepsilon_t$$

Guided by the ACF/PACF in 10.4, we fit a SARIMA(1, 0, 0)(1, 1, 0)$_{12}$ with constant to log sales: seasonally difference once, then regress the result on its lag 1 and lag 12. Pure AR models can be estimated by OLS (conditional least squares), which is what we do here; models with MA terms need numerical maximum likelihood, which statsmodels provides.

IN [Python / Jupyter cell]

```
def fit_seasonal_ar(y, m=12, lags=(1, 12)):
    z = (y - y.shift(m)).dropna().to_numpy()           # seasonal difference
    L = max(lags)
    X = np.column_stack([np.ones(len(z) - L)] + [z[L - k:len(z) - k] for k in lags])
    beta, *_ = np.linalg.lstsq(X, z[L:], rcond=None)
    resid = z[L:] - X @ beta
    return beta, resid, z

def forecast_seasonal_ar(y, beta, h, m=12, lags=(1, 12), shocks=None):
    z = list((y - y.shift(m)).dropna().to_numpy()); hist = list(y.to_numpy())
    for i in range(h):
        zn = beta[0] + sum(b * z[-k] for b, k in zip(beta[1:], lags))
        if shocks is not None: zn += shocks[i]
        z.append(zn); hist.append(hist[-m] + zn)       # undo the seasonal difference
    return np.array(hist[-h:])

beta, resid, zt = fit_seasonal_ar(ly_train)
print("constant, phi_1, Phi_12 =", beta.round(4), "| residual SD =", resid.std(ddof=3).round(4))
print("Ljung-Box on residuals: Q = %.2f, p = %.3f" % ljung_box(resid, h=24, fitted_params=2))
fc["SARIMA(1,0,0)(1,1,0)12"] = np.exp(forecast_seasonal_ar(ly_train, beta, h))
```

OUT

```
constant, phi_1, Phi_12 = [ 0.0406  0.6035 -0.1069] | residual SD = 0.0341
Ljung-Box on residuals: Q = 46.59, p = 0.002
```

The Ljung-Box test rejects: the residuals are not white noise, so the model has missed structure. This is the Box-Jenkins cycle in action (identify, estimate, **diagnose**, revise). Seasonal differencing of a series with stable seasonality tends to create a seasonal moving average

component, which a single seasonal AR lag approximates poorly. Two standard fixes: add a seasonal MA term, SARIMA(1, 0, 0)(0, 1, 1)$_{12}$ (needs maximum likelihood, see 10.11), or add more seasonal AR lags, which OLS can handle. We take the second route with lags 12, 24, 36.

IN [Python / Jupyter cell]

```
LAGS = (1, 12, 24, 36)                                  # SARIMA(1,0,0)(3,1,0)12
beta, resid, zt = fit_seasonal_ar(ly_train, lags=LAGS)
print("coefficients:", beta.round(4))
print("Ljung-Box on residuals: Q = %.2f, p = %.3f" % ljung_box(resid, h=24, fitted_params=4))
fc["SARIMA(1,0,0)(3,1,0)12"] = np.exp(forecast_seasonal_ar(ly_train, beta, h, lags=LAGS))
```

OUT

```
coefficients: [ 0.0354  0.5247 -0.1195  0.0258  0.0491]
Ljung-Box on residuals: Q = 24.80, p = 0.209
```

Residuals now pass the white-noise check (p well above 0.05), so the revised model is adequate by this criterion.

10.8 Comparing forecasts and building prediction intervals

IN [Python / Jupyter cell]

```
board = pd.DataFrame([scores(k, v) for k, v in fc.items()]).set_index("model")
print(board.sort_values("RMSE").round(2))
```

OUT

```
                          MAE    RMSE  MAPE%  MASE
model
Holt-Winters (log)      17.98   21.30   3.52  0.67
SARIMA(1,0,0)(3,1,0)12  19.35   21.97   3.68  0.72
SARIMA(1,0,0)(1,1,0)12  31.94   34.73   6.01  1.19
seasonal naive          33.35   36.96   6.09  1.24
drift                   79.49  107.73  13.26  2.95
naive                   95.07  124.19  15.98  3.53
mean                   182.74  199.63  32.65  6.78
```

Holt-Winters and the revised SARIMA beat every benchmark, with MASE below 1 (they beat the seasonal naive method's in-sample accuracy). The first SARIMA, whose residuals failed Ljung-Box, barely beats seasonal naive: failed diagnostics usually show up as worse forecasts. A point forecast without an interval is incomplete. A general way to get intervals is to **simulate future paths** by feeding bootstrapped residuals through the fitted model, then take percentiles across paths.

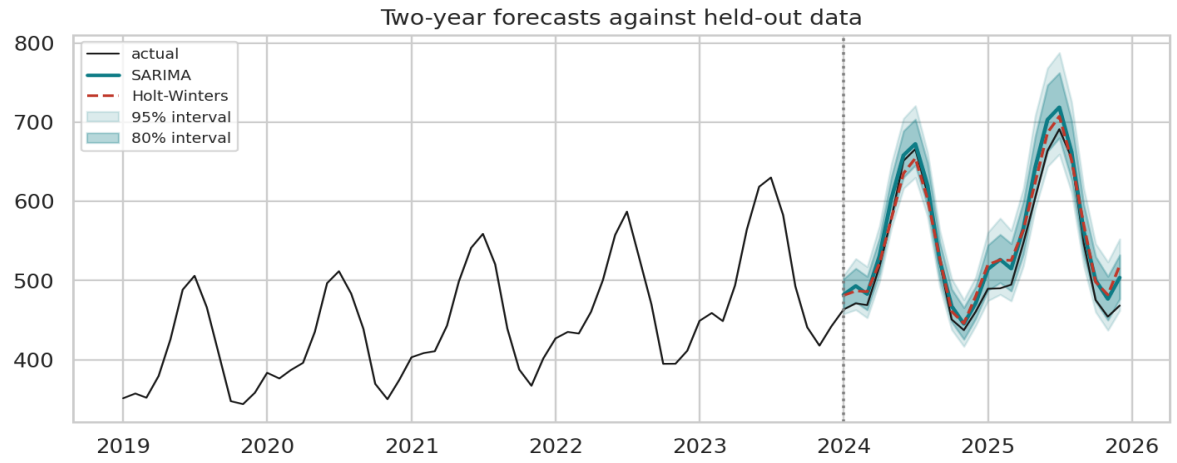
IN [Python / Jupyter cell]

```
rng = np.random.default_rng(10)
paths = np.array([np.exp(forecast_seasonal_ar(ly_train, beta, h, lags=LAGS,
                                              shocks=rng.choice(resid, h)))
                  for _ in range(2000)])
lo80, hi80 = np.percentile(paths, [10, 90], axis=0)
lo95, hi95 = np.percentile(paths, [2.5, 97.5], axis=0)
cover = np.mean((test.values >= lo95) & (test.values <= hi95))
print(f"share of test months inside the 95% interval: {cover:.2f}")

plt.figure(figsize=(10, 4))
plt.plot(sales["2019":].index, sales["2019":], color="k", lw=1, label="actual")
plt.plot(test.index, fc["SARIMA(1,0,0)(3,1,0)12"], color="#0E7C86", lw=2, label="SARIMA")
plt.plot(test.index, fc["Holt-Winters (log)"], color="#C0392B", ls="--", lw=1.5,
         label="Holt-Winters")
plt.fill_between(test.index, lo95, hi95, alpha=.15, color="#0E7C86", label="95% interval")
plt.fill_between(test.index, lo80, hi80, alpha=.3, color="#0E7C86", label="80% interval")
plt.axvline(test.index[0], color="grey", ls=":"); plt.legend(fontsize=8)
plt.title("Two-year forecasts against held-out data"); plt.show()
```

OUT

```
share of test months inside the 95% interval: 1.00
```



10.9 Time series cross-validation

A single train/test split can be lucky. **Rolling-origin** evaluation (time series cross-validation) refits the model at successive forecast origins and averages the errors. Never shuffle time series data: the training set must always precede the test set. scikit-learn's `TimeSeriesSplit` produces such splits.

IN [Python / Jupyter cell]

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5, test_size=12)        # five consecutive 12-month test folds
rows = []
for k, (tr_idx, te_idx) in enumerate(tscv.split(sales)):
    tr, te = np.log(sales.iloc[tr_idx]), sales.iloc[te_idx]
    b_, *_ = fit_seasonal_ar(tr, lags=LAGS)
    f_sar = np.exp(forecast_seasonal_ar(tr, b_, len(te), lags=LAGS))
    f_snv = np.exp(tr.iloc[-12:].to_numpy())
    rows.append({"fold": k, "test_start": te.index[0].date(),
                 "RMSE SARIMA": np.sqrt(np.mean((te - f_sar)**2)),
                 "RMSE seasonal naive": np.sqrt(np.mean((te - f_snv)**2))})
cvres = pd.DataFrame(rows).set_index("fold")
print(cvres.round(2)); print("\nmean:\n", cvres.mean(numeric_only=True).round(2))
```

OUT

```
      test_start  RMSE SARIMA  RMSE seasonal naive
fold
0     2021-01-01        15.07                35.58
1     2022-01-01        21.96                20.59
2     2023-01-01        14.08                39.89
3     2024-01-01        14.38                22.99
4     2025-01-01        15.86                25.34

mean:
 RMSE SARIMA            16.27
RMSE seasonal naive    28.88
dtype: float64
```

10.10 Forecasting with regression on lagged features

Any regression or machine-learning model can forecast if you engineer lag, seasonal and calendar features. This is the bridge to gradient boosting and neural forecasters. The key rule is the same: features must use only information available at forecast time.

IN [Python / Jupyter cell]

```
from sklearn.linear_model import LinearRegression
d = pd.DataFrame({"y": np.log(sales)})
d["lag12"] = d.y.shift(12); d["lag24"] = d.y.shift(24)
d["t"] = np.arange(len(d))
d = pd.concat([d, pd.get_dummies(d.index.month, prefix="m", drop_first=True,
                                 dtype=float).set_index(d.index)], axis=1).dropna()
tr_, te_ = d[:"2023-12"], d["2024-01":]
feats = [c for c in d.columns if c != "y"]
lr = LinearRegression().fit(tr_[feats], tr_.y)
pred = np.exp(lr.predict(te_[feats]))    # lags 12 and 24 are known 2 years ahead: no leakage
print("lag-feature regression:", {k: round(float(v), 2)
                                  for k, v in scores("lagreg", pred).items() if k != "model"})
```

OUT

```
lag-feature regression: {'MAE': 14.09, 'RMSE': 16.22, 'MAPE%': 2.74, 'MASE': 0.52}
```

10.11 The statsmodels time series toolkit

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from statsmodels.tsa.seasonal import seasonal_decompose, STL
from statsmodels.tsa.stattools import adfuller, kpss, acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.statespace.sarimax import SARIMAX

seasonal_decompose(sales, model="multiplicative", period=12).plot()   # 10.2
STL(np.log(sales), period=12, robust=True).fit().plot()                # modern, robust
stat, p, usedlag, nobs, crit, icbest = adfuller(np.log(sales).diff(12).dropna(),
                                                regression="c", autolag="AIC")
kpss(np.log(sales).diff(12).dropna(), regression="c", nlags="auto")
z = np.log(sales).diff(12).dropna()
plot_acf(z, lags=36); plot_pacf(z, lags=36, method="ywm")

hw = ExponentialSmoothing(train, trend="add", seasonal="mul", seasonal_periods=12,
                          initialization_method="estimated").fit()
hw.forecast(24)

model = SARIMAX(np.log(train), order=(1, 0, 0), seasonal_order=(3, 1, 0, 12),
                trend="c").fit(disp=False)
airline_like = SARIMAX(np.log(train), order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
                       trend="c").fit(disp=False)       # seasonal MA alternative
print(model.aic, airline_like.aic)
print(model.summary())                                  # coefficients, AIC, Ljung-Box, JB tests
acorr_ljungbox(model.resid[13:], lags=[12, 24], model_df=4)
fc = model.get_forecast(steps=24)
point = np.exp(fc.predicted_mean); ci = np.exp(fc.conf_int(alpha=0.05))
model.plot_diagnostics(figsize=(10, 7))

# Simple order search by AIC (the idea behind auto_arima in pmdarima)
import itertools
best = min(((p, q, P, Q) for p, q, P, Q in itertools.product(range(3), range(3),
                                                             range(2), range(2))),
           key=lambda o: SARIMAX(np.log(train), order=(o[0], 0, o[1]),
                                 seasonal_order=(o[2], 1, o[3], 12), trend="c")
                         .fit(disp=False).aic)
```

RUN THIS IN YOUR JUPYTER

SARIMAX estimates by exact maximum likelihood rather than the conditional least squares used above, so coefficients differ slightly (third decimal), and forecasts should be very close to the executed SARIMA forecasts. ExponentialSmoothing with seasonal='mul' is the multiplicative counterpart of our additive-on-logs Holt-Winters.

PRACTICAL TIP
Other tools worth knowing: **pmdarima.auto_arima** (automatic SARIMA order selection), **statsmodels VAR** for several interacting series, **arch** for volatility (GARCH) models of financial returns, and **Prophet** or **statsforecast** for fast forecasting at scale.

Chapter summary

Plot first; decompose into trend, season and remainder; transform (log) and difference until stationary, confirming with ADF/KPSS. Read ACF and PACF to choose AR and MA orders; check that model residuals are white noise with Ljung-Box. Always compare against naive and seasonal naive benchmarks on held-out data, report intervals as well as point forecasts, and use rolling-origin evaluation rather than a single split.

PRACTICAL TIP
References: Hyndman & Athanasopoulos, *Forecasting: Principles and Practice* (3rd ed., free online; a Python edition is also available); Box, Jenkins, Reinsel & Ljung, *Time Series Analysis: Forecasting and Control*; Shumway & Stoffer, *Time Series Analysis and Its Applications*; Brockwell & Davis, *Introduction to Time Series and Forecasting*.

CHAPTER 11

Factor Analysis and Principal Component Analysis

Many data sets contain many correlated variables that are driven by a few underlying dimensions. A questionnaire with 12 items might really measure 3 traits; 13 chemical measurements of wine might vary along 2 or 3 main axes. **Principal component analysis** (PCA) and **factor analysis** (FA) both reduce dimension, but they answer different questions, and confusing them is one of the most common errors in applied multivariate statistics.

	Principal component analysis	Factor analysis (exploratory)
Goal	Summarise total variance with few uncorrelated components	Explain the correlations among variables by latent factors
Model	Components are exact linear combinations of the variables	Variables are linear combinations of factors plus unique error: x = Λf + ε
Variance analysed	All variance (diagonal of correlation matrix = 1)	Only shared (common) variance; unique variance modelled separately
Typical use	Compression, visualisation, collinearity fix, preprocessing	Scale development, psychometrics, theory of latent constructs
Rotation	Not usual (components defined by maximal variance)	Essential for interpretation (varimax, promax)

11.1 PCA: the mathematics

PCA finds orthogonal directions of maximal variance. They are the eigenvectors of the covariance matrix (or of the correlation matrix when variables are standardised, which is almost always appropriate when units differ). The eigenvalue λ_k is the variance captured by component k, so $\lambda_k/\Sigma\lambda$ is its share of total variance.

$$\mathbf{R} = \mathbf{V}\Lambda\mathbf{V}^\top, \qquad \mathbf{z}_k = \mathbf{X}_{std}\mathbf{v}_k, \qquad \text{loading}_{jk} = v_{jk}\sqrt{\lambda_k} = \text{corr}(x_j, z_k)$$

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)
np.set_printoptions(linewidth=100)

wine_bunch = load_wine(as_frame=True)
W = wine_bunch.data; cultivar = wine_bunch.target
print(W.shape); print(W.iloc[:3, :6])

Z = StandardScaler().fit_transform(W)              # standardise: PCA on the correlation matrix
R = np.corrcoef(Z, rowvar=False)
eigval, eigvec = np.linalg.eigh(R)                   # eigh: for symmetric matrices
order = np.argsort(eigval)[::-1]; eigval, eigvec = eigval[order], eigvec[:, order]
print("\neigenvalues (from scratch):", eigval.round(3))

pca = PCA().fit(Z)
```

```
print("sklearn explained_variance_:", pca.explained_variance_.round(3))
print("share of variance          :", pca.explained_variance_ratio_.round(3))
print("cumulative                 :", np.cumsum(pca.explained_variance_ratio_).round(3))
```

OUT

```
(178, 13)
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80

eigenvalues (from scratch): [4.706 2.497 1.446 0.919 0.853 0.642 0.551 0.348 0.289 0.251 0.226 0.169 0.103]
sklearn explained_variance_: [4.732 2.511 1.454 0.924 0.858 0.645 0.554 0.35  0.291 0.252 0.227 0.17  0.104]
share of variance          : [0.362 0.192 0.111 0.071 0.066 0.049 0.042 0.027 0.022 0.019 0.017 0.013 0.008]
cumulative                 : [0.362 0.554 0.665 0.736 0.802 0.851 0.893 0.92  0.942 0.962 0.979 0.992 1.   ]
```

PRACTICAL TIP

sklearn's `explained_variance_` uses $n - 1$ in the denominator, so it matches the eigenvalues of the correlation matrix only up to a factor $n/(n-1)$ when you standardise with StandardScaler (which uses n). Here the factor is 178/177, too small to see at 3 decimals.

11.2 How many components?

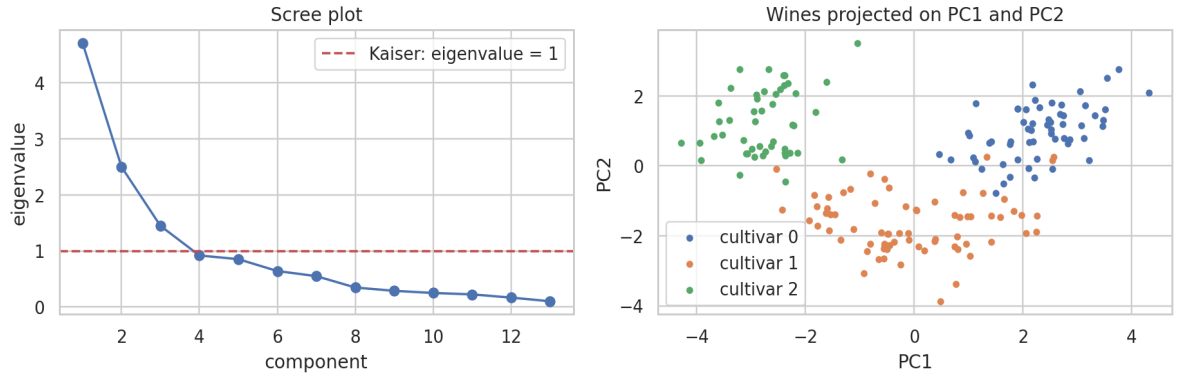
Three common rules: the **Kaiser criterion** (keep components with eigenvalue > 1, i.e. those explaining more than one original variable), the **scree plot** (keep components before the 'elbow'), and a **target cumulative variance** (e.g. 70 to 90%). Parallel analysis (11.5) is the most reliable of all.

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
k = np.arange(1, len(eigval) + 1)
ax[0].plot(k, eigval, "o-")
ax[0].axhline(1, color="r", ls="--", label="Kaiser: eigenvalue = 1")
ax[0].set(title="Scree plot", xlabel="component", ylabel="eigenvalue"); ax[0].legend()
scores = pca.transform(Z)
for c in np.unique(cultivar):
    sel = cultivar == c
    ax[1].scatter(scores[sel, 0], scores[sel, 1], s=12, label=f"cultivar {c}")
ax[1].set(title="Wines projected on PC1 and PC2", xlabel="PC1", ylabel="PC2"); ax[1].legend()
plt.tight_layout(); plt.show()
print("Kaiser rule keeps", np.sum(eigval > 1), "components")
```

OUT

```
Kaiser rule keeps 3 components
```



The first two components capture about 55% of the variance, and the three cultivars separate clearly in that plane, even though PCA never saw the cultivar labels. That is the power of unsupervised dimension reduction.

IN [Python / Jupyter cell]

```
loadings = pd.DataFrame(pca.components_[:3].T * np.sqrt(pca.explained_variance_[:3]),
                        index=W.columns, columns=["PC1", "PC2", "PC3"])
print(loadings.round(2).sort_values("PC1"))
```

OUT

```
                               PC1   PC2   PC3
nonflavanoid_phenols         -0.65  0.05  0.21
malic_acid                   -0.53  0.36  0.11
alcalinity_of_ash            -0.52 -0.02  0.74
color_intensity              -0.19  0.84 -0.17
ash                          -0.00  0.50  0.76
alcohol                       0.31  0.77 -0.25
magnesium                     0.31  0.47  0.16
proline                       0.62  0.58 -0.15
hue                           0.65 -0.44  0.10
proanthocyanins               0.68  0.06  0.18
od280/od315_of_diluted_wines  0.82 -0.26  0.20
total_phenols                 0.86  0.10  0.18
flavanoids                    0.92 -0.01  0.18
```

Loadings are correlations between the variables and the components. PC1 contrasts flavonoids, total phenols and OD280 (positive) with non-flavonoid phenols and alcalinity (negative): a 'phenolic richness' axis. PC2 is dominated by colour intensity, alcohol, proline and ash: an 'intensity/body' axis.

11.3 The factor analysis model

Factor analysis posits that each observed variable is driven by a small number m of latent common factors plus a unique factor. Under the model, the correlation matrix decomposes into a low-rank common part and a diagonal unique part. The **communality** h_j^2 is the share of a variable's variance explained by the common factors; the **uniqueness** $\psi_j = 1 - h_j^2$ is the rest.

$$\mathbf{x} = \boldsymbol{\Lambda}\mathbf{f} + \boldsymbol{\varepsilon}, \qquad \mathbf{R} = \boldsymbol{\Lambda}\boldsymbol{\Lambda}^{\top} + \boldsymbol{\Psi}, \qquad h_j^2 = \sum_{k=1}^{m} \lambda_{jk}^2$$

To learn FA properly we use a questionnaire with a **known** structure: 12 items answered by 500 students on a 1 to 7 scale, written to measure three traits. Items Q1 to Q4 measure *maths anxiety*, Q5 to Q8 *study motivation*, and Q9 to Q12 *social engagement*. The anxiety and motivation factors are mildly negatively correlated (r = −0.3).

IN [Python / Jupyter cell]

```
rng = np.random.default_rng(11)
n = 500
phi = np.array([[1, -.3, 0], [-.3, 1, .1], [0, .1, 1]])        # factor correlations
F = rng.multivariate_normal(np.zeros(3), phi, size=n)
true_L = np.zeros((12, 3))
true_L[0:4, 0] = [.80, .75, .70, .65]
true_L[4:8, 1] = [.80, .70, .70, .60]
true_L[8:12, 2] = [.75, .70, .65, .55]
true_L[3, 1] = .30                                              # Q4 cross-loads a little
uniq = 1 - (true_L**2).sum(1)
latent = F @ true_L.T + rng.normal(size=(n, 12)) * np.sqrt(np.clip(uniq, .05, None))
items = pd.DataFrame(np.clip(np.round(4 + 1.3 * latent), 1, 7).astype(int),
                     columns=[f"Q{i}" for i in range(1, 13)])
print(items.head()); print("\nitem means:", items.mean().round(2).tolist())
```

OUT

```
   Q1  Q2  Q3  Q4  Q5  Q6  Q7  Q8  Q9  Q10  Q11  Q12
0   5   5   5   6   5   5   6   5   6    5    4    4
1   3   4   3   3   4   3   4   4   4    3    4    7
2   5   5   4   5   4   4   5   6   4    4    4    4
```

```
3   5   5   6   3   2   2   2   3   5    5    4    4
4   4   4   3   4   4   4   6   4   4    4    4    3

item means: [4.08, 4.02, 4.04, 3.92, 4.06, 4.06, 4.09, 4.09, 3.95, 3.96, 4.02, 3.95]
```

11.4 Is the data suitable for factor analysis?

Two checks come before extraction. **Bartlett's test of sphericity** tests H_0: the correlation matrix is an identity matrix (no correlations to explain). The **Kaiser-Meyer-Olkin (KMO)** measure compares correlations with partial correlations: if variables share common factors, partial correlations (controlling for all others) are small relative to raw correlations. KMO above 0.8 is meritorious, 0.7 middling, below 0.6 poor, below 0.5 unacceptable.

$$\chi^2 = -\left(n - 1 - \frac{2p+5}{6}\right)\ln|\mathbf{R}|, \quad df = \frac{p(p-1)}{2} \qquad KMO = \frac{\sum_{i \neq j} r_{ij}^2}{\sum_{i \neq j} r_{ij}^2 + \sum_{i \neq j} a_{ij}^2}$$

IN [Python / Jupyter cell]

```python
def bartlett_sphericity(X):
    n, p = X.shape; R = np.corrcoef(X, rowvar=False)
    chi2 = -(n - 1 - (2 * p + 5) / 6) * np.log(np.linalg.det(R))
    dof = p * (p - 1) / 2
    return chi2, dof, stats.chi2.sf(chi2, dof)

def kmo(X):
    R = np.corrcoef(X, rowvar=False); Rinv = np.linalg.inv(R)
    d = np.sqrt(np.outer(np.diag(Rinv), np.diag(Rinv)))
    A = -Rinv / d                                   # partial correlations
    np.fill_diagonal(R, 0); np.fill_diagonal(A, 0)
    per_item = (R**2).sum(0) / ((R**2).sum(0) + (A**2).sum(0))
    overall = (R**2).sum() / ((R**2).sum() + (A**2).sum())
    return overall, per_item

chi2, dof, p = bartlett_sphericity(items.values)
overall, per_item = kmo(items.values)
print(f"Bartlett: chi2 = {chi2:.1f}, df = {dof:.0f}, p = {p:.2e}")
print(f"KMO overall = {overall:.3f}")
print("KMO per item:", dict(zip(items.columns, per_item.round(2).tolist())))
```

OUT

```
Bartlett: chi2 = 1684.8, df = 66, p = 2.34e-308
KMO overall = 0.778
KMO per item: {'Q1': 0.79, 'Q2': 0.79, 'Q3': 0.81, 'Q4': 0.74, 'Q5': 0.75, 'Q6': 0.8, 'Q7': 0.81, 'Q8': 0.83,...
```

11.5 How many factors? Parallel analysis

Parallel analysis (Horn, 1965) compares the observed eigenvalues with eigenvalues from random data of the same size. Keep factors whose eigenvalue exceeds the 95th percentile of the random eigenvalues. It is more accurate than the Kaiser rule, which tends to keep too many.

IN [Python / Jupyter cell]

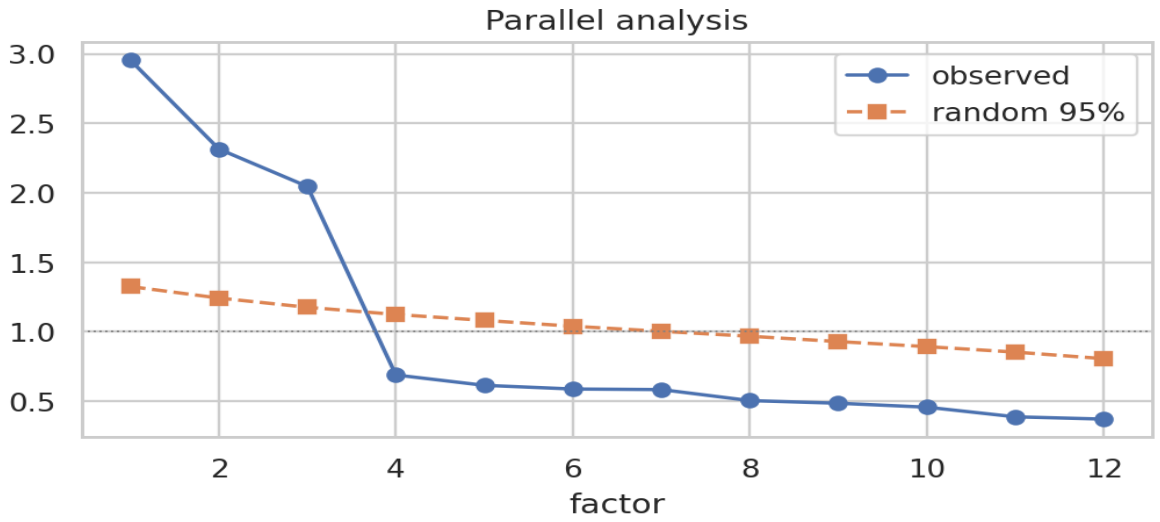
```python
def parallel_analysis(X, n_iter=500, q=95, seed=0):
    r = np.random.default_rng(seed); n, p = X.shape
    obs = np.sort(np.linalg.eigvalsh(np.corrcoef(X, rowvar=False)))[::-1]
    rand = np.array([np.sort(np.linalg.eigvalsh(np.corrcoef(r.normal(size=(n, p)),
                                                            rowvar=False)))[::-1]
                     for _ in range(n_iter)])
    thr = np.percentile(rand, q, axis=0)
    return obs, thr, int(np.argmax(obs < thr))      # first index where obs falls below

obs, thr, n_fac = parallel_analysis(items.values)
```

```
print("observed eigenvalues :", obs.round(2))
print("random 95th pct      :", thr.round(2))
print("factors to retain    :", n_fac)
plt.figure(figsize=(7, 3.2))
plt.plot(range(1, 13), obs, "o-", label="observed")
plt.plot(range(1, 13), thr, "s--", label="random 95%")
plt.axhline(1, color="grey", lw=.8, ls=":"); plt.legend(); plt.xlabel("factor")
plt.title("Parallel analysis"); plt.show()
```

OUT

```
observed eigenvalues : [2.95 2.31 2.05 0.69 0.61 0.59 0.58 0.51 0.49 0.46 0.39 0.37]
random 95th pct      : [1.33 1.24 1.18 1.12 1.08 1.04 1.   0.97 0.93 0.89 0.85 0.81]
factors to retain    : 3
```



11.6 Extraction and rotation

Common extraction methods are **maximum likelihood** (gives fit statistics and assumes multivariate normality) and **principal axis factoring** (no distributional assumption). The initial solution is mathematically valid but hard to interpret, because it is only determined up to a rotation. **Rotation** seeks 'simple structure': each item loading strongly on one factor and near zero on the others. **Varimax** is orthogonal (factors stay uncorrelated); **promax** and **oblimin** are oblique (factors may correlate), which is usually more realistic in the social sciences.

IN [Python / Jupyter cell]

```
from sklearn.decomposition import FactorAnalysis

Zi = StandardScaler().fit_transform(items)
fa_none = FactorAnalysis(n_components=3, random_state=0).fit(Zi)            # ML, unrotated
fa_vmx = FactorAnalysis(n_components=3, rotation="varimax", random_state=0).fit(Zi)

def show(L, title):
    df_ = pd.DataFrame(L, index=items.columns, columns=["F1", "F2", "F3"])
    df_["communality"] = (L**2).sum(1)
    print(title); print(df_.round(2).to_string()); print()
show(fa_none.components_.T, "UNROTATED loadings (hard to read):")
show(fa_vmx.components_.T, "VARIMAX-rotated loadings (simple structure):")
```

OUT

```
UNROTATED loadings (hard to read):
       F1    F2    F3  communality
Q1   0.67  0.36  0.12         0.59
Q2   0.62  0.40  0.18         0.57
Q3   0.56  0.36  0.17         0.47
Q4   0.35  0.53  0.27         0.48
Q5  -0.61  0.46  0.25         0.65
```

```
Q6  -0.54  0.36  0.21         0.46
Q7  -0.53  0.37  0.23         0.47
Q8  -0.51  0.32  0.14         0.38
Q9  -0.06  0.33 -0.56         0.43
Q10 -0.04  0.39 -0.59         0.50
Q11 -0.04  0.31 -0.62         0.49
Q12 -0.05  0.27 -0.48         0.31

VARIMAX-rotated loadings (simple structure):
       F1    F2    F3  communality
Q1   0.20  0.74 -0.03         0.59
Q2   0.13  0.74 -0.00         0.57
Q3   0.11  0.68  0.01         0.47
Q4  -0.18  0.67  0.00         0.48
Q5  -0.80 -0.07 -0.03         0.65
Q6  -0.67 -0.09 -0.01         0.46
Q7  -0.68 -0.07  0.00         0.47
Q8  -0.61 -0.12 -0.06         0.38
Q9  -0.04  0.00 -0.66         0.43
Q10 -0.04  0.04 -0.70         0.50
Q11  0.01 -0.01 -0.70         0.49
Q12 -0.02 -0.01 -0.55         0.31
```

The rotated solution recovers the design: each block of four items loads on its own factor, with loadings a little below the true values (0.55 to 0.80) because answers were rounded to a 1 to 7 scale, which adds measurement error. Q4 shows its secondary loading on the motivation factor. Varimax forces factors to be uncorrelated, so the real −0.3 correlation between anxiety and motivation leaks into small cross-loadings (for example Q1); an oblique rotation would absorb it. A common rule is to interpret loadings above about 0.4 in absolute value. Factor order and sign are arbitrary: flip a factor's signs if that makes it easier to name.

Principal axis factoring from scratch

Principal axis factoring makes the FA model concrete: replace the 1s on the diagonal of R with communality estimates, take eigenvectors, recompute communalities, and iterate. Varimax is implemented below too, so nothing is a black box.

IN [Python / Jupyter cell]

```python
def principal_axis(R, m, iters=200, tol=1e-6):
    h2 = 1 - 1 / np.diag(np.linalg.inv(R))           # start: squared multiple correlations
    for _ in range(iters):
        Rr = R.copy(); np.fill_diagonal(Rr, h2)
        val, vec = np.linalg.eigh(Rr); idx = np.argsort(val)[::-1][:m]
        L = vec[:, idx] * np.sqrt(np.clip(val[idx], 0, None))
        h2_new = (L**2).sum(1)
        if np.max(np.abs(h2_new - h2)) < tol: break
        h2 = h2_new
    return L

def varimax(L, gamma=1.0, iters=100, tol=1e-8):
    p, k = L.shape; Rot = np.eye(k); d = 0
    for _ in range(iters):
        Lam = L @ Rot
        u, s, vt = np.linalg.svd(L.T @ (Lam**3 - (gamma / p) * Lam @ np.diag((Lam**2).sum(0))))
        Rot = u @ vt; d_new = s.sum()
        if d_new < d * (1 + tol): break
        d = d_new
    return L @ Rot

L_paf = varimax(principal_axis(np.corrcoef(Zi, rowvar=False), 3))
show(L_paf, "PRINCIPAL AXIS + VARIMAX (from scratch):")
```

OUT

```
PRINCIPAL AXIS + VARIMAX (from scratch):
       F1    F2    F3  communality
Q1  -0.74  0.03  0.20         0.59
```

```
Q2  -0.74  0.00  0.12         0.57
Q3  -0.68 -0.01  0.11         0.47
Q4  -0.66 -0.00 -0.18         0.47
Q5   0.07  0.03 -0.80         0.65
Q6   0.10  0.01 -0.67         0.46
Q7   0.07 -0.00 -0.69         0.47
Q8   0.12  0.06 -0.61         0.38
Q9  -0.00  0.66 -0.04         0.44
Q10 -0.04  0.70 -0.04         0.50
Q11  0.01  0.69  0.01         0.48
Q12  0.01  0.56 -0.02         0.31
```

11.7 Factor scores and reliability

Factor scores estimate each respondent's position on each factor and can feed later analyses (regression, clustering). For reporting scales, researchers often use the simpler **sum score** of each factor's items, after checking internal consistency with **Cronbach's α**: 0.7 is acceptable, 0.8 good, 0.9 excellent (very high values can mean redundant items).

$$\alpha = \frac{k}{k-1}\left(1 - \frac{\sum_{i=1}^{k} s_i^2}{s_{\text{total}}^2}\right)$$

IN [Python / Jupyter cell]

```
def cronbach_alpha(X):
    X = np.asarray(X, float); k = X.shape[1]
    return k / (k - 1) * (1 - X.var(axis=0, ddof=1).sum() / X.sum(axis=1).var(ddof=1))

scales = {"maths anxiety": ["Q1", "Q2", "Q3", "Q4"],
          "motivation": ["Q5", "Q6", "Q7", "Q8"],
          "social": ["Q9", "Q10", "Q11", "Q12"]}
for name, cols in scales.items():
    print(f"{name:<14} alpha = {cronbach_alpha(items[cols]):.3f}")

fscores = fa_vmx.transform(Zi)              # regression-method factor scores
sums = pd.DataFrame({k: items[v].sum(1) for k, v in scales.items()})
print("\ncorrelation of factor scores with sum scores:")
print(pd.DataFrame(np.corrcoef(fscores.T, sums.T.values)[:3, 3:],
                   index=["F1", "F2", "F3"], columns=sums.columns).round(2))
```

OUT

```
maths anxiety  alpha = 0.798
motivation     alpha = 0.789
social         alpha = 0.748

correlation of factor scores with sum scores:
    maths anxiety  motivation  social
F1           0.10       -0.98   -0.03
F2           1.00       -0.12    0.01
F3          -0.01       -0.03   -1.00
```

11.8 The factor_analyzer library

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
from factor_analyzer import FactorAnalyzer
from factor_analyzer.factor_analyzer import calculate_kmo, calculate_bartlett_sphericity

chi2, p = calculate_bartlett_sphericity(items)         # matches 11.4
kmo_items, kmo_total = calculate_kmo(items)            # matches 11.4

fa = FactorAnalyzer(n_factors=3, method="ml", rotation="promax")   # oblique rotation
fa.fit(items)
pd.DataFrame(fa.loadings_, index=items.columns)        # pattern loadings
```

```
fa.phi_                                                  # factor correlation matrix (oblique)
fa.get_communalities(); fa.get_uniquenesses()
fa.get_factor_variance()                                 # SS loadings, proportion, cumulative
ev, _ = fa.get_eigenvalues()                             # for the scree plot
scores = fa.transform(items)

FactorAnalyzer(n_factors=3, method="principal", rotation="varimax").fit(items)
```

RUN THIS IN YOUR JUPYTER

With rotation='promax', fa.phi_ should show a negative correlation of roughly −0.3 between the anxiety and motivation factors, recovering the design. Varimax cannot show this because it forces factors to be uncorrelated.

EXAM / VIVA ANGLE

Exploratory FA discovers structure; **confirmatory factor analysis** (CFA) tests a structure specified in advance, with fit indices such as CFI, TLI and RMSEA. In Python, use `semopy` (lavaan-style syntax: 'anxiety =~ Q1 + Q2 + Q3 + Q4'). Never run EFA and CFA on the same sample.

Chapter summary

PCA re-expresses total variance through orthogonal components; FA models the shared variance through latent factors plus unique errors. Standardise first. Check suitability (Bartlett, KMO), choose the number of factors by parallel analysis rather than the Kaiser rule alone, rotate for interpretability (oblique when factors plausibly correlate), interpret loadings above about 0.4, and report communalities, variance explained and reliability (Cronbach's α).

PRACTICAL TIP

References: Johnson & Wichern, *Applied Multivariate Statistical Analysis* (6th ed.), chapters 8 and 9; Hair, Black, Babin & Anderson, *Multivariate Data Analysis*, chapter 3; Fabrigar & Wegener, *Exploratory Factor Analysis*; Jolliffe, *Principal Component Analysis*; Everitt & Hothorn, *An Introduction to Applied Multivariate Analysis with R*.

CHAPTER 12

Cluster Analysis

Cluster analysis groups observations so that members of a cluster are more similar to each other than to members of other clusters. Unlike classification, there are no labels to learn from: the method must discover structure. That makes clustering powerful (customer segments, patient subtypes, document topics) and dangerous (every algorithm returns clusters, even on pure noise). This chapter covers distances, hierarchical methods, k-means, Gaussian mixtures and DBSCAN, and, crucially, how to choose the number of clusters and validate the result.

12.1 Distance and the importance of scaling

Measure	Formula	Use when
Euclidean	$\sqrt{\Sigma(x_i - y_i)^2}$	Continuous variables on comparable scales (default)
Manhattan (city block)	$\Sigma\lvert x_i - y_i\rvert$	Robust to outliers in single coordinates
Cosine distance	$1 - x \cdot y / (\lVert x\rVert \lVert y\rVert)$	Direction matters, not magnitude (text, profiles)
Mahalanobis	$\sqrt{((x - y)^T S^{-1}(x - y))}$	Correlated variables; accounts for covariance
Gower	Average of per-variable similarities	Mixed numeric and categorical data

Distance-based methods are dominated by the variables with the largest numerical range. In the wine data, proline ranges in the hundreds to over 1600 while hue ranges from 0.5 to 1.7, so unscaled clustering is essentially 'clustering on proline'. Standardise unless the variables share a natural common unit.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine, make_moons, make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (silhouette_score, silhouette_samples, adjusted_rand_score,
                             calinski_harabasz_score, davies_bouldin_score)
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster, cophenet
from scipy.spatial.distance import pdist, cdist
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

wb = load_wine(as_frame=True); W, truth = wb.data, wb.target.to_numpy()
print(W[["proline", "magnesium", "hue"]].describe().loc[["mean", "std", "min", "max"]].round(2))
Z = StandardScaler().fit_transform(W)

for name, data in [("raw", W.to_numpy()), ("standardised", Z)]:
    lab = KMeans(3, n_init=10, random_state=0).fit_predict(data)
    ari = adjusted_rand_score(truth, lab)
    print(f"k-means on {name:<12}: ARI vs true cultivars = {ari:.3f}")
```

OUT

```
      proline  magnesium   hue
mean   746.89      99.74  0.96
std    314.91      14.28  0.23
min    278.00      70.00  0.48
max   1680.00     162.00  1.71
```

```
k-means on raw         : ARI vs true cultivars = 0.371
k-means on standardised: ARI vs true cultivars = 0.897
```

The **adjusted Rand index** (ARI) measures agreement between two partitions, corrected for chance: 1 is perfect agreement, 0 is what random labelling gives. Standardising lifts it from about 0.37 to about 0.9. We can use ARI here only because the wine data happen to have known cultivars; in real clustering problems such labels are usually unavailable.

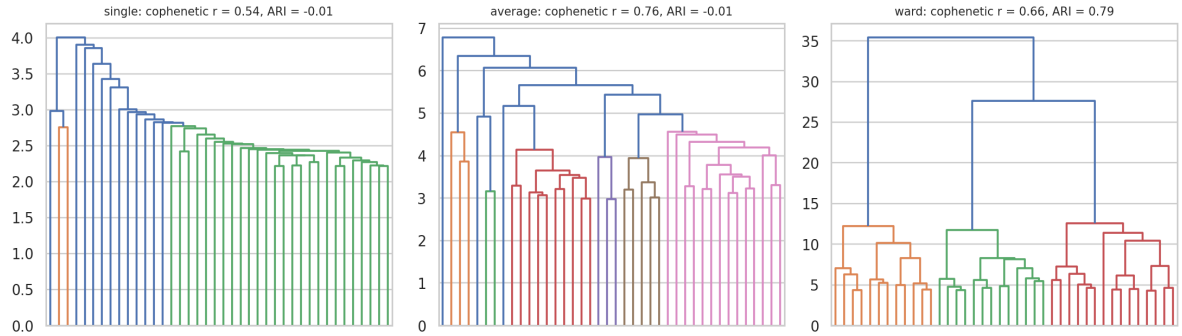
12.2 Hierarchical (agglomerative) clustering

Agglomerative clustering starts with every observation as its own cluster and repeatedly merges the two closest clusters. The **linkage** defines 'closest' between clusters. The full merge history is drawn as a **dendrogram**; cutting it at a height gives a partition.

Linkage	Distance between clusters A and B	Behaviour
Single	Minimum pairwise distance	Finds elongated shapes; prone to 'chaining'
Complete	Maximum pairwise distance	Compact clusters of similar diameter
Average (UPGMA)	Mean pairwise distance	Compromise; often a good default
Ward	Increase in within-cluster sum of squares	Compact, similar-size clusters; most popular with Euclidean data

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))
for a_, method in zip(ax, ["single", "average", "ward"]):
    Lk = linkage(Z, method=method)
    dendrogram(Lk, ax=a_, no_labels=True, color_threshold=None, truncate_mode="lastp", p=40)
    c, _ = cophenet(Lk, pdist(Z))
    lab = fcluster(Lk, t=3, criterion="maxclust")
    ari = adjusted_rand_score(truth, lab)
    a_.set_title(f"{method}: cophenetic r = {c:.2f}, ARI = {ari:.2f}", fontsize=9)
plt.tight_layout(); plt.show()
```



The **cophenetic correlation** measures how faithfully the dendrogram preserves the original pairwise distances. Average linkage scores highest on it here, yet cut at three clusters both single and average linkage put 174 of the 178 wines into one giant cluster, while Ward recovers the cultivars well. A faithful tree is not the same as a useful partition; the best linkage depends on what shape of cluster you expect.

IN [Python / Jupyter cell]

```
# Cutting the Ward tree and profiling the clusters
Lw = linkage(Z, method="ward")
hc = fcluster(Lw, t=3, criterion="maxclust")
print("cluster sizes:", np.bincount(hc)[1:])
print(pd.crosstab(hc, truth, rownames=["cluster"], colnames=["cultivar"]))
# sklearn equivalent
```

```
hc_sk = AgglomerativeClustering(n_clusters=3, linkage="ward").fit_predict(Z)
print("agreement scipy vs sklearn (ARI):", round(adjusted_rand_score(hc, hc_sk), 3))
```

OUT

```
cluster sizes: [56 64 58]
cultivar   0   1   2
cluster
1          0   8  48
2         59   5   0
3          0  58   0
agreement scipy vs sklearn (ARI): 1.0
```

12.3 K-means

K-means partitions the data into k clusters to minimise the within-cluster sum of squares (inertia). Lloyd's algorithm alternates two steps until assignments stop changing: assign each point to its nearest centroid, then move each centroid to the mean of its points. It converges to a local minimum, so it is run from several starts (`n_init`), and k-means++ chooses well-spread starting centroids.

$$\min_{C_1,\dots,C_k} \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

IN [Python / Jupyter cell]

```
def kmeans_scratch(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    cent = X[rng.choice(len(X), k, replace=False)]
    for it in range(iters):
        lab = np.argmin(cdist(X, cent), axis=1)                    # assignment step
        new = np.array([X[lab == j].mean(0) for j in range(k)])    # update step
        if np.allclose(new, cent): break
        cent = new
    inertia = sum(((X[lab == j] - cent[j])**2).sum() for j in range(k))
    return lab, cent, inertia, it

best = min((kmeans_scratch(Z, 3, seed=s) for s in range(10)), key=lambda r: r[2])
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Z)
print(f"scratch: inertia = {best[2]:.2f}, converged after {best[3]} iterations")
print(f"sklearn: inertia = {km.inertia_:.2f}")
print("agreement (ARI):", round(adjusted_rand_score(best[0], km.labels_), 3),
      "| ARI vs cultivars:", round(adjusted_rand_score(truth, km.labels_), 3))
```

OUT

```
scratch: inertia = 1277.93, converged after 6 iterations
sklearn: inertia = 1277.93
agreement (ARI): 1.0 | ARI vs cultivars: 0.897
```

WATCH OUT

K-means assumes roughly spherical clusters of similar size and spread, because it uses Euclidean distance to a single centre. It fails on elongated, nested or very unequal clusters (see 12.6) and is sensitive to outliers (k-medoids is a robust alternative).

12.4 Choosing the number of clusters

There is no single right answer, so use several criteria and look for agreement. The **elbow** method looks for the k after which inertia stops falling steeply. The **silhouette** of a point, s = (b − a)/max(a, b), compares its mean distance to its own cluster (a) with that to the nearest other cluster (b): near 1 is well clustered, near 0 is on a boundary, negative is probably misassigned.

Higher **Calinski-Harabasz** and lower **Davies-Bouldin** indices are better. The **gap statistic** compares inertia with that expected under a no-cluster reference distribution.

IN [Python / Jupyter cell]

```
ks = range(2, 9); rows = []
for k in ks:
    lab = KMeans(k, n_init=10, random_state=0).fit(Z)
    rows.append({"k": k, "inertia": lab.inertia_,
                 "silhouette": silhouette_score(Z, lab.labels_),
                 "Calinski-Harabasz": calinski_harabasz_score(Z, lab.labels_),
                 "Davies-Bouldin": davies_bouldin_score(Z, lab.labels_)})
crit = pd.DataFrame(rows).set_index("k"); print(crit.round(3))

def gap_statistic(X, ks, B=20, seed=0):
    rng = np.random.default_rng(seed); lo, hi = X.min(0), X.max(0); out = []
    for k in ks:
        Wk = np.log(KMeans(k, n_init=5, random_state=0).fit(X).inertia_)
        ref = [np.log(KMeans(k, n_init=5, random_state=0)
                      .fit(rng.uniform(lo, hi, X.shape)).inertia_) for _ in range(B)]
        out.append((k, np.mean(ref) - Wk, np.std(ref) * np.sqrt(1 + 1 / B)))
    return pd.DataFrame(out, columns=["k", "gap", "s_k"]).set_index("k")
gap = gap_statistic(Z, range(1, 8))
# Tibshirani's rule: smallest k with gap(k) >= gap(k+1) - s(k+1)
k_gap = next(k for k in gap.index[:-1] if gap.gap[k] >= gap.gap[k + 1] - gap.s_k[k + 1])
print("\ngap statistic chooses k =", k_gap)
```

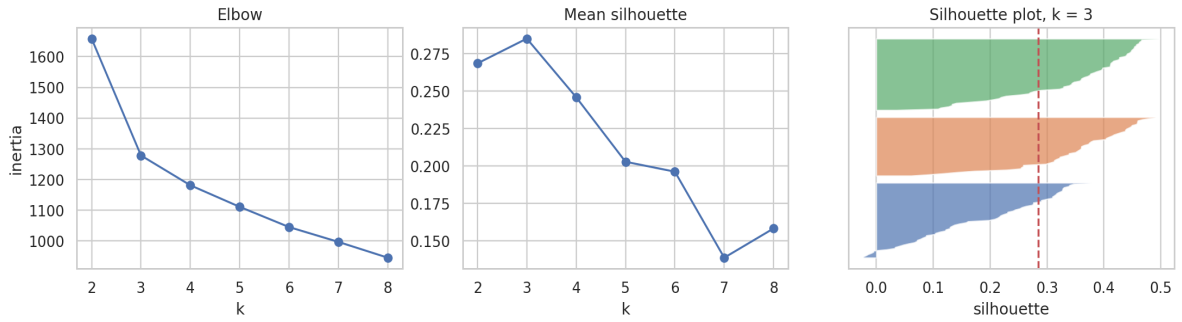
OUT

```
   inertia  silhouette  Calinski-Harabasz  Davies-Bouldin
k
2  1659.008       0.268             69.486           1.448
3  1277.928       0.285             70.940           1.389
4  1180.738       0.246             55.668           1.765
5  1110.365       0.203             46.883           1.820
6  1044.470       0.196             41.812           1.812
7   995.995       0.139             37.714           1.922
8   944.559       0.158             35.210           1.769

gap statistic chooses k = 3
```

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(1, 3, figsize=(13, 3.6))
ax[0].plot(crit.index, crit.inertia, "o-")
ax[0].set(title="Elbow", xlabel="k", ylabel="inertia")
ax[1].plot(crit.index, crit.silhouette, "o-"); ax[1].set(title="Mean silhouette", xlabel="k")
lab3 = km.labels_; s = silhouette_samples(Z, lab3); y0 = 0
for j in range(3):
    sj = np.sort(s[lab3 == j])
    ax[2].fill_betweenx(np.arange(y0, y0 + len(sj)), 0, sj, alpha=.7)
    y0 += len(sj) + 5
ax[2].axvline(s.mean(), color="r", ls="--"); ax[2].set(title="Silhouette plot, k = 3",
                                                       xlabel="silhouette", yticks=[])
plt.tight_layout(); plt.show()
```



12.5 Gaussian mixture models: probabilistic clustering

A **Gaussian mixture model** (GMM) assumes the data come from k multivariate normal distributions with their own means and covariance matrices, mixed in proportions π_j. It is fitted by the EM algorithm (an iterative maximum likelihood method, Chapter 13) and gives **soft** assignments: each point gets a probability of belonging to each cluster. Because it has a likelihood, the number of clusters and the covariance structure can be chosen by BIC. K-means is the limiting case of a GMM with equal spherical covariances.

$$p(\mathbf{x}) = \sum_{j=1}^{k} \pi_j N(\mathbf{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

IN [Python / Jupyter cell]

```
Z2 = PCA(n_components=5).fit_transform(Z)       # GMMs on 13-D with 178 rows overfit; reduce
bic = pd.DataFrame({cov: [GaussianMixture(k, covariance_type=cov, n_init=5, random_state=0)
                          .fit(Z2).bic(Z2) for k in range(1, 7)]
                    for cov in ["spherical", "diag", "tied", "full"]}, index=range(1, 7))
bic.index.name = "k"; print(bic.round(1))
k_best, cov_best = np.unravel_index(np.argmin(bic.values), bic.shape)
k_best += 1; cov_best = bic.columns[cov_best]
gmm = GaussianMixture(k_best, covariance_type=cov_best, n_init=5, random_state=0).fit(Z2)
prob = gmm.predict_proba(Z2)
print(f"\nBIC chooses k = {k_best}, covariance = '{cov_best}'; "
      f"ARI vs cultivars = {adjusted_rand_score(truth, gmm.predict(Z2)):.3f}")
print("share of points with max membership probability < 0.9:",
      np.mean(prob.max(1) < .9).round(3))
```

OUT

```
   spherical    diag    tied    full
k
1     3210.4  3038.5  3090.3  3090.3
2     3020.1  2991.8  2988.9  2965.8
3     2865.6  2904.0  2934.0  2914.6
4     2860.7  2880.1  2911.5  2933.8
5     2870.5  2876.4  2896.6  2928.8
6     2878.8  2900.2  2895.0  3014.1

BIC chooses k = 4, covariance = 'spherical'; ARI vs cultivars = 0.717
share of points with max membership probability < 0.9: 0.236
```

BIC prefers four spherical components, though k = 3 is nearly tied (the BIC values differ by about 5). This is typical: different criteria and models disagree at the margin, and the decision should also weigh interpretability and stability. The soft assignments are a real advantage: about a quarter of the wines have no cluster probability above 0.9, which flags them as genuinely intermediate rather than forcing a confident label.

12.6 DBSCAN: density-based clusters of any shape

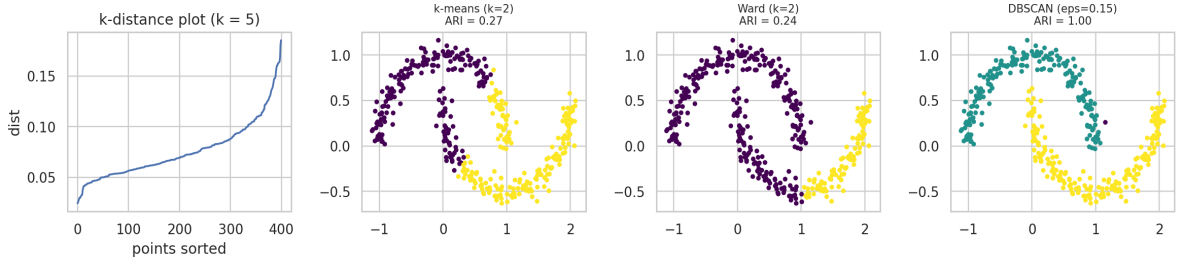
DBSCAN groups points that are densely packed (at least `min_samples` neighbours within radius `eps`) and labels isolated points as **noise** (label −1). It needs no k, finds non-convex shapes and handles outliers, but struggles when clusters have very different densities. A k-distance plot (sorted distance to the k-th nearest neighbour) helps choose eps: look for the knee.

IN [Python / Jupyter cell]

```
from sklearn.neighbors import NearestNeighbors
Xm, ym = make_moons(n_samples=400, noise=0.07, random_state=0)
d_k = np.sort(NearestNeighbors(n_neighbors=5).fit(Xm).kneighbors(Xm)[0][:, -1])

fig, ax = plt.subplots(1, 4, figsize=(14, 3.2))
```

```
ax[0].plot(d_k)
ax[0].set(title="k-distance plot (k = 5)", xlabel="points sorted", ylabel="dist")
models = {"k-means (k=2)": KMeans(2, n_init=10, random_state=0),
          "Ward (k=2)": AgglomerativeClustering(2, linkage="ward"),
          "DBSCAN (eps=0.15)": DBSCAN(eps=0.15, min_samples=5)}
for a_, (name, mdl) in zip(ax[1:], models.items()):
    lab = mdl.fit_predict(Xm)
    a_.scatter(Xm[:, 0], Xm[:, 1], c=lab, s=8, cmap="viridis")
    a_.set_title(f"{name}\nARI = {adjusted_rand_score(ym, lab):.2f}", fontsize=9)
plt.tight_layout(); plt.show()
```



12.7 Clusters are always found: a sanity check on noise

Run k-means on data with no structure at all and it will still return k tidy clusters. Compare your validity indices with what the same algorithm achieves on structureless data of the same size and spread. If your silhouette is no better than the noise benchmark, there is no real cluster structure to report.

IN [Python / Jupyter cell]

```
rng = np.random.default_rng(1)
noise = rng.normal(size=Z.shape)
for name, data in [("wine (real structure)", Z), ("pure noise", noise)]:
    lab = KMeans(3, n_init=10, random_state=0).fit_predict(data)
    print(f"{name:<22} silhouette at k=3: {silhouette_score(data, lab):.3f}")
```

OUT

```
wine (real structure)  silhouette at k=3: 0.285
pure noise             silhouette at k=3: 0.067
```

12.8 Profiling and interpreting clusters

Once clusters are chosen, describe them in the original units, ideally with a plot in a low-dimensional projection. Give each cluster a name a non-statistician understands. Be careful with formal tests: running ANOVA on the variables that *created* the clusters is circular and always 'significant'. Validate instead on external variables or with a hold-out sample.

IN [Python / Jupyter cell]

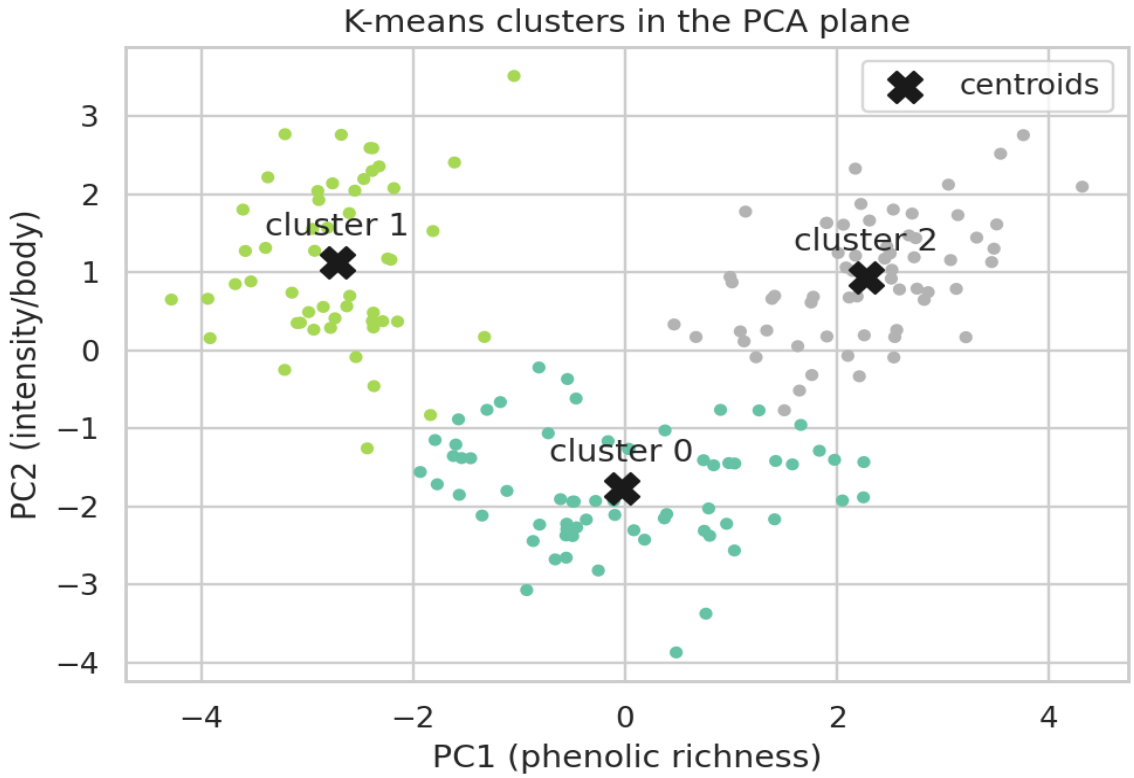
```
prof = W.assign(cluster=km.labels_).groupby("cluster").mean()
key = ["alcohol", "flavanoids", "color_intensity", "proline", "hue", "malic_acid"]
print(prof[key].round(2))
print("\nsizes:", np.bincount(km.labels_))

P2 = PCA(2).fit(Z); S2 = P2.transform(Z)
plt.figure(figsize=(6.5, 4.5))
plt.scatter(S2[:, 0], S2[:, 1], c=km.labels_, s=14, cmap="Set2")
cent2 = P2.transform(km.cluster_centers_)
plt.scatter(cent2[:, 0], cent2[:, 1], c="k", marker="X", s=150, label="centroids")
for j, (x_, y_) in enumerate(cent2): plt.annotate(f"cluster {j}", (x_, y_ + .35), ha="center")
plt.xlabel("PC1 (phenolic richness)"); plt.ylabel("PC2 (intensity/body)")
plt.title("K-means clusters in the PCA plane"); plt.legend(); plt.show()
```

OUT

```
         alcohol  flavanoids  color_intensity  proline   hue  malic_acid
cluster
0          12.25        2.05             2.97   510.17  1.06        1.90
1          13.13        0.82             7.23   619.06  0.69        3.31
2          13.68        3.00             5.45  1100.23  1.07        2.00

sizes: [65 51 62]
```



Method	Need k?	Cluster shape	Outliers	Scales to large n?	sklearn / scipy
K-means	Yes	Spherical, similar size	Sensitive	Yes (MiniBatchKMeans)	KMeans
Hierarchical (Ward)	Cut the tree	Compact	Sensitive	No ($O(n^2)$ memory)	linkage, AgglomerativeClustering
GMM	Yes (choose by BIC)	Ellipsoidal	Moderate	Moderate	GaussianMixture
DBSCAN / HDBSCAN	No	Arbitrary	Labels as noise	Yes	DBSCAN, HDBSCAN

Chapter summary

Standardise, choose a distance that suits the data, and match the algorithm to the cluster shapes you expect: k-means and Ward for compact groups, GMM for ellipsoids with soft membership, DBSCAN for arbitrary shapes and noise. Choose k with several criteria (elbow, silhouette, gap, BIC) and compare against a structureless benchmark. Validate with external information where possible, and profile clusters in interpretable units.

PRACTICAL TIP

References: Kaufman & Rousseeuw, *Finding Groups in Data*; Everitt, Landau, Leese & Stahl, *Cluster Analysis* (5th ed.); Hastie, Tibshirani & Friedman, *Elements of Statistical Learning*, chapter 14; Tibshirani, Walther & Hastie (2001), 'Estimating the number of clusters via the gap statistic'; Johnson & Wichern, chapter 12.

CHAPTER 13

Maximum Likelihood Methods

Maximum likelihood (ML) is the most important general method of estimation in statistics. The idea, due to R. A. Fisher, is simple: choose the parameter values that make the observed data most probable. It produces OLS for normal regression, the estimates in logistic regression, ARIMA and GMM, and it comes with a complete theory of standard errors and tests. This chapter builds ML from first principles in NumPy and SciPy, so the outputs of every model in this guide become transparent.

13.1 The likelihood function

For independent observations $x_1, \ldots, x_n$ from a density $f(x \mid \theta)$, the likelihood is the joint density viewed as a function of θ with the data fixed. Products of many small numbers underflow, so we work with the **log-likelihood**; the maximiser is the same because log is increasing. Optimisers minimise, so in code we minimise the *negative* log-likelihood.

$$L(\theta) = \prod_{i=1}^{n} f(x_i \mid \theta), \qquad \ell(\theta) = \sum_{i=1}^{n} \ln f(x_i \mid \theta), \qquad \hat{\theta}_{ML} = \arg\max_{\theta} \ell(\theta)$$

Worked example (Bernoulli). With k successes in n trials, $\ell(p) = k \ln p + (n - k) \ln(1 - p)$. Setting the score $d\ell/dp = k/p - (n - k)/(1 - p)$ to zero gives $\hat{p} = k/n$, the sample proportion.

IN [Python / Jupyter cell]

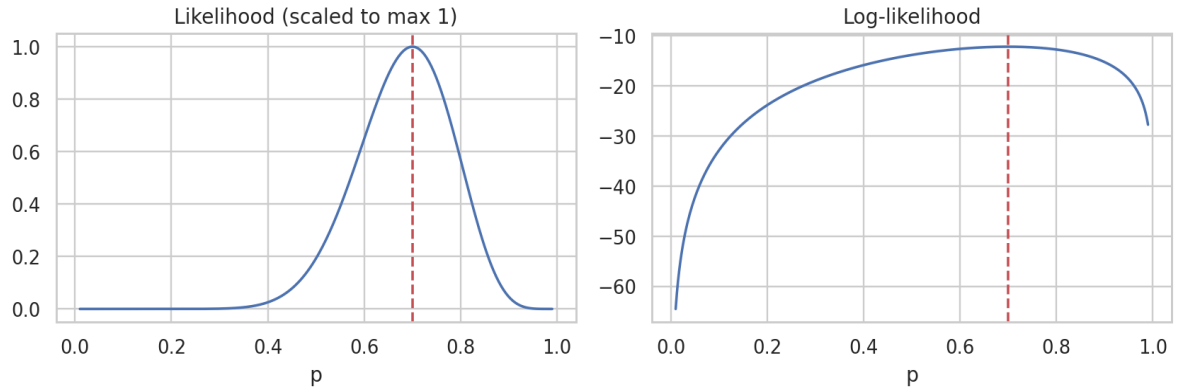
```
import numpy as np
import pandas as pd
import scipy.stats as stats
from scipy.optimize import minimize, minimize_scalar
import matplotlib.pyplot as plt

k, n = 14, 20
p = np.linspace(0.01, 0.99, 400)
loglik = k * np.log(p) + (n - k) * np.log(1 - p)
res = minimize_scalar(lambda q: -(k * np.log(q) + (n - k) * np.log(1 - q)),
                      bounds=(1e-6, 1 - 1e-6), method="bounded")
print(f"analytic MLE = {k / n:.4f} | numerical MLE = {res.x:.4f}")

fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
ax[0].plot(p, np.exp(loglik - loglik.max())); ax[0].set_title("Likelihood (scaled to max 1)")
ax[1].plot(p, loglik); ax[1].set_title("Log-likelihood")
for a_ in ax: a_.axvline(k / n, color="r", ls="--"); a_.set_xlabel("p")
plt.tight_layout(); plt.show()
```

OUT

```
analytic MLE = 0.7000 | numerical MLE = 0.7000
```



13.2 Closed-form MLEs for common models

Model	Log-likelihood (up to constants)	MLE
Bernoulli(p)	$\Sigma x \ln p + (n - \Sigma x) \ln(1 - p)$	$\hat{p} = \bar{x}$
Poisson(λ)	$\Sigma x \ln \lambda - n\lambda$	$\hat{\lambda} = \bar{x}$
Exponential(λ)	$n \ln \lambda - \lambda \Sigma x$	$\hat{\lambda} = 1/\bar{x}$
Normal(μ, σ^2)	$-(n/2) \ln \sigma^2 - \Sigma(x - \mu)^2/(2\sigma^2)$	$\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \Sigma(x - \bar{x})^2/n$ (biased!)
Uniform(0, θ)	$-n \ln \theta$ if $\theta \geq \max x$	$\hat{\theta} = \max x$ (boundary, not a zero derivative)

EXAM / VIVA ANGLE
The normal MLE of σ^2 divides by n, not n − 1. MLEs are often biased in small samples; their good properties are asymptotic. The uniform case shows that calculus is not always the route: there the likelihood is maximised at a boundary.

13.3 Numerical maximum likelihood

Most models have no closed form. The recipe is always the same: (1) write the negative log-likelihood as a Python function of a parameter vector; (2) reparametrise so the optimiser works on an unconstrained scale (optimise log σ rather than σ > 0); (3) call `scipy.optimize.minimize`; (4) get standard errors from the curvature at the optimum.

IN [Python / Jupyter cell]

```
rng = np.random.default_rng(13)
x = rng.normal(loc=10, scale=3, size=80)

def nll_normal(theta, x):
    mu, log_sigma = theta
    return -np.sum(stats.norm.logpdf(x, mu, np.exp(log_sigma)))

fit = minimize(nll_normal, x0=[0.0, 0.0], args=(x,), method="BFGS")
mu_hat, sigma_hat = fit.x[0], np.exp(fit.x[1])
print(f"numerical: mu = {mu_hat:.4f}, sigma = {sigma_hat:.4f}, converged = {fit.success}")
print(f"analytic : mu = {x.mean():.4f}, sigma = {x.std(ddof=0):.4f}  (ddof=0!)")
print("scipy norm.fit:", np.round(stats.norm.fit(x), 4))
```

OUT

```
numerical: mu = 10.4124, sigma = 2.9822, converged = True
analytic : mu = 10.4124, sigma = 2.9822  (ddof=0!)
scipy norm.fit: [10.4124  2.9822]
```

13.4 Properties of MLEs and the Fisher information

Under regularity conditions, MLEs are **consistent**, **asymptotically normal** and **asymptotically efficient** (they reach the Cramér-Rao lower bound). They are also **invariant**: the MLE of g(θ) is g(θ̂), so the MLE of σ is the square root of the MLE of σ^2. The variance of the MLE is governed by the **Fisher information**, the expected curvature of the log-likelihood: a sharply peaked likelihood means precise estimates.

$$I(\theta) = -E\left[\frac{\partial^2 \ell}{\partial \theta \, \partial \theta^\top}\right], \qquad \hat{\theta} \overset{a}{\sim} N(\theta,\, I(\theta)^{-1}), \qquad SE(\hat{\theta}_j) \approx \sqrt{[J(\hat{\theta})^{-1}]_{jj}}$$

In practice we use the **observed information** J(θ̂): the Hessian of the negative log-likelihood evaluated at the MLE. Below, a numerical Hessian by central finite differences, a routine worth keeping in your toolbox.

IN [Python / Jupyter cell]

```
def numerical_hessian(f, theta, eps=1e-4):
    theta = np.asarray(theta, float); k = len(theta); H = np.zeros((k, k))
    for i in range(k):
        for j in range(k):
            e_i, e_j = np.eye(k)[i] * eps, np.eye(k)[j] * eps
            H[i, j] = (f(theta + e_i + e_j) - f(theta + e_i - e_j)
                       - f(theta - e_i + e_j) + f(theta - e_i - e_j)) / (4 * eps**2)
    return H

# Work on the natural scale (mu, sigma) for interpretable SEs
nll_nat = lambda th: -np.sum(stats.norm.logpdf(x, th[0], th[1]))
H = numerical_hessian(nll_nat, [mu_hat, sigma_hat])
cov = np.linalg.inv(H); se = np.sqrt(np.diag(cov))
print("observed information matrix:\n", H.round(3))
print(f"SE(mu) = {se[0]:.4f}  (theory sigma/sqrt(n) = {sigma_hat / np.sqrt(len(x)):.4f})")
print(f"SE(sigma) = {se[1]:.4f}"
      f"  (theory sigma/sqrt(2n) = {sigma_hat / np.sqrt(2 * len(x)):.4f})")
print("BFGS inverse-Hessian SEs (mu, log-sigma scale):",
      np.sqrt(np.diag(fit.hess_inv)).round(4), "<- rough approximation only")
```

OUT

```
observed information matrix:
 [[ 8.996  0.   ]
 [ 0.    17.991]]
SE(mu) = 0.3334  (theory sigma/sqrt(n) = 0.3334)
SE(sigma) = 0.2358  (theory sigma/sqrt(2n) = 0.2358)
BFGS inverse-Hessian SEs (mu, log-sigma scale): [0.3383 0.0782] <- rough approximation only
```

IN [Python / Jupyter cell]

```
# Asymptotic normality by simulation: the Poisson MLE (lambda_hat = mean)
lam, n_obs = 3.0, 40
mles = rng.poisson(lam, size=(20_000, n_obs)).mean(axis=1)
print(f"mean of MLEs = {mles.mean():.4f} | SD = {mles.std():.4f} | "
      f"sqrt(1/I) = sqrt(lambda/n) = {np.sqrt(lam / n_obs):.4f}")
```

OUT

```
mean of MLEs = 2.9996 | SD = 0.2738 | sqrt(1/I) = sqrt(lambda/n) = 0.2739
```

13.5 The trinity of likelihood tests

Three asymptotically equivalent tests of H_0: θ = θ_0. The **Wald test** measures the distance between θ̂ and θ_0 in standard errors (it is what regression software prints). The **likelihood ratio test (LRT)** compares the maximised log-likelihoods of the full and restricted models. The **score (Lagrange multiplier) test** checks the slope of the log-likelihood at θ_0 and needs only the

restricted fit. For nested models with q restrictions:

$$W = \frac{(\hat\theta - \theta_0)^2}{\widehat{\mathrm{Var}}(\hat\theta)}, \qquad LR = 2\left[\ell(\hat\theta) - \ell(\hat\theta_0)\right], \qquad S = \frac{U(\theta_0)^2}{I(\theta_0)}; \qquad \text{all } \overset{a}{\sim} \chi^2_q$$

IN [Python / Jupyter cell]

```
# Poisson example: defect counts on n = 40 days. H0: lambda = 0.5 defects per day
counts = rng.poisson(0.35, 40); n_d = len(counts); lam_hat = counts.mean(); lam0 = 0.5
ll = lambda l: np.sum(stats.poisson.logpmf(counts, l))
W = (lam_hat - lam0)**2 / (lam_hat / n_d)                # Var(lam_hat) = lam/n at the MLE
LR = 2 * (ll(lam_hat) - ll(lam0))
U = counts.sum() / lam0 - n_d; I0 = n_d / lam0           # score and information at lambda0
S = U**2 / I0
print(f"lambda_hat = {lam_hat:.3f}")
for name, stat in [("Wald", W), ("Likelihood ratio", LR), ("Score", S)]:
    print(f"{name:<17} stat = {stat:.3f}, p = {stats.chi2.sf(stat, 1):.4f}")
```

OUT

```
lambda_hat = 0.150
Wald              stat = 32.667, p = 0.0000
Likelihood ratio  stat = 13.552, p = 0.0002
Score             stat = 9.800, p = 0.0017
```

All three reject, but the statistics differ noticeably. They agree only asymptotically; with 40 small counts the log-likelihood is far from quadratic. The LRT is usually the most reliable of the three in small samples, and the Wald test (which uses the variance at the estimate, here very small) the least.

LRT for nested distributions and model comparison

The exponential distribution is a gamma with shape 1. An LRT therefore tests whether the extra shape parameter is needed. For non-nested models (gamma versus lognormal), use AIC instead.

IN [Python / Jupyter cell]

```
waits = rng.gamma(shape=1.6, scale=2.0, size=150)          # true shape is not 1
ll_exp = np.sum(stats.expon.logpdf(waits, *stats.expon.fit(waits, floc=0)))
g_par = stats.gamma.fit(waits, floc=0)
ll_gam = np.sum(stats.gamma.logpdf(waits, *g_par))
LR = 2 * (ll_gam - ll_exp)
print(f"gamma shape MLE = {g_par[0]:.3f}")
print(f"LR = {LR:.2f}, p = {stats.chi2.sf(LR, df=1):.2e} -> exponential rejected")
print(f"AIC exponential = {2 * 1 - 2 * ll_exp:.1f}, AIC gamma = {2 * 2 - 2 * ll_gam:.1f}")
```

OUT

```
gamma shape MLE = 1.491
LR = 13.00, p = 3.11e-04 -> exponential rejected
AIC exponential = 652.4, AIC gamma = 641.4
```

13.6 Confidence intervals: Wald versus profile likelihood

A Wald interval $\hat\theta \pm 1.96$ SE assumes a symmetric, quadratic log-likelihood. When the likelihood is skewed (small samples, parameters near a boundary), the **profile likelihood interval** is better: it contains all θ_0 not rejected by the LRT, i.e. those with $2[\ell(\hat\theta) - \ell(\theta_0)] \le \chi^2_{1,\,0.95} = 3.84$. It is invariant to reparametrisation and never strays outside the parameter space.

IN [Python / Jupyter cell]

```
from scipy.optimize import brentq
k, n = 2, 25                                              # 2 successes in 25 trials
ll_b = lambda q: k * np.log(q) + (n - k) * np.log(1 - q)
ph = k / n; se_w = np.sqrt(ph * (1 - ph) / n)
```

```
cut = ll_b(ph) - stats.chi2.ppf(0.95, 1) / 2
lo = brentq(lambda q: ll_b(q) - cut, 1e-9, ph)
hi = brentq(lambda q: ll_b(q) - cut, ph, 1 - 1e-9)
print(f"Wald interval   : ({ph - 1.96 * se_w:.4f}, {ph + 1.96 * se_w:.4f})"
      "  <- negative lower limit!")
print(f"profile interval: ({lo:.4f}, {hi:.4f})")
```

OUT

```
Wald interval   : (-0.0263, 0.1863)  <- negative lower limit!
profile interval: (0.0138, 0.2271)
```

13.7 Linear regression as maximum likelihood

With normal errors, maximising the likelihood over β is the same as minimising the sum of squared residuals, so the ML estimate of β is exactly OLS. The ML estimate of σ^2 is SSE/n. This is the log-likelihood statsmodels reports, and the basis of AIC and BIC for regression models.

$$\ell(\beta, \sigma^2) = -\tfrac{n}{2}\ln(2\pi\sigma^2) - \tfrac{1}{2\sigma^2}\sum (y_i - x_i^\top \beta)^2$$

IN [Python / Jupyter cell]

```
n = 200
X = np.column_stack([np.ones(n), rng.normal(size=(n, 2))])
y = X @ np.array([1.0, 2.0, -0.5]) + rng.normal(0, 1.5, n)
nll_reg = lambda th: -np.sum(stats.norm.logpdf(y, X @ th[:3], np.exp(th[3])))
fit_r = minimize(nll_reg, x0=np.zeros(4), method="BFGS")
beta_ols, *_ = np.linalg.lstsq(X, y, rcond=None)
print("ML beta :", fit_r.x[:3].round(4))
print("OLS beta:", beta_ols.round(4))
print(f"ML sigma = {np.exp(fit_r.x[3]):.4f} = sqrt(SSE/n) = "
      f"{np.sqrt(np.sum((y - X @ beta_ols)**2) / n):.4f}")
```

OUT

```
ML beta : [ 0.8685  1.9862 -0.5833]
OLS beta: [ 0.8685  1.9862 -0.5833]
ML sigma = 1.5786 = sqrt(SSE/n) = 1.5786
```

13.8 An advanced example: censored survival data

ML shines when data are incomplete. In a reliability study, some components are still working when the study ends, so we only know their lifetime exceeds the censoring time. Each failure contributes its density f(t) to the likelihood; each censored unit contributes the survival probability S(t) = P(T > t). Ignoring censoring, or dropping censored units, badly underestimates lifetimes.

$$\ell(\theta) = \sum_{\text{failed}} \ln f(t_i \mid \theta) + \sum_{\text{censored}} \ln S(c_i \mid \theta)$$

IN [Python / Jupyter cell]

```
shape_true, scale_true = 1.8, 100.0
T = stats.weibull_min.rvs(shape_true, scale=scale_true, size=300, random_state=4)
C = rng.uniform(20, 150, 300)                       # random end-of-study times
time = np.minimum(T, C); failed = T <= C
print(f"censored share: {1 - failed.mean():.2f}")

def nll_weibull_cens(th):
    k_, lam_ = np.exp(th)
    return -(stats.weibull_min.logpdf(time[failed], k_, scale=lam_).sum()
             + stats.weibull_min.logsf(time[~failed], k_, scale=lam_).sum())

fit_c = minimize(nll_weibull_cens, x0=[0.0, np.log(time.mean())], method="Nelder-Mead")
```

```
k_hat, lam_hat = np.exp(fit_c.x)
naive = stats.weibull_min.fit(time[failed], floc=0)     # WRONG: ignores censoring
print(f"censored-data MLE: shape = {k_hat:.3f}, scale = {lam_hat:.1f}   (truth 1.8, 100)")
print(f"naive (failures only): shape = {naive[0]:.3f}, scale = {naive[2]:.1f}")
from scipy.special import gamma as G               # Weibull mean = scale * Gamma(1 + 1/k)
print(f"mean lifetime: MLE = {lam_hat * G(1 + 1 / k_hat):.1f}, "
      f"truth = {scale_true * G(1 + 1 / shape_true):.1f}")
```

OUT

```
censored share: 0.53
censored-data MLE: shape = 1.830, scale = 102.5   (truth 1.8, 100)
naive (failures only): shape = 2.005, scale = 65.6
mean lifetime: MLE = 91.1, truth = 88.9
```

13.9 The EM algorithm

When the likelihood involves unobserved (latent) variables, the **expectation-maximisation (EM)** algorithm maximises it by alternating two easy steps. **E-step**: given current parameters, compute the expected values of the latent variables (for a mixture, each point's membership probabilities). **M-step**: re-estimate the parameters as if those expectations were data (weighted means and variances). Each iteration never decreases the likelihood. This is exactly how the Gaussian mixture models of Chapter 12 are fitted.

IN [Python / Jupyter cell]

```
# Two-component normal mixture, from scratch
data = np.concatenate([rng.normal(-2, 1.0, 300), rng.normal(3, 1.5, 200)])
pi, mu, sd = 0.5, np.array([-1.0, 1.0]), np.array([1.0, 1.0])
history = []
for it in range(200):
    dens = np.column_stack([(1 - pi) * stats.norm.pdf(data, mu[0], sd[0]),
                            pi * stats.norm.pdf(data, mu[1], sd[1])])
    loglik = np.log(dens.sum(1)).sum(); history.append(loglik)
    r = dens / dens.sum(1, keepdims=True)                          # E-step: responsibilities
    nk = r.sum(0)                                                   # M-step: weighted estimates
    mu = (r * data[:, None]).sum(0) / nk
    sd = np.sqrt((r * (data[:, None] - mu)**2).sum(0) / nk)
    pi = nk[1] / len(data)
    if it > 0 and abs(history[-1] - history[-2]) < 1e-8: break
print(f"converged in {it} iterations, log-lik = {loglik:.3f}")
print(f"weights = ({1 - pi:.3f}, {pi:.3f}) (truth 0.6, 0.4)")
print(f"means = {mu.round(3)} (truth -2, 3) | SDs = {sd.round(3)} (truth 1, 1.5)")
print("log-likelihood never decreases:", bool(np.all(np.diff(history) >= -1e-9)))
```

OUT

```
converged in 37 iterations, log-lik = -1092.928
weights = (0.598, 0.402) (truth 0.6, 0.4)
means = [-1.984  2.957] (truth -2, 3) | SDs = [0.952 1.63 ] (truth 1, 1.5)
log-likelihood never decreases: True
```

13.10 Maximum likelihood in statsmodels

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import statsmodels.api as sm
from statsmodels.base.model import GenericLikelihoodModel

class WeibullCensored(GenericLikelihoodModel):
    """Any custom likelihood: define loglike, get SEs, CIs, AIC and a summary for free."""
    def loglike(self, params):
        k_, lam_ = np.exp(params)
        t, d = self.endog, self.exog[:, 0] == 1          # d: failure indicator
        return (stats.weibull_min.logpdf(t[d], k_, scale=lam_).sum()
                + stats.weibull_min.logsf(t[~d], k_, scale=lam_).sum())
```

```
mod = WeibullCensored(time, failed.astype(float)[:, None])
res = mod.fit(start_params=[0.0, np.log(time.mean())], method="nm", maxiter=5000)
print(res.summary())                  # log-scale estimates, SEs, z, p, CIs, AIC

# Every GLM in statsmodels is fitted by ML (IRLS): e.g. a Poisson regression
x_c = rng.normal(size=200); y_counts = rng.poisson(np.exp(0.5 + 0.8 * x_c))
print(sm.GLM(y_counts, sm.add_constant(x_c), family=sm.families.Poisson()).fit().summary())
```

RUN THIS IN YOUR JUPYTER

The point estimates match the executed censored-Weibull fit in 13.8 (after exponentiating). For survival analysis in practice, the lifelines library adds Kaplan-Meier curves, Cox regression and parametric models with censoring built in.

Chapter summary

Write the log-likelihood, maximise it (analytically or numerically on an unconstrained scale), and get standard errors from the inverse of the observed information (the Hessian). MLEs are consistent, asymptotically normal and efficient, and invariant to transformation. Test with Wald, likelihood ratio or score statistics, prefer profile-likelihood intervals in small samples, compare non-nested models with AIC, handle censoring by using survival probabilities, and use EM when latent variables make direct maximisation hard.

PRACTICAL TIP

References: Casella & Berger, chapters 7 and 10; Pawitan, *In All Likelihood: Statistical Modelling and Inference Using Likelihood*; Wasserman, *All of Statistics*, chapter 9; Millar, *Maximum Likelihood Estimation and Inference*; Dempster, Laird & Rubin (1977), the original EM paper.

CHAPTER 14

Logistic Regression

When the response is binary (pass/fail, disease/healthy, default/repay), linear regression is the wrong tool: it can predict probabilities below 0 or above 1, and its constant-variance assumption fails because a binary outcome's variance p(1 − p) depends on its mean. Logistic regression models the **log-odds** of the outcome as linear in the predictors. It is the workhorse of medical statistics, credit scoring and the social sciences, and the simplest member of the generalised linear model (GLM) family.

14.1 Odds, log-odds and the logistic function

If p is a probability, the **odds** are p/(1 − p) (p = 0.75 gives odds of 3, 'three to one') and the **log-odds** or **logit** is ln(p/(1 − p)), which ranges over the whole real line. Logistic regression sets the logit equal to a linear predictor η; inverting gives the S-shaped logistic (sigmoid) curve, which always stays between 0 and 1.

$$\operatorname{logit}(p_i) = \ln\frac{p_i}{1-p_i} = \eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik}, \qquad p_i = \frac{1}{1+e^{-\eta_i}}$$

Interpretation: a one-unit increase in x_j adds β_j to the log-odds, which **multiplies the odds** by $e^{\beta j}$, the **odds ratio**. The effect on the probability itself is not constant: it is largest near p = 0.5 and small near 0 or 1.

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()

import scipy.stats as stats
import matplotlib.pyplot as plt

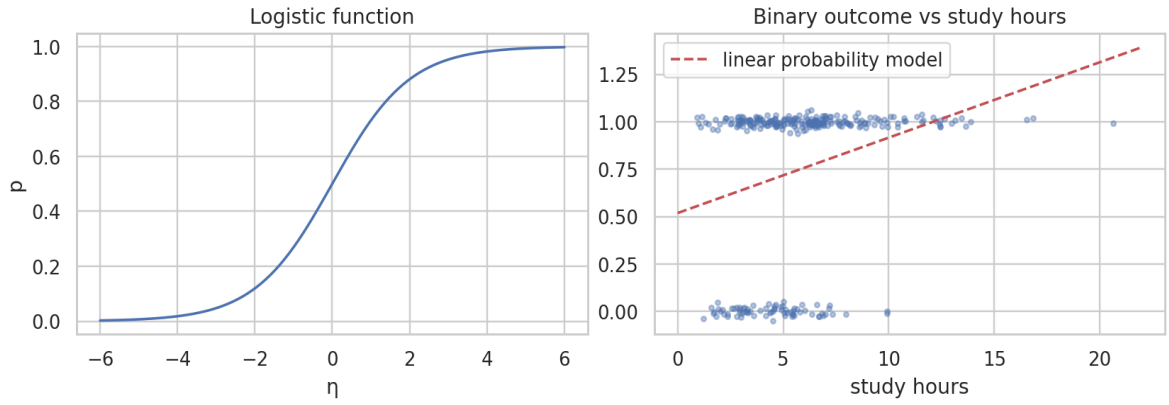
print("pass rate:", df.passed.mean())
# Why not linear regression? Fit a straight line to the 0/1 outcome
b = np.polyfit(df.study_hours, df.passed, 1)
```

```
grid = np.linspace(0, 22, 200)
print("linear probability model predicts at 20 hours:", round(np.polyval(b, 20), 3), "> 1!")

fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
eta = np.linspace(-6, 6, 200)
ax[0].plot(eta, 1 / (1 + np.exp(-eta))); ax[0].set(title="Logistic function", xlabel="η",
                                                   ylabel="p")
jit = np.random.default_rng(0).normal(0, .02, len(df))
ax[1].scatter(df.study_hours, df.passed + jit, s=8, alpha=.4)
ax[1].plot(grid, np.polyval(b, grid), "r--", label="linear probability model")
ax[1].set(title="Binary outcome vs study hours", xlabel="study hours"); ax[1].legend()
plt.tight_layout(); plt.show()
```

OUT

```
pass rate: 0.75
linear probability model predicts at 20 hours: 1.313 > 1!
```



14.2 Estimation by maximum likelihood (IRLS)

There is no closed form, so the coefficients are found by maximising the Bernoulli log-likelihood. Its gradient and Hessian are simple, and Newton-Raphson on this problem is known as **iteratively reweighted least squares** (IRLS): each step is a weighted regression with weights $w_i = p_i(1-p_i)$. The inverse of the information matrix $X^\top W X$ at convergence gives the standard errors.

$$\ell(\beta) = \sum_i [y_i \ln p_i + (1-y_i)\ln(1-p_i)], \qquad \nabla \ell = X^\top (y - p), \qquad -\nabla^2 \ell = X^\top W X$$

$$\beta^{(t+1)} = \beta^{(t)} + (X^\top W X)^{-1} X^\top (y - p), \qquad \widehat{\mathrm{Var}}(\hat\beta) = (X^\top \hat W X)^{-1}$$

IN [Python / Jupyter cell]

```
class Logit:
    """Logistic regression by Newton-Raphson / IRLS, with a full inference table."""
    def __init__(self, X, y, names):
        self.X, self.y, self.names = np.asarray(X, float), np.asarray(y, float), names

    def fit(self, tol=1e-10, max_iter=50):
        X, y = self.X, self.y; beta = np.zeros(X.shape[1])
        for it in range(max_iter):
            p = 1 / (1 + np.exp(-X @ beta))
            W = p * (1 - p)
            step = np.linalg.solve(X.T @ (X * W[:, None]), X.T @ (y - p))
            beta += step
            if np.max(np.abs(step)) < tol: break
        self.beta, self.iters = beta, it + 1
        self.p = 1 / (1 + np.exp(-X @ beta))
        self.cov = np.linalg.inv(X.T @ (X * (self.p * (1 - self.p))[:, None]))
        self.se = np.sqrt(np.diag(self.cov)); self.z = beta / self.se
```

```
        self.pval = 2 * stats.norm.sf(np.abs(self.z))
        eps = 1e-12
        self.llf = np.sum(y * np.log(self.p + eps) + (1 - y) * np.log(1 - self.p + eps))
        p0 = y.mean(); self.llnull = len(y) * (p0 * np.log(p0) + (1 - p0) * np.log(1 - p0))
        self.deviance, self.null_deviance = -2 * self.llf, -2 * self.llnull
        self.aic = -2 * self.llf + 2 * len(beta)
        self.pseudo_r2 = 1 - self.llf / self.llnull                 # McFadden
        return self

    def summary(self):
        tab = pd.DataFrame({"coef": self.beta, "std_err": self.se, "z": self.z,
                            "P>|z|": self.pval, "odds_ratio": np.exp(self.beta),
                            "OR_2.5%": np.exp(self.beta - 1.96 * self.se),
                            "OR_97.5%": np.exp(self.beta + 1.96 * self.se)}, index=self.names)
        print(tab.to_string(float_format=lambda v: f"{v:.4f}"))
        LR = self.null_deviance - self.deviance; k = len(self.beta) - 1
        print(f"converged in {self.iters} iterations | log-lik = {self.llf:.3f} | "
              f"AIC = {self.aic:.2f} | McFadden R2 = {self.pseudo_r2:.4f}")
        print(f"LR test vs null: chi2({k}) = {LR:.2f}, p = {stats.chi2.sf(LR, k):.2e}")

cols = ["study_hours", "prior_gpa", "attendance"]
X = np.column_stack([np.ones(len(df)), df[cols]])
lg = Logit(X, df.passed, ["Intercept"] + cols).fit()
lg.summary()
print("\ntrue coefficients: Intercept -9, study_hours 0.35, prior_gpa 2.0, attendance 0.03")
```

OUT

```
               coef  std_err       z  P>|z|  odds_ratio  OR_2.5%  OR_97.5%
Intercept   -9.7582   1.6888 -5.7783 0.0000      0.0001   0.0000    0.0016
study_hours  0.3601   0.0752  4.7882 0.0000      1.4334   1.2370    1.6611
prior_gpa    2.0758   0.3820  5.4341 0.0000      7.9708   3.7700   16.8523
attendance   0.0372   0.0115  3.2296 0.0012      1.0379   1.0147    1.0616
converged in 7 iterations | log-lik = -134.240 | AIC = 276.48 | McFadden R2 = 0.2043
LR test vs null: chi2(3) = 68.92, p = 7.27e-15

true coefficients: Intercept -9, study_hours 0.35, prior_gpa 2.0, attendance 0.03
```

IN [Python / Jupyter cell]

```
from sklearn.linear_model import LogisticRegression
sk = LogisticRegression(penalty=None, max_iter=1000).fit(df[cols], df.passed)
print("sklearn (no penalty):", np.round(np.r_[sk.intercept_, sk.coef_.ravel()], 4))
print("from scratch        :", lg.beta.round(4))
sk_def = LogisticRegression(max_iter=1000).fit(df[cols], df.passed)
print("sklearn DEFAULT     :", np.round(np.r_[sk_def.intercept_, sk_def.coef_.ravel()], 4),
      "<- L2-penalised!")
```

OUT

```
sklearn (no penalty): [-9.7582  0.3601  2.0758  0.0372]
from scratch        : [-9.7582  0.3601  2.0758  0.0372]
sklearn DEFAULT     : [-8.8218  0.3471  1.8187  0.0355] <- L2-penalised!
```

WATCH OUT

scikit-learn's `LogisticRegression` applies an L2 penalty by default (C = 1.0), so its coefficients are shrunk and are NOT the classical maximum likelihood estimates. For inference, use `penalty=None` (or statsmodels) and note that sklearn provides no standard errors.

14.3 Interpreting the model

Odds ratios. From the table: each extra weekly study hour multiplies the odds of passing by about $e^{0.36} \approx 1.43$ (a 43% increase in the odds), holding GPA and attendance fixed. One point of prior GPA multiplies the odds by roughly 7. Odds ratios for continuous predictors depend on the unit, so it is often clearer to report them per SD or per meaningful increment (e.g. 10 attendance points: $e^{10\beta}$).

Predicted probabilities and marginal effects. Odds ratios confuse non-specialists. Report predicted probabilities for typical profiles, and the **average marginal effect** (AME): the average, over the sample, of $\partial p/\partial x_j = \beta_j p_i(1-p_i)$, i.e. the change in pass probability per unit of x_j.

IN [Python / Jupyter cell]

```
def predict_proba(beta, rows):
    Xn = np.column_stack([np.ones(len(rows)), rows[cols]]); return 1 / (1 + np.exp(-Xn @ beta))

profiles = pd.DataFrame({"study_hours": [3, 6, 10, 6], "prior_gpa": [3.0, 3.0, 3.0, 2.5],
                         "attendance": [80, 80, 80, 80]})
profiles["P(pass)"] = predict_proba(lg.beta, profiles).round(3)
print(profiles)

ame = {c: np.mean(lg.beta[j + 1] * lg.p * (1 - lg.p)) for j, c in enumerate(cols)}
print("\naverage marginal effects (change in P(pass) per unit):")
print({k: round(float(v), 4) for k, v in ame.items()})
print("per 10 attendance points, odds ratio =", round(np.exp(10 * lg.beta[3]), 3))
```

OUT

```
   study_hours  prior_gpa  attendance  P(pass)
0            3        3.0          80    0.629
1            6        3.0          80    0.833
2           10        3.0          80    0.955
3            6        2.5          80    0.639

average marginal effects (change in P(pass) per unit):
{'study_hours': 0.0525, 'prior_gpa': 0.3025, 'attendance': 0.0054}
per 10 attendance points, odds ratio = 1.451
```

14.4 Comparing models: deviance and likelihood ratio tests

The **deviance** -2ℓ plays the role of the residual sum of squares. The difference in deviance between nested models is a likelihood ratio statistic with a χ^2 distribution, which is the right way to test several coefficients at once, or a categorical predictor with many levels.

IN [Python / Jupyter cell]

```
X_more = np.column_stack([X, df.sleep_hours,
                          pd.get_dummies(df.method, drop_first=True, dtype=float)])
lg_more = Logit(X_more, df.passed, ["Intercept"] + cols + ["sleep", "mB", "mC"]).fit()
LR = lg.deviance - lg_more.deviance
print(f"adding sleep and method: deviance {lg.deviance:.2f} -> {lg_more.deviance:.2f}, "
      f"LR chi2(3) = {LR:.2f}, p = {stats.chi2.sf(LR, 3):.3f}")
print(f"AIC: {lg.aic:.2f} (smaller model) vs {lg_more.aic:.2f}")
print("(truth: sleep and method do not affect passing in the generating model)")
```

OUT

```
adding sleep and method: deviance 268.48 -> 266.53, LR chi2(3) = 1.95, p = 0.582
AIC: 276.48 (smaller model) vs 280.53
(truth: sleep and method do not affect passing in the generating model)
```

Goodness of fit: the Hosmer-Lemeshow test

Group observations into deciles of predicted risk and compare observed with expected numbers of events. A small p-value signals poor calibration. The test depends on the grouping, so pair it with a calibration plot (14.6).

IN [Python / Jupyter cell]

```
def hosmer_lemeshow(y, p, g=10):
    d = pd.DataFrame({"y": y, "p": p})
    d["grp"] = pd.qcut(d.p, g, labels=False, duplicates="drop")
    t = d.groupby("grp").agg(obs=("y", "sum"), exp=("p", "sum"), n=("y", "size"))
    H = np.sum((t.obs - t.exp)**2 / (t.exp * (1 - t.exp / t.n)))
```

```
    return H, stats.chi2.sf(H, len(t) - 2), t

H, p_hl, tab = hosmer_lemeshow(df.passed, lg.p)
print(tab.round(2).T); print(f"\nHosmer-Lemeshow H = {H:.3f}, p = {p_hl:.3f}")
```

OUT

```
grp      0      1      2      3      4      5      6      7      8      9
obs   9.00  19.00  18.00  17.00  25.00  26.00  24.00  27.00  30.00  30.00
exp   9.38  15.63  18.84  21.12  23.38  24.79  26.21  27.52  28.62  29.52
n    30.00  30.00  30.00  30.00  30.00  30.00  30.00  30.00  30.00  30.00

Hosmer-Lemeshow H = 8.732, p = 0.365
```

14.5 Classification: thresholds, confusion matrices and ROC

To classify, predict 1 when $\hat{p}$ exceeds a threshold. The default of 0.5 is rarely optimal; the right threshold depends on the costs of the two kinds of error. The **ROC curve** plots the true positive rate against the false positive rate over all thresholds, and the **AUC** (area under it) is the probability that a random positive gets a higher score than a random negative: 0.5 is chance, 1 is perfect.

Metric	Formula	Question answered
Accuracy	(TP + TN) / N	How often is the classifier right? Misleading with imbalance.
Sensitivity (recall, TPR)	TP / (TP + FN)	Of the actual positives, how many did we catch?
Specificity (TNR)	TN / (TN + FP)	Of the actual negatives, how many did we clear?
Precision (PPV)	TP / (TP + FP)	Of the predicted positives, how many are real?
F1 score	2 · precision · recall / (precision + recall)	Balance of precision and recall
Brier score	mean $(\hat{p} - y)^2$	Accuracy of the probabilities themselves (lower is better)

We now switch to a real prediction problem: the Breast Cancer Wisconsin data (569 tumours, 30 features, malignant or benign), with a held-out test set.

IN [Python / Jupyter cell]

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegressionCV
from sklearn.metrics import (confusion_matrix, classification_report, roc_auc_score,
                             roc_curve, brier_score_loss)

bc = load_breast_cancer(as_frame=True)
Xb, yb = bc.data, 1 - bc.target          # recode so that 1 = malignant (the 'positive' class)
print("malignant share:", round(yb.mean(), 3))
X_tr, X_te, y_tr, y_te = train_test_split(Xb, yb, test_size=0.3, stratify=yb, random_state=0)

clf = make_pipeline(StandardScaler(),
                    LogisticRegressionCV(Cs=20, cv=5, l1_ratios=None, max_iter=5000))
clf.fit(X_tr, y_tr)
p_te = clf.predict_proba(X_te)[:, 1]
pred = (p_te >= 0.5).astype(int)
print("confusion matrix [rows = truth 0/1, cols = predicted 0/1]:\n",
      confusion_matrix(y_te, pred))
print(classification_report(y_te, pred, target_names=["benign", "malignant"], digits=3))
print(f"AUC = {roc_auc_score(y_te, p_te):.4f} | Brier = {brier_score_loss(y_te, p_te):.4f}")
```

OUT

```
malignant share: 0.373
confusion matrix [rows = truth 0/1, cols = predicted 0/1]:
 [[103   4]
 [  4  60]]
              precision    recall  f1-score   support

      benign      0.963     0.963     0.963       107
   malignant      0.938     0.938     0.938        64

    accuracy                          0.953       171
   macro avg      0.950     0.950     0.950       171
weighted avg      0.953     0.953     0.953       171

AUC = 0.9912 | Brier = 0.0342
```

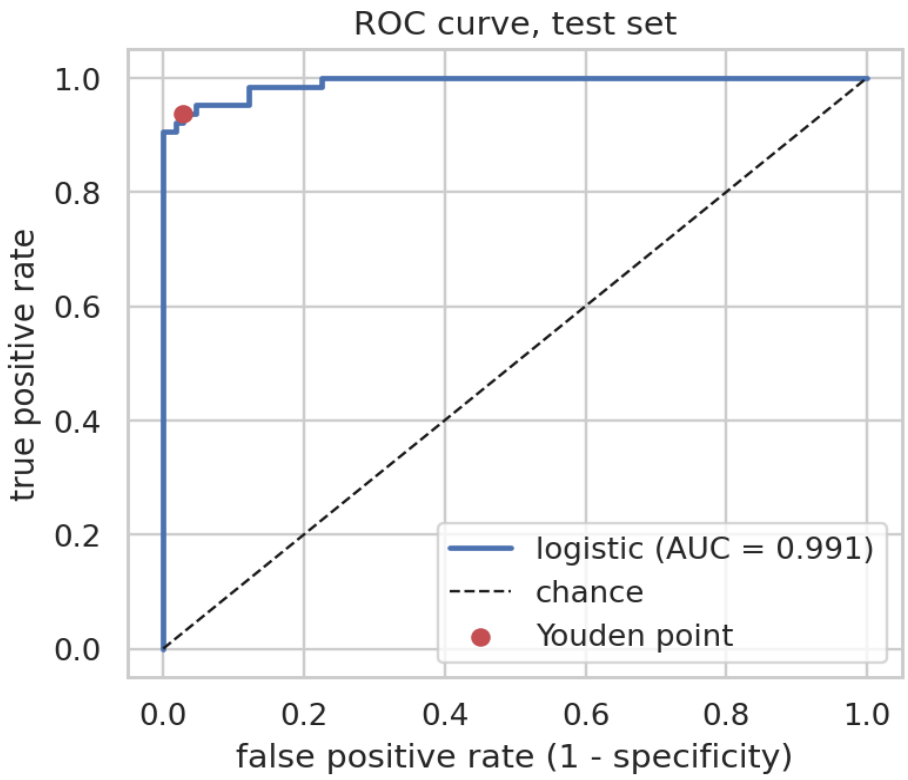
IN [Python / Jupyter cell]

```
fpr, tpr, thr = roc_curve(y_te, p_te)
j = np.argmax(tpr - fpr)              # Youden's J = sensitivity + specificity - 1
print(f"Youden-optimal threshold = {thr[j]:.3f}: sensitivity = {tpr[j]:.3f}, "
      f"specificity = {1 - fpr[j]:.3f}")
# A screening setting might demand >= 99% sensitivity: choose the threshold accordingly
k = np.argmax(tpr >= 0.99)
print(f"threshold for >= 99% sensitivity = {thr[k]:.3f}: specificity = {1 - fpr[k]:.3f}")

plt.figure(figsize=(5, 4.3))
plt.plot(fpr, tpr, lw=2, label=f"logistic (AUC = {roc_auc_score(y_te, p_te):.3f})")
plt.plot([0, 1], [0, 1], "k--", lw=1, label="chance")
plt.scatter(fpr[j], tpr[j], color="r", zorder=3, label="Youden point")
plt.xlabel("false positive rate (1 - specificity)"); plt.ylabel("true positive rate")
plt.title("ROC curve, test set"); plt.legend(); plt.show()
```

OUT

```
Youden-optimal threshold = 0.516: sensitivity = 0.938, specificity = 0.972
threshold for >= 99% sensitivity = 0.022: specificity = 0.776
```



14.6 Calibration

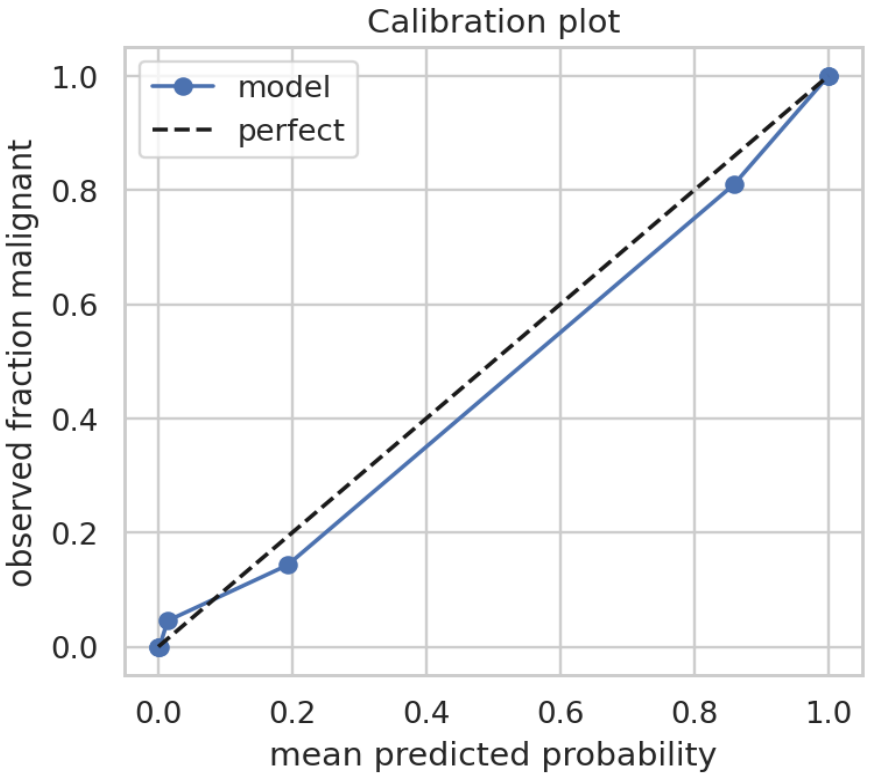
A model can rank cases well (high AUC) yet give poorly calibrated probabilities. For decisions based on risk (a 20% predicted probability should mean 20 in 100 such patients have the condition), check a **calibration plot**: observed event rate against mean predicted probability in bins.

IN [Python / Jupyter cell]

```
from sklearn.calibration import calibration_curve
frac_pos, mean_pred = calibration_curve(y_te, p_te, n_bins=8, strategy="quantile")
cv5 = StratifiedKFold(5, shuffle=True, random_state=0)
auc_cv = cross_val_score(clf, Xb, yb, cv=cv5, scoring="roc_auc")
print(f"5-fold CV AUC = {auc_cv.mean():.4f} ± {auc_cv.std():.4f}")
plt.figure(figsize=(4.8, 4.3))
plt.plot(mean_pred, frac_pos, "o-", label="model")
plt.plot([0, 1], [0, 1], "k--", label="perfect")
plt.xlabel("mean predicted probability"); plt.ylabel("observed fraction malignant")
plt.title("Calibration plot"); plt.legend(); plt.show()
```

OUT

```
5-fold CV AUC = 0.9950 ± 0.0058
```



14.7 Practical issues

Separation. If a predictor (or combination) perfectly separates the classes, the MLE does not exist: coefficients run off to infinity and standard errors explode. Symptoms are enormous coefficients and convergence warnings. Remedies are penalised (ridge) or Firth logistic regression, or a Bayesian prior (Chapter 15).

IN [Python / Jupyter cell]

```
xs = np.array([1, 2, 3, 4, 5, 6, 7, 8.]); ys = np.array([0, 0, 0, 0, 1, 1, 1, 1.])
try:
    sep = Logit(np.column_stack([np.ones(8), xs]), ys, ["Int", "x"]).fit(max_iter=25)
    print("coefficients after 25 Newton steps:", sep.beta.round(1), "| SEs:", sep.se.round(1))
except np.linalg.LinAlgError as e:
    print("Newton-Raphson broke down:", e)
pen = LogisticRegression(C=1.0).fit(xs.reshape(-1, 1), ys)
print("ridge-penalised fit stays finite:", np.round([pen.intercept_[0], pen.coef_[0, 0]], 3))
```

OUT

```
coefficients after 25 Newton steps: [-203.4   45.2] | SEs: [518402.7 114496. ]
ridge-penalised fit stays finite: [-5.264  1.17 ]
```

Class imbalance. With rare events, accuracy is meaningless (predicting 'no event' for everyone scores 99% if events are 1%). Use AUC, precision-recall curves and calibrated probabilities, choose thresholds by cost, and consider `class_weight='balanced'`. **Events per variable.** A classic rule asks for at least 10 to 20 events per estimated coefficient to avoid overfitting and biased estimates. **Linearity in the logit.** Check continuous predictors with splines or by binning, just as in linear regression.

14.8 More than two categories: multinomial logistic regression

For a nominal outcome with K classes, multinomial (softmax) regression models $K - 1$ log-odds relative to a reference class. For ordered outcomes (ratings, stages), the **proportional odds (ordinal logistic)** model uses a single set of slopes with $K - 1$ cut-points; see statsmodels' `OrderedModel`.

IN [Python / Jupyter cell]

```
from sklearn.datasets import load_iris
ir = load_iris(as_frame=True)
sepal = ir.data[["sepal length (cm)", "sepal width (cm)"]]
mn = make_pipeline(StandardScaler(), LogisticRegression(penalty=None, max_iter=5000))
acc = cross_val_score(mn, sepal, ir.target, cv=StratifiedKFold(5, shuffle=True, random_state=0))
print(f"multinomial logistic on 2 sepal features: CV accuracy = {acc.mean():.3f}")
mn.fit(sepal, ir.target)
print("P(setosa, versicolor, virginica) for 3 flowers:\n",
      mn.predict_proba(sepal.iloc[[0, 60, 120]]).round(3))
```

OUT

```
multinomial logistic on 2 sepal features: CV accuracy = 0.807
P(setosa, versicolor, virginica) for 3 flowers:
 [[1.    0.    0.   ]
 [0.    0.938 0.062]
 [0.    0.202 0.798]]
```

14.9 Logistic regression in statsmodels

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

m = smf.logit("passed ~ study_hours + prior_gpa + attendance", data=df).fit()
print(m.summary())                      # matches the from-scratch Logit table in 14.2
print(np.exp(m.params))                 # odds ratios
print(np.exp(m.conf_int()))             # CIs for odds ratios
print(m.get_margeff(at="overall").summary())    # average marginal effects (14.3)
m.llr, m.llr_pvalue, m.prsquared, m.aic         # LR test vs null, McFadden R2, AIC
```

```
m2 = smf.logit("passed ~ study_hours + prior_gpa + attendance + sleep_hours + C(method)",
               data=df).fit()
lr_stat = 2 * (m2.llf - m.llf)                  # LR test of the extra terms (14.4)

# The same model as a GLM (binomial family, logit link), fitted by IRLS
glm = smf.glm("passed ~ study_hours + prior_gpa + attendance", data=df,
              family=sm.families.Binomial()).fit()
print(glm.summary())                            # deviance, Pearson chi2

iris_df = ir.data.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
iris_df["species"] = ir.target
mn = smf.mnlogit("species ~ sepal_length + sepal_width", data=iris_df).fit()   # multinomial
```

RUN THIS IN YOUR JUPYTER

Coefficients, SEs, z values, p-values, log-likelihood, AIC and McFadden R^2 match the executed Logit class above; get_margeff reproduces the average marginal effects.

Chapter summary

Logistic regression models the log-odds as linear; coefficients exponentiate to odds ratios. It is fitted by maximum likelihood via IRLS, with Wald z tests and likelihood ratio (deviance) tests for nested models. Communicate with predicted probabilities and average marginal effects. For prediction, evaluate on held-out data with AUC, a confusion matrix at a cost-based threshold, and a calibration plot. Watch for separation, imbalance and too few events per variable, and remember that sklearn's default is penalised.

PRACTICAL TIP

References: Hosmer, Lemeshow & Sturdivant, *Applied Logistic Regression* (3rd ed.); Agresti, *Categorical Data Analysis*, chapters 4 to 8; ISLP chapter 4; McCullagh & Nelder, *Generalized Linear Models*; Harrell, *Regression Modeling Strategies*.

CHAPTER 15

Introduction to Bayesian Inference

Bayesian inference treats unknown parameters as random quantities with probability distributions that express our uncertainty. We start with a **prior** distribution, observe data through the **likelihood**, and combine them by Bayes' theorem (Chapter 2.5) into a **posterior** distribution. Everything we want (point estimates, intervals, probabilities of hypotheses, predictions) is then read directly off the posterior. This makes Bayesian answers unusually easy to state: 'there is a 95% probability that the pass rate lies between 0.70 and 0.80, given the data and the prior.'

$$\underset{\text{posterior}}{p(\theta \mid y)} = \frac{\overset{\text{likelihood}}{p(y \mid \theta)}\ \overset{\text{prior}}{p(\theta)}}{p(y)} \ \propto\ p(y \mid \theta)\, p(\theta), \qquad p(y) = \int p(y \mid \theta) p(\theta)\, d\theta$$

	Frequentist	Bayesian
Parameters	Fixed, unknown constants	Random variables with distributions
Probability means	Long-run frequency	Degree of belief, updated by evidence
Prior information	Not formally used	Encoded in the prior distribution
Interval	95% confidence interval: the procedure covers θ in 95% of repeated samples	95% credible interval: P(θ in interval | data) = 0.95
Testing	p-value: P(data this extreme | H_0)	P(H | data), posterior odds, Bayes factors
Computation	Optimisation (MLE)	Integration (conjugacy, grids, MCMC)

15.1 The beta-binomial model: conjugacy

For a probability p with binomial data (k successes in n trials), the **beta** distribution is the **conjugate prior**: a Beta(a, b) prior gives a Beta(a + k, b + n − k) posterior. The prior acts like a + b pseudo-observations, a of them successes. Beta(1, 1) is uniform (no preference), and as n grows the data overwhelm any reasonable prior.

$$p \sim \text{Beta}(a, b), \quad k \mid p \sim \text{Bin}(n, p) \ \Rightarrow\ p \mid k \sim \text{Beta}(a + k,\ b + n - k), \qquad E[p \mid k] = \frac{a + k}{a + b + n}$$

IN [Python / Jupyter cell]

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 100); pd.set_option("display.max_columns", 20)

def make_students(n=300, seed=42):
    """Synthetic 'student performance' data with KNOWN true parameters.
    Because we know the truth, we can check whether each method recovers it."""
    rng = np.random.default_rng(seed)
    method = rng.choice(["A", "B", "C"], size=n, p=[0.35, 0.35, 0.30])
    gender = rng.choice(["F", "M"], size=n)
    study = np.clip(rng.gamma(shape=4, scale=1.5, size=n), 0.5, None)   # hours/week
    sleep = np.clip(rng.normal(7, 1.1, n), 4, 10)                       # hours/night
    attend = np.clip(rng.beta(8, 2, n) * 100, 30, 100)                  # % attendance
    prior = np.clip(rng.normal(3.0, 0.45, n), 1.5, 4.0)                 # prior GPA
    effect = pd.Series(method).map({"A": 0.0, "B": 4.0, "C": 7.0}).to_numpy()
    score = (10 + 2.5 * study + 1.5 * sleep + 0.15 * attend + 6 * prior
             + effect + rng.normal(0, 6, n))
    score = np.clip(score, 0, 100)
    # pass/fail generated from a TRUE logistic model
```

```
    eta = -9 + 0.35 * study + 2.0 * prior + 0.03 * attend
    passed = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    return pd.DataFrame({"method": method, "gender": gender,
                         "study_hours": study.round(2), "sleep_hours": sleep.round(2),
                         "attendance": attend.round(1), "prior_gpa": prior.round(2),
                         "final_score": score.round(1), "passed": passed})

df = make_students()

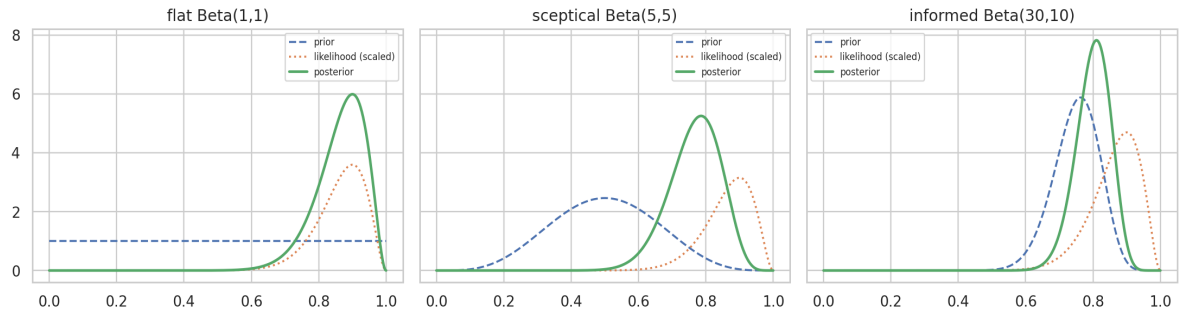
import scipy.stats as stats
import matplotlib.pyplot as plt

# Question: what is the pass rate under teaching method C? Data: the first 20 method-C students
c_students = df[df.method == "C"].head(20)
k, n = int(c_students.passed.sum()), len(c_students)
print(f"observed: {k} passes out of {n}")

priors = {"flat Beta(1,1)": (1, 1), "sceptical Beta(5,5)": (5, 5),
          "informed Beta(30,10)": (30, 10)}       # e.g. last year: about 75% from 40 students
grid = np.linspace(0, 1, 500)
fig, ax = plt.subplots(1, 3, figsize=(13, 3.5), sharey=True)
for a_, (name, (a, b)) in zip(ax, priors.items()):
    post = stats.beta(a + k, b + n - k)
    a_.plot(grid, stats.beta.pdf(grid, a, b), "--", label="prior")
    lik = stats.binom.pmf(k, n, grid)
    a_.plot(grid, lik / lik.max() * post.pdf(grid).max() * .6, ":", label="likelihood (scaled)")
    a_.plot(grid, post.pdf(grid), lw=2, label="posterior")
    a_.set_title(name); a_.legend(fontsize=7)
    lo, hi = post.ppf([.025, .975])
    print(f"{name:<22} posterior mean = {post.mean():.3f}, 95% credible interval = "
          f"({lo:.3f}, {hi:.3f})")
plt.tight_layout(); plt.show()
```

OUT

```
observed: 18 passes out of 20
flat Beta(1,1)         posterior mean = 0.864, 95% credible interval = (0.696, 0.970)
sceptical Beta(5,5)    posterior mean = 0.767, 95% credible interval = (0.603, 0.897)
informed Beta(30,10)   posterior mean = 0.800, 95% credible interval = (0.691, 0.890)
```



With a flat prior, the posterior mean (k + 1)/(n + 2) is close to the sample proportion and the credible interval is numerically similar to a frequentist interval. The informed prior pulls the estimate toward 0.75 and narrows the interval, because it adds 40 pseudo-observations. Whether that is appropriate depends on whether last year's students are genuinely comparable: priors must be justified, and a sensitivity analysis across reasonable priors (as here) is good practice.

Sequential updating

Today's posterior is tomorrow's prior. Updating with the data in batches gives exactly the same posterior as updating with all the data at once, which makes Bayesian methods natural for streaming data and adaptive trials.

IN [Python / Jupyter cell]

```
allC = df[df.method == "C"].passed.to_numpy()
a, b = 1, 1
for batch in np.array_split(allC, 4):
    a, b = a + batch.sum(), b + len(batch) - batch.sum()
    print(f"after {a + b - 2:>3} students: Beta({a}, {b}), mean = {a / (a + b):.3f}, "
          f"95% CrI = ({stats.beta.ppf(.025, a, b):.3f}, {stats.beta.ppf(.975, a, b):.3f})")
print("all at once:", f"Beta({1 + allC.sum()}, {1 + len(allC) - allC.sum()})")
```

OUT

```
after  22 students: Beta(20, 4), mean = 0.833, 95% CrI = (0.664, 0.950)
after  44 students: Beta(35, 11), mean = 0.761, 95% CrI = (0.629, 0.871)
after  66 students: Beta(51, 17), mean = 0.750, 95% CrI = (0.641, 0.845)
after  87 students: Beta(66, 23), mean = 0.742, 95% CrI = (0.646, 0.827)
all at once: Beta(66, 23)
```

15.2 Summarising a posterior

Common point summaries are the posterior **mean** (minimises squared-error loss), the **median** (absolute-error loss) and the **mode** (the MAP estimate, which equals the MLE under a flat prior). Intervals come in two kinds: the **equal-tailed** interval (2.5% in each tail) and the **highest posterior density (HPD)** interval, the narrowest interval containing 95% of the probability.

IN [Python / Jupyter cell]

```
post = stats.beta(1 + k, 1 + n - k)
mode = (k) / (n) if 0 < k < n else None                  # Beta(a,b) mode = (a-1)/(a+b-2)
print(f"mean = {post.mean():.4f}, median = {post.median():.4f}, mode (MAP) = {mode:.4f}")

def hpd(dist, mass=0.95, grid_n=10_000):
    lows = np.linspace(0, 1 - mass, grid_n)
    widths = dist.ppf(lows + mass) - dist.ppf(lows)
    i = np.argmin(widths); return dist.ppf(lows[i]), dist.ppf(lows[i] + mass)
print("equal-tailed 95%:", np.round(post.ppf([.025, .975]), 4))
print("HPD 95%         :", np.round(hpd(post), 4))
print(f"P(pass rate > 0.7 | data) = {post.sf(0.7):.4f}   <- a direct probability statement")
```

OUT

```
mean = 0.8636, median = 0.8747, mode (MAP) = 0.9000
equal-tailed 95%: [0.6962 0.9695]
HPD 95%         : [0.7234 0.9824]
P(pass rate > 0.7 | data) = 0.9729   <- a direct probability statement
```

15.3 Posterior predictive distribution

To predict new data, average the likelihood over the posterior: p(ỹ | y) = ∫ p(ỹ | θ) p(θ | y) dθ. This automatically includes parameter uncertainty, so predictive intervals are wider than those from plugging in a point estimate. For the beta-binomial it has a closed form (beta-binomial distribution); in general, simulate: draw θ from the posterior, then ỹ given θ.

IN [Python / Jupyter cell]

```
rng = np.random.default_rng(15)
a_post, b_post = 1 + k, 1 + n - k
theta = rng.beta(a_post, b_post, 100_000)
y_new = rng.binomial(30, theta)                          # passes among the next 30 students
exact = stats.betabinom(30, a_post, b_post)
print("posterior predictive 95% interval (simulated):", np.percentile(y_new, [2.5, 97.5]))
print("exact beta-binomial interval                 :", exact.ppf([.025, .975]))
plug = stats.binom(30, k / n)
print("plug-in binomial interval (ignores uncertainty):", plug.ppf([.025, .975]))
```

OUT

```
posterior predictive 95% interval (simulated): [19. 30.]
exact beta-binomial interval                 : [19. 30.]
plug-in binomial interval (ignores uncertainty): [23. 30.]
```

15.4 Other conjugate models

Likelihood	Conjugate prior	Posterior	Typical use
Binomial(n, p)	Beta(a, b)	Beta(a + k, b + n − k)	Rates, proportions, A/B tests
Poisson(λ)	Gamma(α, β)	Gamma(α + Σy, β + n)	Counts per unit time
Normal(μ, σ^2 known)	Normal(μ_0, τ_0^2)	Normal, precision-weighted mean	Measurements
Normal(μ known, σ^2)	Inverse-gamma	Inverse-gamma	Variances
Multinomial	Dirichlet	Dirichlet	Category shares

The normal-normal model shows Bayesian **shrinkage** clearly. The posterior mean is a precision-weighted average of the prior mean and the sample mean; the posterior precision is the sum of the two precisions (precision = 1/variance).

$$\mu \mid y \sim N\left(\frac{\mu_0/\tau_0^2 + n\bar{y}/\sigma^2}{1/\tau_0^2 + n/\sigma^2},\ \frac{1}{1/\tau_0^2 + n/\sigma^2}\right)$$

IN [Python / Jupyter cell]

```
# Mean exam score for method C, sigma assumed known (= 10), prior N(65, 5^2)
y = df[df.method == "C"].final_score.to_numpy(); sigma = 10.0
mu0, tau0 = 65.0, 5.0
for n_use in (5, 20, len(y)):
    ys = y[:n_use]
    prec = 1 / tau0**2 + n_use / sigma**2
    m_post = (mu0 / tau0**2 + n_use * ys.mean() / sigma**2) / prec
    w = (n_use / sigma**2) / prec
    print(f"n = {n_use:>3}: sample mean = {ys.mean():6.2f}, posterior mean = {m_post:6.2f}, "
          f"SD = {np.sqrt(1 / prec):.2f}, weight on data = {w:.2f}")

# Gamma-Poisson: defects per day with a Gamma(2, 1) prior
counts = rng.poisson(1.7, 30); al, be = 2 + counts.sum(), 1 + len(counts)
print(f"\nPoisson rate posterior: Gamma({al}, {be}), mean = {al / be:.3f}, "
      f"95% CrI = {np.round(stats.gamma.ppf([.025, .975], al, scale=1 / be), 3)}")
```

OUT

```
n =   5: sample mean =  74.82, posterior mean =  70.46, SD = 3.33, weight on data = 0.56
n =  20: sample mean =  75.01, posterior mean =  73.34, SD = 2.04, weight on data = 0.83
n =  87: sample mean =  72.92, posterior mean =  72.57, SD = 1.05, weight on data = 0.96

Poisson rate posterior: Gamma(65, 31), mean = 2.097, 95% CrI = [1.618 2.636]
```

15.5 Bayesian A/B testing

Comparing two proportions is especially natural in the Bayesian framework: sample both posteriors and compute any quantity of interest, such as $P(p_B > p_A)$, the distribution of the difference, or the expected loss from choosing the wrong option.

IN [Python / Jupyter cell]

```
res = {}
for m in ["A", "B"]:
    s = df[df.method == m].passed
    res[m] = rng.beta(1 + s.sum(), 1 + len(s) - s.sum(), 200_000)
diff = res["B"] - res["A"]
print(f"posterior mean pass rate A = {res['A'].mean():.3f}, B = {res['B'].mean():.3f}")
```

```
print(f"P(p_B > p_A | data) = {np.mean(diff > 0):.3f}")
print("95% credible interval for p_B - p_A:", np.round(np.percentile(diff, [2.5, 97.5]), 3))
print(f"expected loss if we choose B but A is better: {np.mean(np.maximum(-diff, 0)):.4f}")
```

OUT

```
posterior mean pass rate A = 0.796, B = 0.692
P(p_B > p_A | data) = 0.038
95% credible interval for p_B - p_A: [-0.219  0.011]
expected loss if we choose B but A is better: 0.1051
```

15.6 Grid approximation for non-conjugate models

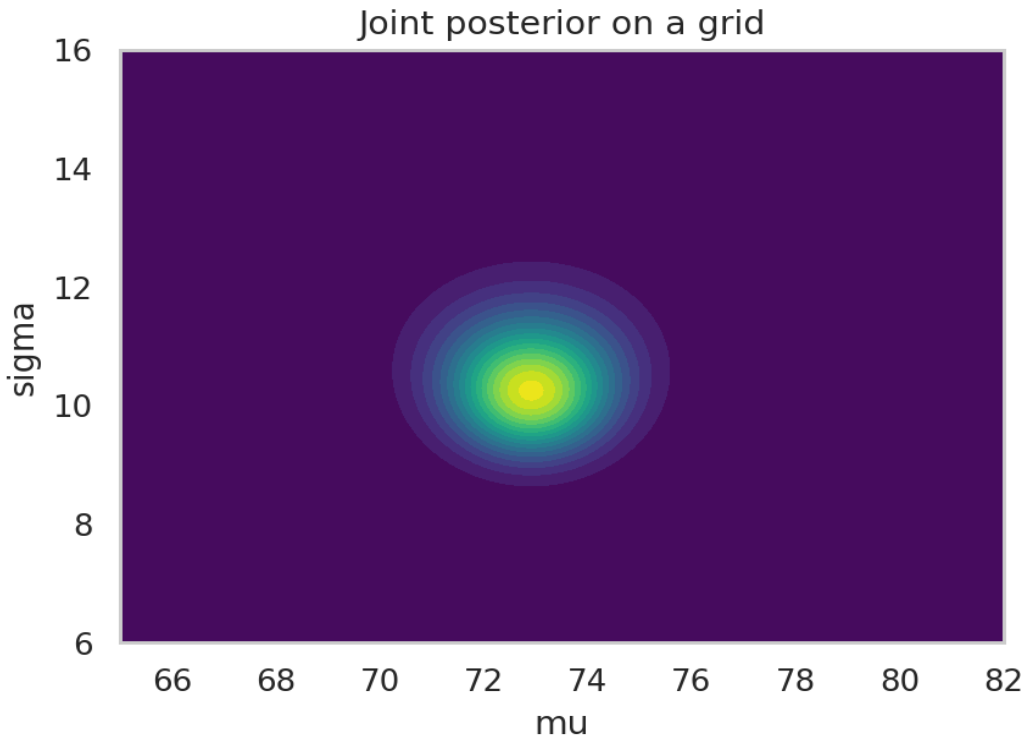
When the posterior has no closed form, but there are only one or two parameters, evaluate prior × likelihood on a grid and normalise. This makes the mechanics completely visible. Here we infer both the mean and the SD of method C scores, with weakly informative priors.

IN [Python / Jupyter cell]

```
mu_g = np.linspace(65, 82, 300); sd_g = np.linspace(6, 16, 300)
M, Sg = np.meshgrid(mu_g, sd_g)
log_prior = stats.norm.logpdf(M, 70, 20) + stats.halfnorm.logpdf(Sg, scale=20)
log_lik = stats.norm.logpdf(y[:, None, None], M, Sg).sum(axis=0)
log_post = log_prior + log_lik
post_g = np.exp(log_post - log_post.max()); post_g /= post_g.sum()
mu_marg = post_g.sum(axis=0); sd_marg = post_g.sum(axis=1)
mu_mean = np.sum(mu_g * mu_marg); sd_mean = np.sum(sd_g * sd_marg)
cdf = np.cumsum(mu_marg)
print(f"posterior mean of mu = {mu_mean:.3f}, 95% CrI = "
      f"({mu_g[np.searchsorted(cdf, .025)]:.2f}, {mu_g[np.searchsorted(cdf, .975)]:.2f})")
print(f"posterior mean of sigma = {sd_mean:.3f}")
plt.figure(figsize=(5.5, 4))
plt.contourf(M, Sg, post_g, levels=20, cmap="viridis")
plt.xlabel("mu"); plt.ylabel("sigma"); plt.title("Joint posterior on a grid"); plt.show()
```

OUT

```
posterior mean of mu = 72.913, 95% CrI = (70.69, 75.12)
posterior mean of sigma = 10.457
```



KEY IDEA
Grids scale exponentially: 300 points per parameter is 300^d evaluations, already 2.7×10^7 for three parameters. For realistic models we need Markov chain Monte Carlo.

15.7 Markov chain Monte Carlo: Metropolis-Hastings from scratch

MCMC draws a dependent sequence of samples whose long-run distribution is the posterior, using only the *unnormalised* posterior (so the hard integral p(y) is never needed). The random-walk Metropolis algorithm: propose θ* = θ + noise; accept it with probability min(1, p(θ* | y)/p(θ | y)); otherwise stay put. We apply it to the logistic regression of Chapter 14 with weakly informative N(0, 5^2) priors on the coefficients, and run four chains to check convergence.

IN [Python / Jupyter cell]

```
cols = ["study_hours", "prior_gpa", "attendance"]
Xl = np.column_stack([np.ones(len(df)), df[cols]]); yl = df.passed.to_numpy()
means, sds = Xl[:, 1:].mean(0), Xl[:, 1:].std(0)
Xs = np.column_stack([np.ones(len(df)), (Xl[:, 1:] - means) / sds])   # standardised

def log_post(beta):
    eta = Xs @ beta
    loglik = np.sum(yl * eta - np.logaddexp(0, eta))          # stable Bernoulli log-lik
    return loglik + np.sum(stats.norm.logpdf(beta, 0, 5))

def metropolis(n_iter, start, step, seed):
    r = np.random.default_rng(seed); cur = np.array(start, float); lp = log_post(cur)
    out = np.empty((n_iter, len(cur))); acc = 0
    for i in range(n_iter):
        prop = cur + r.normal(0, step, len(cur))
        lp_prop = log_post(prop)
        if np.log(r.random()) < lp_prop - lp:                  # accept/reject
            cur, lp = prop, lp_prop; acc += 1
        out[i] = cur
    return out, acc / n_iter

chains, rates = [], []
for c in range(4):
    start = np.random.default_rng(100 + c).normal(0, 2, 4)      # dispersed starting points
    draws, rate = metropolis(12_000, start, step=0.2, seed=c)
    chains.append(draws[2000:]); rates.append(rate)            # discard 2000 burn-in draws
chains = np.array(chains)                                      # shape (chains, draws, params)
print("acceptance rates:", np.round(rates, 3), "(about 0.2 to 0.4 is good for random walk)")
```

OUT

```
acceptance rates: [0.291 0.295 0.293 0.29 ] (about 0.2 to 0.4 is good for random walk)
```

Convergence diagnostics: trace plots, R-hat and effective sample size

Always check that chains have converged before using the draws. **Trace plots** should look like stationary 'hairy caterpillars' that overlap across chains. The **Gelman-Rubin R̂** compares between-chain with within-chain variance; values below 1.01 indicate convergence. The **effective sample size** (ESS) discounts the draws for autocorrelation: 40,000 correlated draws carry the information of far fewer independent ones.

IN [Python / Jupyter cell]

```
def rhat(ch):
    m, n = ch.shape; B = n * ch.mean(1).var(ddof=1); Wv = ch.var(1, ddof=1).mean()
    return np.sqrt(((n - 1) / n * Wv + B / n) / Wv)
```

```
def ess(ch):
    x = ch.ravel() - ch.mean(); n = len(x); ac = []
    for lag in range(1, 1000):
        r = np.dot(x[:-lag], x[lag:]) / np.dot(x, x)
        if r < 0.05: break
        ac.append(r)
    return n / (1 + 2 * np.sum(ac))

# Back-transform standardised coefficients to the original scale
b_std = chains.reshape(-1, 4)
slopes = b_std[:, 1:] / sds
intercept = b_std[:, 0] - np.sum(b_std[:, 1:] * means / sds, axis=1)
orig = np.column_stack([intercept, slopes])
names = ["Intercept"] + cols
summ = pd.DataFrame({"mean": orig.mean(0), "sd": orig.std(0),
                     "2.5%": np.percentile(orig, 2.5, 0), "97.5%": np.percentile(orig, 97.5, 0),
                     "R_hat": [rhat(chains[:, :, j]) for j in range(4)],
                     "ESS": [ess(chains[:, :, j]) for j in range(4)]}, index=names)
print(summ.round(4))
print("\nMLE from Chapter 14: Intercept -9.758, study 0.360, gpa 2.076, attendance 0.037")
```

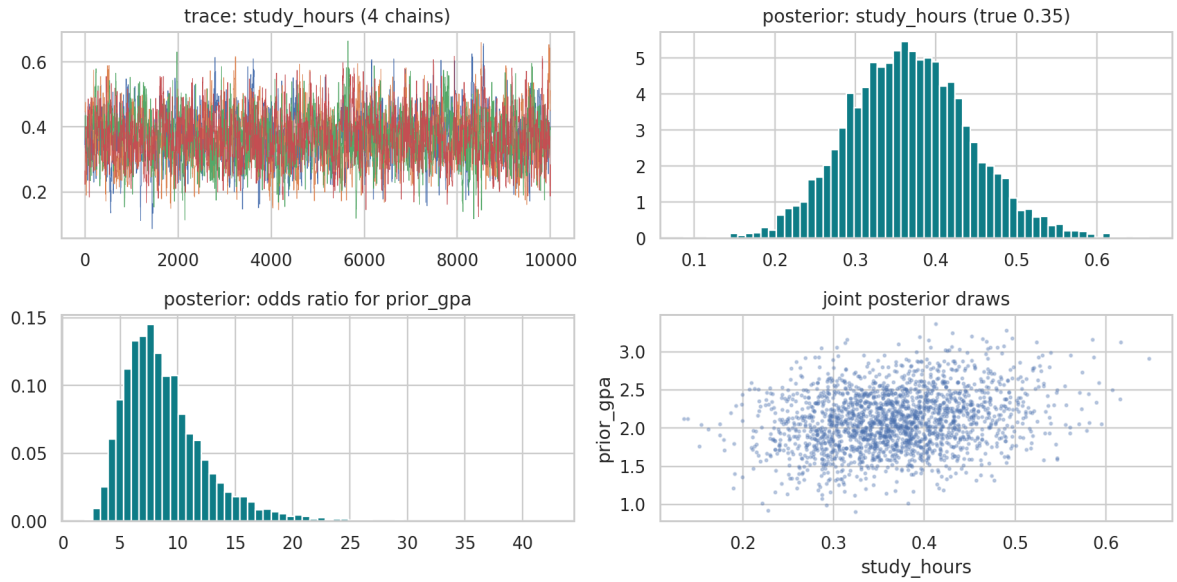
OUT

```
               mean      sd     2.5%   97.5%   R_hat        ESS
Intercept   -9.9002  1.6947 -13.3415 -6.6806  1.0006  1900.0103
study_hours  0.3669  0.0761   0.2214  0.5241  1.0006  1727.9849
prior_gpa    2.1078  0.3815   1.3805  2.8791  1.0001  2615.6347
attendance   0.0376  0.0119   0.0147  0.0607  1.0008  3083.0137

MLE from Chapter 14: Intercept -9.758, study 0.360, gpa 2.076, attendance 0.037
```

IN [Python / Jupyter cell]

```
fig, ax = plt.subplots(2, 2, figsize=(11, 5.5))
for j, a_ in enumerate(ax.ravel()):
    lab = names[j]
    if j == 0:
        for c in range(4): a_.plot(chains[c, :, 1] / sds[0], lw=.4)
        a_.set_title("trace: study_hours (4 chains)")
    elif j == 1:
        a_.hist(orig[:, 1], bins=60, density=True, color="#0E7C86")
        a_.set_title("posterior: study_hours (true 0.35)")
    elif j == 2:
        a_.hist(np.exp(orig[:, 2]), bins=60, density=True, color="#0E7C86")
        a_.set_title("posterior: odds ratio for prior_gpa")
    else:
        a_.scatter(orig[::20, 1], orig[::20, 2], s=3, alpha=.3)
        a_.set(title="joint posterior draws", xlabel="study_hours", ylabel="prior_gpa")
plt.tight_layout(); plt.show()
```



With n = 300 and weak priors, the posterior means and credible intervals sit very close to the MLEs and Wald intervals of Chapter 14, as asymptotic theory predicts (the Bernstein-von Mises theorem). The Bayesian version, however, delivers the full joint distribution, so any derived quantity (an odds ratio, a predicted probability for a profile, a difference between profiles) comes with its uncertainty for free: just transform the draws.

IN [Python / Jupyter cell]

```
# Posterior distribution of P(pass) for a student with 6 hours, GPA 3.0, 80% attendance
x_new = np.array([1, 6, 3.0, 80])
p_new = 1 / (1 + np.exp(-(orig @ x_new)))
print(f"P(pass) for this profile: posterior mean {p_new.mean():.3f}, "
      f"95% CrI ({np.percentile(p_new, 2.5):.3f}, {np.percentile(p_new, 97.5):.3f})")
```

OUT

```
P(pass) for this profile: posterior mean 0.835, 95% CrI (0.778, 0.885)
```

15.8 Bayes factors and Bayesian model comparison

The **Bayes factor** $BF_{10} = p(y \mid H_1)/p(y \mid H_0)$ is the ratio of marginal likelihoods, i.e. how much more probable the data are under H_1 than under H_0. It converts prior odds into posterior odds. Jeffreys' scale: 1 to 3 anecdotal, 3 to 10 moderate, 10 to 30 strong, above 100 decisive. Bayes factors can be sensitive to the prior under H_1, so report that prior. For predictive model comparison, Bayesians increasingly use leave-one-out cross-validation (LOO) or WAIC, available in ArviZ.

IN [Python / Jupyter cell]

```
from scipy.special import betaln, comb
# H0: p = 0.5 exactly (a fair coin). H1: p ~ Beta(1, 1). Data: 61 heads in 100 tosses
k, n = 61, 100
m0 = comb(n, k) * 0.5**n                                  # marginal likelihood under H0
m1 = comb(n, k) * np.exp(betaln(1 + k, 1 + n - k) - betaln(1, 1))
BF10 = m1 / m0
print(f"BF10 = {BF10:.3f}  -> posterior P(H1) with 50:50 prior odds = {BF10 / (1 + BF10):.3f}")
print(f"compare the two-sided p-value from Chapter 6: "
      f"{stats.binomtest(k, n, 0.5).pvalue:.4f}")
```

OUT

```
BF10 = 1.392  -> posterior P(H1) with 50:50 prior odds = 0.582
compare the two-sided p-value from Chapter 6: 0.0352
```

EXAM / VIVA ANGLE
A p-value of 0.035 sounds like strong evidence against H_0, yet the Bayes factor is close to 1: these data barely discriminate between 'fair coin' and 'some unknown bias'. This gap (the Jeffreys-Lindley phenomenon) is one reason p-values near 0.05 should be treated as weak evidence.

15.9 Bayesian linear regression and hierarchical models

With a Gaussian prior on the coefficients, Bayesian linear regression has a closed-form posterior, and its posterior mean is exactly ridge regression: the ridge penalty *is* a prior. scikit-learn's `BayesianRidge` estimates the prior precision from the data.

IN [Python / Jupyter cell]

```
from sklearn.linear_model import BayesianRidge, LinearRegression
num = ["study_hours", "sleep_hours", "attendance", "prior_gpa"]
Xr = pd.concat([df[num], pd.get_dummies(df.method, drop_first=True, dtype=float)], axis=1)
br = BayesianRidge(compute_score=True).fit(Xr, df.final_score)
ols = LinearRegression().fit(Xr, df.final_score)
_, sd_pred = br.predict(Xr.iloc[:3], return_std=True)
print(pd.DataFrame({"OLS": ols.coef_, "BayesianRidge": br.coef_}, index=Xr.columns).round(3))
print("predictive SD for 3 students:", np.asarray(sd_pred).round(2), "| estimated noise SD:",
      round(1 / np.sqrt(br.alpha_), 2))
```

OUT

```
               OLS  BayesianRidge
study_hours  2.297          2.292
sleep_hours  1.195          1.179
attendance   0.166          0.165
prior_gpa    6.127          5.926
B            3.672          3.409
C            7.657          7.285
predictive SD for 3 students: [7.62 7.3  7.66] | estimated noise SD: 6.09
```

Hierarchical (multilevel) models are where Bayesian methods are most distinctive. When estimating many related quantities (pass rates for 50 schools, effects in 20 hospitals), a hierarchical model treats the group parameters as draws from a common population distribution whose parameters are also estimated. Small groups are shrunk toward the overall mean (**partial pooling**), a compromise between estimating each group separately (no pooling, noisy) and ignoring groups (complete pooling, biased). This is best fitted with PyMC.

15.10 Probabilistic programming with PyMC and ArviZ

IN [Python / Jupyter cell: not executed in this PDF, run it in your own environment]

```
import pymc as pm
import arviz as az

# 1. Beta-binomial (15.1): should reproduce Beta(1 + k, 1 + n - k)
k_C = int(df[df.method == "C"].head(20).passed.sum())     # 18 passes out of 20
with pm.Model() as beta_binom:
    p = pm.Beta("p", alpha=1, beta=1)
    pm.Binomial("y", n=20, p=p, observed=k_C)
    idata = pm.sample(2000, tune=1000, chains=4, random_seed=1)
az.summary(idata, hdi_prob=0.95)                          # mean, sd, HDI, ess_bulk, r_hat

# 2. Bayesian logistic regression (15.7): NUTS is far more efficient than random walk
coords = {"coef": ["study_hours", "prior_gpa", "attendance"]}
with pm.Model(coords=coords) as logit_model:
    Xd = pm.Data("X", (df[cols] - df[cols].mean()) / df[cols].std())
    alpha = pm.Normal("alpha", 0, 5)
    beta = pm.Normal("beta", 0, 5, dims="coef")
```

```
    pm.Bernoulli("passed", logit_p=alpha + pm.math.dot(Xd, beta), observed=df.passed)
    idata = pm.sample(2000, tune=1000, chains=4, random_seed=1)
    idata.extend(pm.sample_posterior_predictive(idata))
az.plot_trace(idata); az.plot_posterior(idata, var_names=["beta"])
az.summary(idata, var_names=["alpha", "beta"])
az.plot_ppc(idata)                                          # posterior predictive check

# 3. Hierarchical model: pass rates by teaching method with partial pooling
idx, levels = pd.factorize(df.method)
with pm.Model(coords={"method": levels}) as hier:
    mu = pm.Normal("mu", 0, 1.5)                            # population mean (logit scale)
    tau = pm.HalfNormal("tau", 1)                           # between-method SD
    z = pm.Normal("z", 0, 1, dims="method")                 # non-centred parametrisation
    theta = pm.Deterministic("theta", pm.math.invlogit(mu + tau * z), dims="method")
    pm.Bernoulli("y", p=theta[idx], observed=df.passed)
    idata_h = pm.sample(2000, tune=2000, target_accept=0.95, random_seed=1)
az.summary(idata_h, var_names=["mu", "tau", "theta"])
# LOO comparison needs pointwise log-likelihoods:
# pm.compute_log_likelihood(idata, model=logit_model); az.loo(idata)
```

RUN THIS IN YOUR JUPYTER

Model 1 must reproduce the analytic Beta posterior of 15.1. Model 2's coefficients (after back-transforming from the standardised scale) should match the Metropolis results in 15.7, with ESS in the thousands and R-hat = 1.00, because NUTS mixes much better than a random walk. Bambi offers formula syntax on top of PyMC: bmb.Model('passed ~ study_hours + prior_gpa + attendance', df, family='bernoulli').

15.11 A Bayesian workflow

Gelman and colleagues recommend an iterative workflow: (1) specify the model and priors from subject knowledge, (2) run **prior predictive checks** (simulate data from the priors: are they plausible?), (3) fit, (4) check convergence ($\hat{R}$, ESS, divergences, traces), (5) run **posterior predictive checks** (does simulated data resemble the real data?), (6) test sensitivity to priors, (7) compare and expand models (LOO), and (8) communicate the posterior, not just a point estimate.

Chapter summary

Posterior $\propto$ likelihood $\times$ prior. Conjugate pairs (beta-binomial, gamma-Poisson, normal-normal) give closed forms and make shrinkage visible; grids work for one or two parameters; MCMC handles the rest, provided convergence is checked with traces, $\hat{R}$ and ESS. Credible intervals and statements like $P(p_B > p_A \mid \text{data})$ answer the questions people actually ask. With lots of data and weak priors, Bayesian and frequentist answers agree numerically; the Bayesian approach adds coherent uncertainty for any derived quantity, principled use of prior information, and hierarchical partial pooling.

PRACTICAL TIP

References: Gelman, Carlin, Stern, Dunson, Vehtari & Rubin, *Bayesian Data Analysis* (3rd ed., free online); McElreath, *Statistical Rethinking* (2nd ed.; PyMC ports of all examples exist); Kruschke, *Doing Bayesian Data Analysis*; Martin, Kumar & Lao, *Bayesian Modeling and Computation in Python* (free online); Downey, *Think Bayes* (2nd ed.).

CHAPTER 16

Reference Appendix: Choosing Methods, Cheat Sheets and Reading List

This appendix is designed for quick lookup during coursework, revision and real projects. Start with the question you are asking and the type of your outcome variable; the tables point you to the method, the Python function, and the chapter where it is explained and executed.

A. Which test should I use?

Comparing groups on a numeric outcome

Design	Assumptions met (parametric)	Assumptions doubtful (nonparametric)	Chapter
One sample vs a value	One-sample t: stats.ttest_1samp	Wilcoxon signed-rank / sign test	6.3, 6.7
Two independent groups	Welch t: stats.ttest_ind(equal_var=False)	Mann-Whitney U; permutation test	6.4, 6.7
Two paired measurements	Paired t: stats.ttest_rel	Wilcoxon signed-rank: stats.wilcoxon	6.4, 6.7
3+ independent groups	One-way ANOVA (Welch if variances differ)	Kruskal-Wallis: stats.kruskal	7.2 to 7.4
3+ repeated measures	Repeated-measures ANOVA / mixed model	Friedman: stats.friedmanchisquare	7.8
Two factors	Two-way ANOVA (smf.ols + anova_lm)	Aligned rank transform; permutation	7.6
Groups + covariate	ANCOVA (regression with factor + covariate)	Robust regression	7.7
After a significant ANOVA	Tukey HSD, Games-Howell, Dunnett, contrasts	Dunn's test with Holm correction	7.5

Categorical outcomes and associations

Question	Method	Python	Chapter
One proportion vs a value	Exact binomial test; Wilson CI	stats.binomtest	5.5, 6.5
Two proportions	Two-proportion z test; Fisher if small	proportions_ztest; stats.fisher_exact	6.5
Two categorical variables	Chi-square test of independence; Cramér's V	stats.chi2_contingency	6.5
Observed vs expected counts	Chi-square goodness of fit	stats.chisquare	6.5
Two numeric variables	Pearson r (linear); Spearman ρ / Kendall τ (monotonic)	stats.pearsonr, spearmanr	1.7
Data fit a distribution?	Q-Q plot; Shapiro-Wilk; Anderson-Darling; goodness_of_fit	stats.shapiro, stats.goodness_of_fit	3.5, 6.6
Equal variances?	Levene (Brown-Forsythe)	stats.levene(center='median')	6.6

Choosing a model by outcome type

Outcome	Model	Python	Chapter
Continuous	Linear regression (OLS), with robust SEs if heteroscedastic	smf.ols, LinearRegression	8, 9
Continuous, many predictors	Ridge, lasso, elastic net	RidgeCV, LassoCV	9.11
Binary	Logistic regression	smf.logit, LogisticRegression	14
Nominal, 3+ categories	Multinomial logistic	smf.mnlogit	14.8
Ordered categories	Proportional odds model	OrderedModel	14.8
Counts	Poisson GLM; negative binomial if overdispersed	sm.GLM(family=Poisson())	13.10
Time to event (censored)	Parametric survival by ML; Cox model	custom likelihood; lifelines	13.8
Time-ordered	ETS / Holt-Winters, SARIMA, lag regression	ExponentialSmoothing, SARIMAX	10
Clustered / repeated	Mixed (multilevel) models; hierarchical Bayes	smf.mixedlm; PyMC	7.8, 15.9
No outcome: structure	PCA, factor analysis, clustering	PCA, FactorAnalysis, KMeans	11, 12

B. Function cheat sheet

Task	Call	Note
Seeded random numbers	`rng = np.random.default_rng(42)`	Always seed for reproducibility
Summary statistics	`df.describe(); df.groupby(g)[y].agg([...])`	Pass ddof=1 for sample SD
Distribution quantities	`stats.norm(mu, sd).cdf / sf / ppf / rvs`	scale = SD, not variance
Discrete upper tail	`P(X ≥ k) = dist.sf(k - 1)`	Off-by-one trap
t confidence interval	`stats.t.interval(0.95, n-1, loc=m, scale=se)`	or res.confidence_interval()
Bootstrap CI	`stats.bootstrap((x,), np.median, method='BCa')`	Any statistic
Permutation test	`stats.permutation_test(data, stat_fn)`	Minimal assumptions
Multiple testing	`stats.false_discovery_control(p) / multipletests`	BH, Holm, Bonferroni
Power	`noncentral t; TTestIndPower().solve_power`	Plan before collecting data
OLS with summary	`smf.ols('y ~ x1 + C(g)', data=df).fit().summary()`	C() for categorical
Robust SEs	`.fit(cov_type='HC3')`	Heteroscedasticity
Nested model test	`anova_lm(small, big); res.compare_f_test(small)`	F test
VIF	`variance_inflation_factor(X.values, i)`	> 5 to 10: collinearity
Logistic regression	`smf.logit(...).fit(); np.exp(res.params)`	Odds ratios
sklearn logistic (no penalty)	`LogisticRegression(penalty=None)`	Default is L2!
Cross-validation	`cross_val_score(pipe, X, y, cv=KFold(...))`	Pipeline the scaler
ROC / AUC	`roc_curve, roc_auc_score`	Choose threshold by cost
Decomposition	`STL(y, period=12).fit()`	Robust, modern
Stationarity	`adfuller(y); kpss(y)`	Opposite null hypotheses

Task	Call	Note
SARIMA	`SARIMAX(y, order=, seasonal_order=).fit()`	get_forecast(h).conf_int()
PCA	`PCA().fit(StandardScaler().fit_transform(X))`	explained_variance_ratio_
Factor analysis	`FactorAnalysis(n, rotation='varimax')`	factor_analyzer for promax/KMO
K-means / hierarchical	`KMeans(k, n_init=10); linkage(Z, 'ward')`	Standardise first
GMM	`GaussianMixture(k, covariance_type=).bic(X)`	Choose k by BIC
Numerical MLE	`minimize(neg_loglik, x0, method='BFGS')`	Optimise log of positive params
MCMC	`pm.sample(); az.summary(idata)`	Check R-hat, ESS, divergences

C. Essential formula sheet

$$\bar{x} = \frac{1}{n}\sum x_i \qquad s^2 = \frac{1}{n-1}\sum (x_i - \bar{x})^2 \qquad SE(\bar{x}) = \frac{s}{\sqrt{n}} \qquad z = \frac{x-\mu}{\sigma}$$

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)} \qquad E[aX+b] = aE[X] + b \qquad \mathrm{Var}(X+Y) = \mathrm{Var}X + \mathrm{Var}Y + 2\mathrm{Cov}(X,Y)$$

$$\bar{x} \pm t_{n-1}\frac{s}{\sqrt{n}} \qquad \hat{p} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \qquad n = \left(\frac{z\sigma}{E}\right)^2$$

$$t = \frac{\bar{x}-\mu_0}{s/\sqrt{n}} \qquad t_{Welch} = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}} \qquad \chi^2 = \sum \frac{(O-E)^2}{E} \qquad d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}$$

$$F = \frac{SS_B/(k-1)}{SS_W/(N-k)} \qquad \eta^2 = \frac{SS_B}{SS_T} \qquad R^2 = 1 - \frac{SSE}{SST} \qquad \bar{R}^2 = 1 - (1-R^2)\frac{n-1}{n-p}$$

$$\hat{\beta} = (X^\top X)^{-1} X^\top y \qquad \mathrm{Var}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1} \qquad \hat{\beta}_1 = \frac{S_{xy}}{S_{xx}} = r\frac{s_y}{s_x} \qquad VIF_j = \frac{1}{1-R_j^2}$$

$$\mathrm{logit}(p) = X\beta \qquad OR = e^{\beta} \qquad AIC = -2\ell + 2p \qquad BIC = -2\ell + p\ln n \qquad LR = 2(\ell_1 - \ell_0) \sim \chi^2_q$$

$$\hat{\theta} \overset{a}{\sim} N(\theta, I(\theta)^{-1}) \qquad \text{posterior} \propto \text{likelihood} \times \text{prior} \qquad \mathrm{Beta}(a,b) \to \mathrm{Beta}(a+k, b+n-k)$$

$$\rho_k = \frac{\sum (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum (y_t - \bar{y})^2} \qquad Q_{LB} = n(n+2)\sum_{k=1}^{h} \frac{\hat{\rho}_k^2}{n-k} \qquad MASE = \frac{MAE}{MAE_{\text{seasonal naive}}}$$

D. Twelve pitfalls that cost marks (and credibility)

Pitfall	Do this instead
Using np.var / np.std without ddof=1 for sample statistics	Pass ddof=1, or use pandas defaults
Running many t tests instead of ANOVA, or many tests without correction	ANOVA + post-hoc; Holm or BH corrections
Reading 'not significant' as 'no effect'	Report the CI; equivalence test; power analysis
Interpreting a p-value as P(H_0 true)	It is P(data this extreme | H_0); see Bayes factors (15.8)
Choosing Student vs Welch t by a pre-test of variances	Use Welch by default
Checking normality of raw y instead of residuals	Check residuals (Q-Q plot first)

Pitfall	Do this instead
Calling a regression coefficient causal	Say 'associated with, holding X fixed'; think about confounders
Forgetting to standardise before PCA, k-means or penalised regression	StandardScaler inside a Pipeline
Treating sklearn's default LogisticRegression as the MLE	penalty=None, or statsmodels
Shuffling time series in cross-validation	TimeSeriesSplit / rolling origin
Selecting variables and then reporting naive p-values	Pre-specify the model; or use CV for prediction only
Using MCMC output without convergence checks	Traces, R-hat < 1.01, adequate ESS

E. Reading list

The guide draws on the standard texts below. They are grouped by role so you know where to go next. Several are freely and legally available online from their authors (marked 'free').

Area	Core texts
Probability	Blitzstein & Hwang, *Introduction to Probability* (free); Ross, *A First Course in Probability*
Mathematical statistics	Casella & Berger, *Statistical Inference*; Rice, *Mathematical Statistics and Data Analysis*; Wasserman, *All of Statistics*
Applied statistics with Python	Downey, *Think Stats* (free); McKinney, *Python for Data Analysis* (free online); VanderPlas, *Python Data Science Handbook* (free)
Resampling	Efron & Tibshirani, *An Introduction to the Bootstrap*
ANOVA and design	Montgomery, *Design and Analysis of Experiments*; Maxwell, Delaney & Kelley, *Designing Experiments and Analyzing Data*
Regression	Kutner et al., *Applied Linear Statistical Models*; Montgomery, Peck & Vining; Fox, *Applied Regression Analysis and GLMs*; Harrell, *Regression Modeling Strategies*
Statistical learning	James, Witten, Hastie, Tibshirani & Taylor, *ISLP: An Introduction to Statistical Learning with Applications in Python* (free); Hastie, Tibshirani & Friedman, *Elements of Statistical Learning* (free)
Time series	Hyndman & Athanasopoulos, *Forecasting: Principles and Practice* (free); Box, Jenkins, Reinsel & Ljung; Shumway & Stoffer (free)
Multivariate	Johnson & Wichern, *Applied Multivariate Statistical Analysis*; Hair et al., *Multivariate Data Analysis*; Everitt et al., *Cluster Analysis*
Likelihood	Pawitan, *In All Likelihood*; Millar, *Maximum Likelihood Estimation and Inference*
Categorical data / logistic	Agresti, *Categorical Data Analysis*; Hosmer, Lemeshow & Sturdivant, *Applied Logistic Regression*
Bayesian	Gelman et al., *Bayesian Data Analysis* (free); McElreath, *Statistical Rethinking*; Martin, Kumar & Lao, *Bayesian Modeling and Computation in Python* (free); Downey, *Think Bayes* (free)
Documentation	docs.scipy.org (scipy.stats); statsmodels.org; scikit-learn.org (User Guide); pymc.io; python.arviz.org

F. How to keep using this guide

A practical routine for any new data set: (1) Chapter 1 EDA cells to describe it; (2) the tables in section A to pick the method; (3) the executed cells of that chapter, replacing the data frame and column names with yours; (4) the diagnostics cells in the same chapter before you believe any p-value; (5) the statsmodels or PyMC cell for the final, publication-style output. The companion

notebooks follow the same order as this PDF, so each chapter is one notebook you can copy and adapt.